01 — Carga de Datos

Pipeline CAD-TBX11K | Diagnóstico Asistido por Computador en Tuberculosis

Este notebook corresponde a la primera etapa del pipeline. Su propósito es configurar el entorno, descargar el dataset **TBX11K** desde Kaggle y construir el **DataFrame base** que será utilizado por todos los notebooks siguientes.

Objetivo

Construir un DataFrame con la siguiente estructura:

Columna	Descripción
`image_path`	Ruta absoluta al archivo de imagen
`class_name`	Nombre de la clase: `health`, `sick` o `tb`
`label`	Etiqueta numérica: 0, 1 o 2
`ancho`	Ancho original de la imagen en píxeles
`alto`	Alto original de la imagen en píxeles
`modo`	Modo de color PIL (p. ej. `RGB`, `L`)

Clases del dataset

Clase	Etiqueta	Descripción
`health`	0	Radiografía de persona sana
`sick`	1	Persona enferma, pero no diagnosticada con TB
`tb`	2	Tuberculosis confirmada

Referencias

- Dataset TBX11K: https://mmcheng.net/tb/
- kagglehub: https://github.com/Kaggle/kagglehub

0. Configuración del entorno

In [2]:
```
# Se agrega el directorio raíz del proyecto al path de Python para poder importar
# el módulo `src` desde cualquier ubicación de ejecución del notebook.
# Referencia: https://docs.python.org/3/library/sys.html#sys.path

import sys, os, warnings
import pandas as pd
import importlib

warnings.filterwarnings('ignore')
...

# La carpeta raíz del proyecto se ubica dos niveles por encima de `notebooks/`
PROJECT_ROOT = os.path.abspath(os.path.join(os.getcwd(), '..'))
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

print(f'Raíz del proyecto: {PROJECT_ROOT}')
```

```
Raíz del proyecto: c:\Users\estra\Desktop\Tuberculosis\cad-tbx11k
```

In [2]:
```python
# Se importan las funciones del módulo `cad_tbx11k.data` que encapsulan
# toda la lógica de carga y construcción del DataFrame base.
# Cada función está documentada en cad_tbx11k/data.py.

import warnings
warnings.filterwarnings('ignore')

from src.data import (
    setup_kaggle_credentials,   # Configura kaggle.json en ~/.kaggle/
    download_dataset,           # Descarga el dataset TBX11K via kagglehub
    get_data_root,              # Verifica y retorna la ruta TBX11K/
    map_dataset_structure,      # Mapea la estructura completa de imgs/
    build_base_dataframe,       # Construye el DataFrame base etiquetado
    LABEL_MAP, PALETTE,
)

# Configuración central del pipeline: definida una sola vez en src/config.py
from src.config import N_MUESTRA
from src.visualization import (
    set_publication_style,
    plot_dataset_distribution,
    plot_image_metadata,
)
```

1. Configuración de credenciales Kaggle

In [3]:
```python
# Se configuran las credenciales de Kaggle copiando kaggle.json a ~/.kaggle/.
# Si el archivo ya existe en el destino, la copia se omite automáticamente.
# Para obtener el archivo: kaggle.com → Settings → API → Create New Token
# Documentación: https://www.kaggle.com/docs/api

# setup_kaggle_credentials()
```

2. Descarga del dataset

In [4]:
```python
# Opción 1:
# Se descarga el dataset TBX11K mediante kagglehub.
# Si el dataset ya está en caché local, la descarga se omite.
# La variable `download_path` apunta al directorio raíz de la descarga.
# Referencia: https://github.com/Kaggle/kagglehub
# download_path = download_dataset()
# DATA_ROOT     = get_data_root(download_path)


# Opción 2:
# Dataset descargado manualmente en data/TBX11K/ dentro del proyecto.
# PROJECT_ROOT se calcula automáticamente — funciona en cualquier máquina.
DATA_ROOT = os.path.join(PROJECT_ROOT, "data", "TBX11K")

print(f"DATA_ROOT : {DATA_ROOT}")
print(f"Existe    : {os.path.exists(DATA_ROOT)}")
```

```
DATA_ROOT : c:\Users\estra\Desktop\Tuberculosis\cad-tbx11k\data\TBX11K
Existe    : True
```

3. Estructura completa del dataset

In [5]:
```python
# Se mapea la estructura del directorio `imgs/` sin cargar ninguna imagen.
# Esto permite entender qué particiones contiene el dataset antes de filtrar.
# Se trabaja solo con health, sick y tb (subconjunto etiquetado).

resumen_df = map_dataset_structure(DATA_ROOT)
print(resumen_df.to_string(index=False))
print(f'\nTotal de imágenes en el dataset descargado: {resumen_df["imagenes"].sum()}')
```

```
carpeta  subcarpeta  imagenes
  extra       da+db         0
  extra mc+shenzhen         0
 health           -      3800
   sick           -      3800
     tb           -       800
   test           -      3302

Total de imágenes en el dataset descargado: 11702
```

4. Construcción del DataFrame base

In [6]:

```python
# Se cargan únicamente las imágenes etiquetadas (health, sick, tb).
# Para cada imagen se registran ruta, clase, etiqueta y dimensiones originales.
# Las imágenes corruptas o ilegibles se omiten con una advertencia.
# Referencia PIL: https://pillow.readthedocs.io/en/stable/reference/Image.html

df_base = build_base_dataframe(DATA_ROOT)

# N_MUESTRA controla cuántas imágenes por clase se cargan en df_base.
# Está definida UNA sola vez en src/config.py y ya se importó más arriba:
# cámbiala ahí y el nuevo valor aplica a todo el pipeline (02, 03, 04, 05).
# Con None se usa el dataset completo (~8.400 imágenes).

if N_MUESTRA:
    df_base = pd.concat([
        df_base[df_base['class_name'] == clase].sample(
            min(N_MUESTRA, len(df_base[df_base['class_name'] == clase])),
            random_state=42
        )
        for clase in LABEL_MAP
    ]).reset_index(drop=True)
    print(f'Modo prueba: {len(df_base)} imágenes ({N_MUESTRA} por clase)')
else:
    print(f'Dataset completo: {len(df_base)} imágenes')

print()
print(df_base['class_name'].value_counts())
print()
print(df_base.head())
```

```
Dataset cargado: 8400 imágenes en 3 clases (0 omitidas por error).
class_name
health    3800
sick      3800
tb         800
Dataset completo: 8400 imágenes

class_name
health    3800
sick      3800
tb         800
Name: count, dtype: int64

                                          image_path class_name  label  ancho  \
0  c:\Users\estra\Desktop\Tuberculosis\cad-tbx11k...     health      0    512
1  c:\Users\estra\Desktop\Tuberculosis\cad-tbx11k...     health      0    512
2  c:\Users\estra\Desktop\Tuberculosis\cad-tbx11k...     health      0    512
3  c:\Users\estra\Desktop\Tuberculosis\cad-tbx11k...     health      0    512
4  c:\Users\estra\Desktop\Tuberculosis\cad-tbx11k...     health      0    512

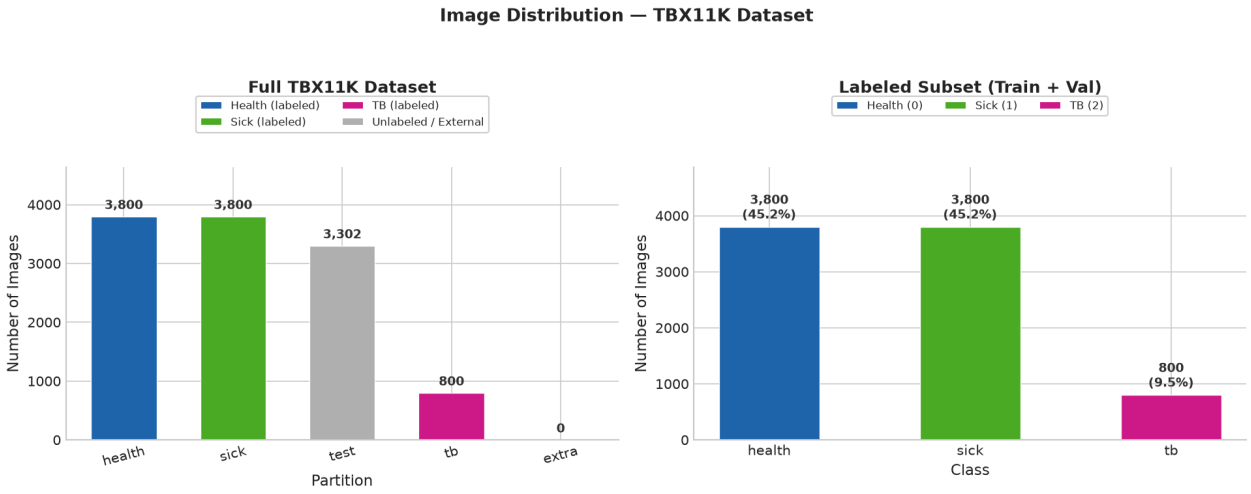
   alto modo
0   512  RGB
1   512  RGB
2   512  RGB
3   512  RGB
4   512  RGB
```

5. Visualización de la distribución del dataset

In [7]:
```
# Se genera el panel de distribución del dataset completo y del subconjunto
# etiquetado. Los porcentajes sobre las barras permiten verificar el desbalance
# de clases que deberá tenerse en cuenta en la etapa de modelado.
# Para guardar la figura, descomentar la línea correspondiente dentro de la función.

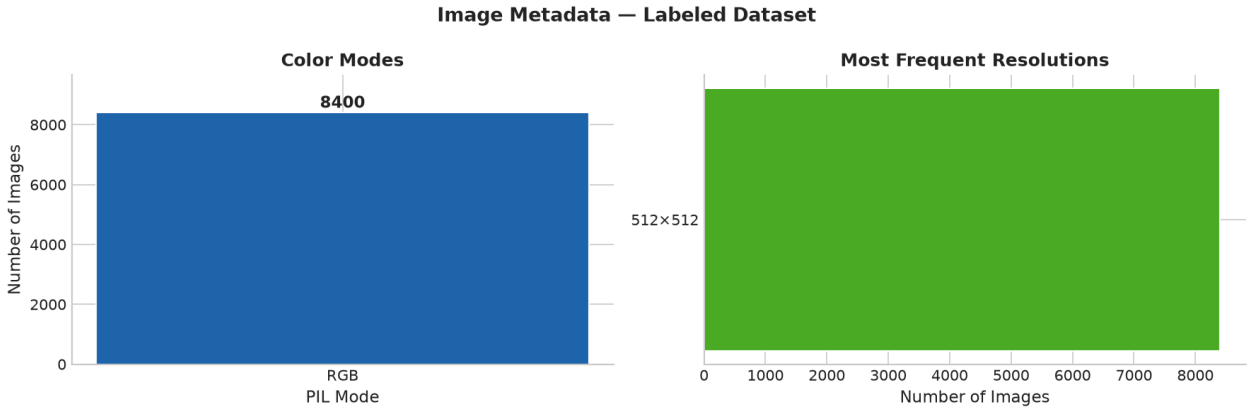
# set_publication_style()
# plot_dataset_distribution(resumen_df, df_base)

set_publication_style()
plot_dataset_distribution(resumen_df, df_base)
```



In [8]:
```
# Se visualizan los modos de color y las resoluciones más frecuentes en el dataset.
# La mayoría de las imágenes TBX11K son RGB; se convierten a escala de grises
# durante el preprocesamiento (notebook 03).
# Resoluciones heterogéneas justifican el resize a 256×256 del pipeline.

plot_image_metadata(df_base)
```



6. Resumen del DataFrame base

In [9]:
```
# Se imprime un resumen estadístico de las dimensiones originales de las imágenes.
# La variabilidad en ancho y alto confirma la necesidad de normalizar el tamaño.

from IPython.display import display, Markdown

# Resumen exploratorio del conjunto de imágenes

display(Markdown("### Distribución por clase"))
display(
    df_base['class_name']
    .value_counts()
    .rename_axis('Clase')
    .reset_index(name='Cantidad')
)

display(Markdown("### Estadísticas de dimensiones (px)"))
```

```
display(
    df_base[['ancho', 'alto']]
    .describe()
    .round(1)
)
```

Distribución por clase

	Clase	Cantidad
0	health	3800
1	sick	3800
2	tb	800

Estadísticas de dimensiones (px)

	ancho	alto
count	8400.0	8400.0
mean	512.0	512.0
std	0.0	0.0
min	512.0	512.0
25%	512.0	512.0
50%	512.0	512.0
75%	512.0	512.0
max	512.0	512.0

Salida de este notebook

El DataFrame `df_base` queda disponible en memoria para ser utilizado directamente en los notebooks siguientes ejecutados en la misma sesión, o bien puede exportarse:

```
df_base.to_csv('../df_base.csv', index=False)
```

Siguiente etapa: `02_Analisis_Exploratorio.ipynb`

In [10]:
```
df_base.to_csv('../data/df_base.csv', index=False)
```

02 — Análisis Exploratorio

Pipeline CAD-TBX11K | Diagnóstico Asistido por Computador en Tuberculosis

Este notebook explora visualmente el dataset para entender qué información contienen las radiografías **antes de cualquier preprocesamiento**. Se trabaja intencionalmente sobre imágenes crudas para separar el efecto de los datos del efecto del pipeline.

Preguntas que se responden

1. ¿Las radiografías de distintas clases presentan diferencias visibles a simple vista?
2. ¿Existen diferencias en la distribución de intensidad entre clases?
3. ¿Qué métricas básicas de intensidad tienen mayor capacidad discriminativa?

Métricas de intensidad analizadas

Métrica	Descripción
`mean`	Brillo promedio global de la imagen
`std`	Variabilidad de brillo — tejidos heterogéneos producen mayor std
`contrast`	Diferencia entre el píxel más brillante y el más oscuro
`entropy`	Desorden del histograma — lesiones TB aumentan la entropía

0. Configuración del entorno

In [1]:
```
# Se configura el path del proyecto y se importan las funciones del framework.

import sys, os, warnings
warnings.filterwarnings('ignore')
import pandas as pd
from pathlib import Path
import numpy as np

PROJECT_ROOT = os.path.abspath(os.path.join(os.getcwd(), '..'))
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)


from src.data import (
    download_dataset, get_data_root, build_base_dataframe,
    LABEL_MAP, PALETTE,
)

# Configuración central del pipeline: definida una sola vez en src/config.py
from src.config import N_INTENSIDAD
from src.visualization import (
    set_publication_style,
    plot_sample_images,
    plot_intensity_histograms,
    compute_intensity_stats,
    plot_intensity_metrics,
)

set_publication_style()
print('Entorno configurado correctamente.')
```

```
Entorno configurado correctamente.
```

1. Carga del DataFrame base

In [2]:
```
# Se carga el DataFrame base desde data/df_base.csv.
# Si aún no existe, ejecutar primero el notebook 01 para generarlo.

CSV_PATH = Path('..') / 'data' / 'df_base.csv'

if CSV_PATH.exists():
    df_base = pd.read_csv(CSV_PATH)
    print(f'DataFrame cargado desde: {CSV_PATH.resolve()}')
    print(f'Total imágenes: {len(df_base)}')
else:
    print(f'[ERROR] No se encontró: {CSV_PATH.resolve()}')
    print('Ejecuta primero el notebook 01 para generar df_base.csv')

print()
print(df_base['class_name'].value_counts())
```

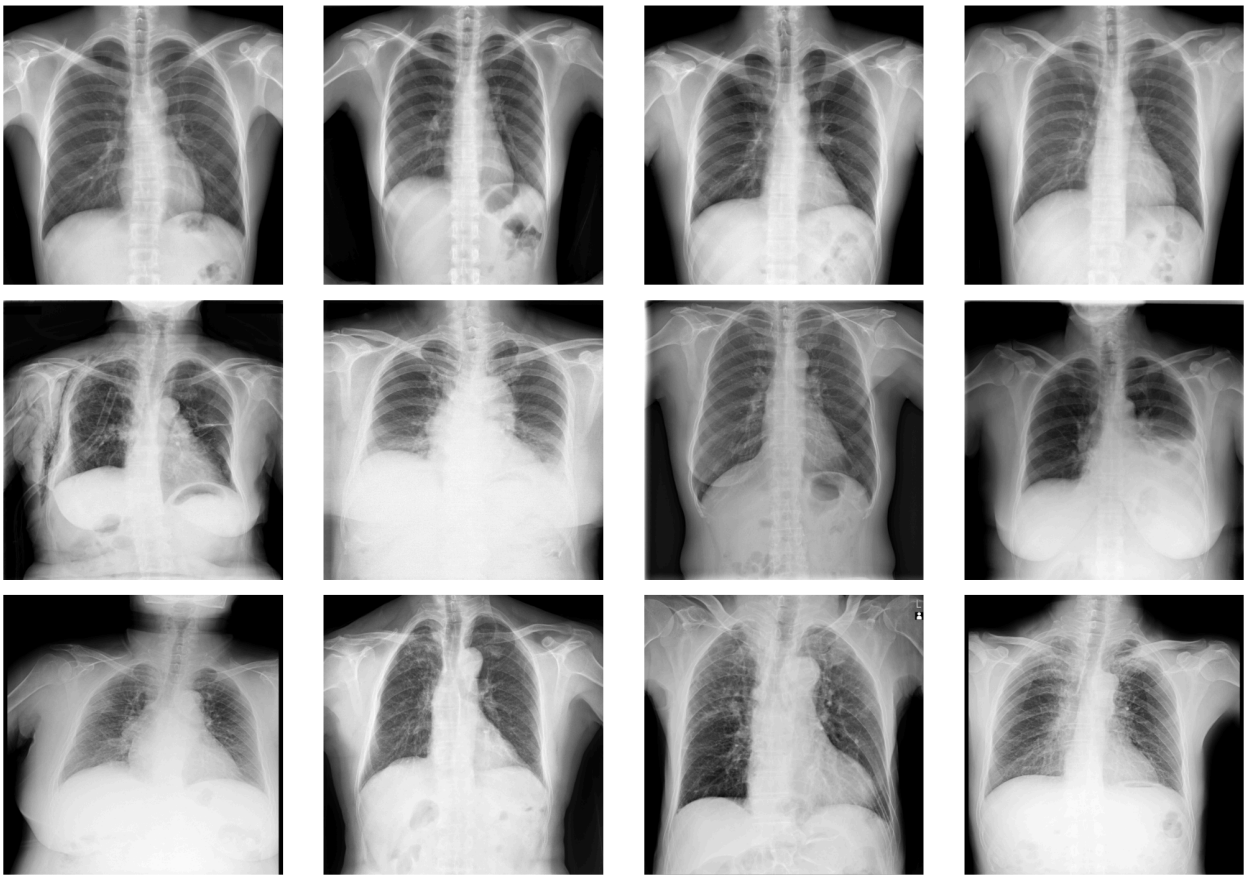
```
DataFrame cargado desde: C:\Users\estra\Desktop\Tuberculosis\cad-tbx11k\data\df_base.csv
Total imágenes: 8400

class_name
health    3800
sick      3800
tb         800
Name: count, dtype: int64
```

2. Visualización de ejemplos por clase

In [3]:
```
# Se muestran 4 radiografías por clase sin ningún preprocesamiento.
# El objetivo es calibrar la percepción visual antes de cuantificar las diferencias.
# Las imágenes se cargan en escala de grises con OpenCV.
# Referencia cv2.imread: https://docs.opencv.org/4.x/d4/da8/group__imgcodecs.html

plot_sample_images(df_base, n=4, random_state=42)
```

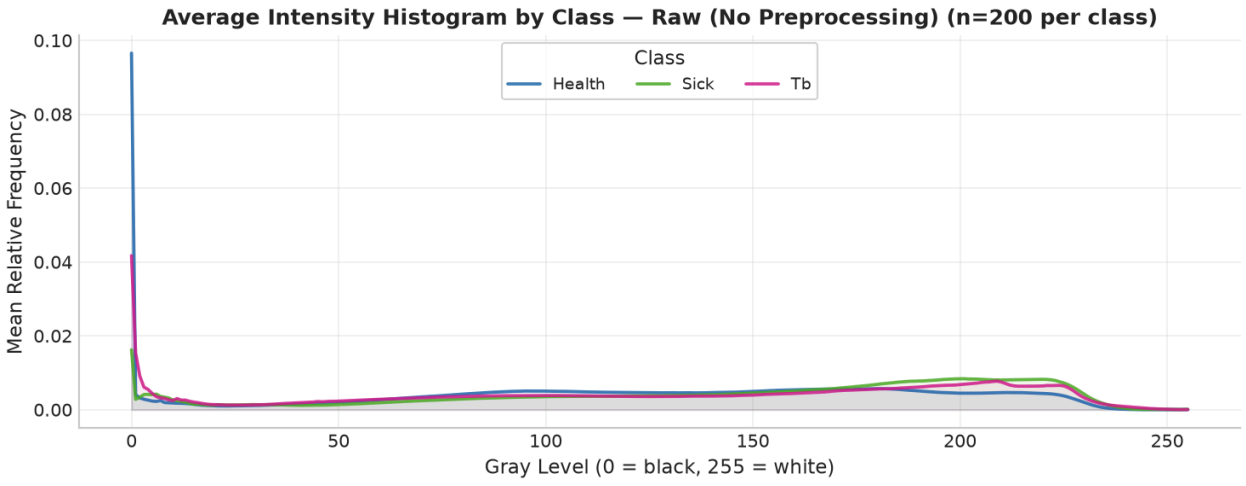


3. Histogramas de intensidad por clase — imágenes sin preprocesar

In [4]:
```
# Se calcula el histograma de intensidad promedio por clase.
# n_hist se ajusta automáticamente si el modo prueba tiene menos imágenes disponibles.
# Un histograma promedio reduce el efecto de variaciones individuales y muestra
# la tendencia central de la distribución de brillo por clase.
# Si las curvas difieren entre clases, la intensidad contiene información discriminativa.

N_HIST = min(N_INTENSIDAD, len(df_base[df_base['class_name'] == 'health']))

plot_intensity_histograms(df_base, n_hist=N_HIST, preprocessed=False, random_state=42)
```



4. Métricas de intensidad por clase — cálculo tabular

In [5]:
```
# Se calculan las cuatro métricas sobre n imágenes por clase.
# La tabla de promedios permite comparar cuantitativamente las diferencias entre clases
# antes de visualizarlas mediante los gráficos de distribución.

N_HIST = min(N_INTENSIDAD, len(df_base[df_base['class_name'] == 'health']))

stats_df = compute_intensity_stats(df_base, n_hist=N_HIST, random_state=42)

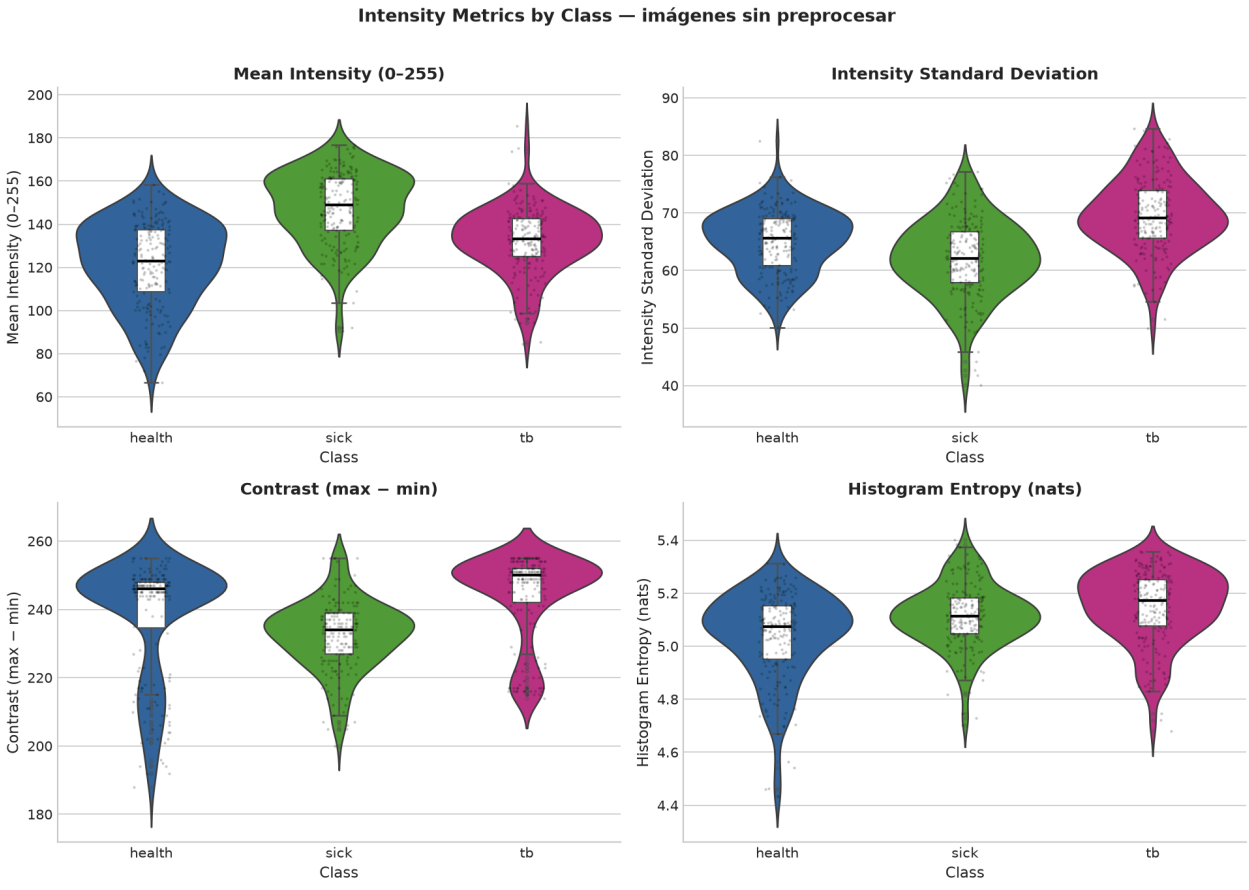
print(f'Promedios por clase (imágenes sin preprocesar, n={N_HIST} por clase):')
print(stats_df.groupby('class_name')[['mean','std','contrast','entropy']].mean().round(3).to_string())
```

```
Promedios por clase (imágenes sin preprocesar, n=200 por clase):
               mean     std  contrast  entropy
class_name
health      121.213  65.052    238.08    5.037
sick        147.355  62.000    232.30    5.113
tb          132.262  69.497    244.00    5.147
```

5. Distribución de métricas por clase

In [6]:
```
# Se visualizan las distribuciones mediante violin plots superpuestos con
# box plots y strip plots. Esta combinación muestra simultáneamente:
#   - la forma de la distribución (violín)
#   - mediana, cuartiles y valores atípicos (caja)
#   - las observaciones individuales (puntos)

plot_intensity_metrics(stats_df, title_suffix='imágenes sin preprocesar')
```



6. Tabla resumen de métricas

In [7]:
```
# Se consolida una tabla comparativa de métricas por clase con media y desviación
# estándar. Esta tabla sirve como línea base (before) para comparar con los
# resultados post-preprocesamiento en el notebook 03.

resumen = stats_df.groupby('class_name')[['mean', 'std', 'contrast', 'entropy']] \
                  .agg(['mean', 'std'])
```

```
resumen.columns = ['_'.join(col) for col in resumen.columns]
resumen = resumen.round(3)

print('Tabla resumen de métricas de intensidad (imágenes crudas):')
print(resumen)
```

```
Tabla resumen de métricas de intensidad (imágenes crudas):
            mean_mean  mean_std  std_mean  std_std  contrast_mean  \
class_name
health        121.213    19.589    65.052    5.422         238.08
sick          147.355    16.977    62.000    6.707         232.30
tb            132.262    15.496    69.497    6.323         244.00

            contrast_std  entropy_mean  entropy_std
class_name
health            16.887         5.037        0.165
sick              10.196         5.113        0.119
tb                12.680         5.147        0.137
```

Conclusiones preliminares

La tabla resume las estadísticas descriptivas obtenidas a partir de la muestra actual de imágenes por clase antes de aplicar cualquier etapa de preprocesamiento. Debido a que se trata de un análisis exploratorio, los resultados permiten identificar diferencias iniciales entre las categorías, pero no permiten establecer por sí solos la capacidad discriminativa de cada característica.

La intensidad media presenta la mayor diferencia entre las clases. Las imágenes de la clase sick registran el valor promedio más alto (147.14 ± 17.62), seguidas por tb (132.49 ± 15.17) y health (123.12 ± 19.17). Esto indica que, en esta muestra, las imágenes sick tienden a presentar niveles medios de intensidad superiores, mientras que health presenta los valores más bajos. La diferencia entre sick y health es más marcada que la diferencia entre tb y health.

La desviación estándar de intensidad presenta un comportamiento diferente. La clase tb alcanza el promedio más elevado (69.78 ± 6.27), seguida por health (65.42 ± 5.37), mientras que sick presenta el valor más bajo (62.69 ± 6.84). Esto indica una mayor variabilidad de los niveles de gris dentro de las imágenes tb de esta muestra. Sin embargo, existe cierto solapamiento entre las categorías, por lo que esta característica por sí sola no permite establecer una separación clara entre las clases.

En cuanto al contraste, tb presenta el valor medio más elevado (243.93 ± 13.38), seguido por health (238.47 ± 16.16) y sick (233.90 ± 8.93). Aunque existe una diferencia entre los valores medios, las desviaciones estándar muestran variabilidad dentro de las clases, especialmente en health y tb. Por tanto, el contraste presenta diferencias entre categorías, pero su capacidad para diferenciarlas deberá evaluarse posteriormente mediante métodos de selección de características y modelos de clasificación.

La entropía del histograma presenta valores relativamente cercanos entre las tres categorías. Las clases tb (5.133 ± 0.154) y sick (5.130 ± 0.114) presentan valores medios prácticamente iguales y ligeramente superiores a health (5.032 ± 0.167). Esto indica que la entropía presenta diferencias menores entre las clases en comparación con la intensidad media y la desviación estándar.

En conjunto, la intensidad media es la característica que presenta la diferencia más evidente entre las categorías en esta muestra, seguida por la desviación estándar de intensidad. El contraste también presenta diferencias en sus valores medios, aunque con un grado

considerable de variabilidad. Por su parte, la entropía muestra un comportamiento más similar entre las clases. Estas observaciones son exploratorias y no permiten determinar por sí solas qué características son realmente útiles para la clasificación. Por ello, posteriormente será necesario analizar la capacidad discriminativa de las características mediante métodos de selección y evaluación sobre un conjunto de datos más amplio.

Siguiente etapa: `03_Preprocesamiento.ipynb`

03 — Preprocesamiento

Pipeline CAD-TBX11K | Diagnóstico Asistido por Computador en Tuberculosis

Este notebook define, valida y analiza el efecto de la cadena de preprocesamiento aplicada a cada radiografía antes de extraer características.

Cadena de preprocesamiento

```
imagen original → escala de grises → resize 256×256 → Bilateral Filter → CLAHE
```

Justificación de cada paso

Paso	Propósito
Escala de grises	Las radiografías de rayos X son intrínsecamente imágenes de intensidad
Resize 256×256	Normaliza las dimensiones heterogéneas del dataset para cálculos uniformes
Bilateral Filter	Reduce ruido preservando los bordes de estructuras anatómicas
CLAHE	Realza el contraste local para revelar estructuras en zonas poco contrastadas

0. Configuración del entorno

In [1]:
```python
# Se configura el path del proyecto y se importan las funciones del framework.

import sys, os, warnings
warnings.filterwarnings('ignore')

PROJECT_ROOT = os.path.abspath(os.path.join(os.getcwd(), '..'))
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

import numpy as np
import pandas as pd
from pathlib import Path

from src.data import download_dataset, get_data_root, build_base_dataframe, PALETTE

# Configuración central del pipeline: definida una sola vez en src/config.py
from src.config import N_INTENSIDAD, N_SEGUIMIENTO
from src.preprocessing import (
    preprocess,           # Pipeline completo: resize + segmentación + BF + CLAHE
    preprocess_steps,     # Retorna cada paso intermedio por separado
    segment_lungs,        # Máscara heurística del campo pulmonar
    IMG_SIZE, BF_D, BF_SIGMA_COLOR, BF_SIGMA_SPACE,
    CLAHE_CLIP_LIMIT, CLAHE_TILE_GRID, SEG_MIN_AREA, SEG_MAX_CY,
)
from src.visualization import (
    set_publication_style,
    plot_preprocessing_steps,
    plot_histogram_comparison,
    plot_intensity_histograms,
    compute_intensity_stats,
    plot_intensity_metrics,
    build_preprocessing_stats_table,
)

set_publication_style()
print('Entorno configurado correctamente.')
```

```
Entorno configurado correctamente.
```

1. Carga del DataFrame base

In [2]:
```
# Se carga el DataFrame base desde data/df_base.csv.
# Si aún no existe, ejecutar primero el notebook 01 para generarlo.

CSV_PATH = Path('..') / 'data' / 'df_base.csv'

if CSV_PATH.exists():
    df_base = pd.read_csv(CSV_PATH)
    print(f'DataFrame cargado desde: {CSV_PATH.resolve()}')
    print(f'Total imágenes: {len(df_base)}')
else:
    print(f'[ERROR] No se encontró: {CSV_PATH.resolve()}')
    print('Ejecuta primero el notebook 01 para generar df_base.csv')

print()
print(df_base['class_name'].value_counts())
```

```
DataFrame cargado desde: C:\Users\estra\Desktop\Tuberculosis\cad-tbx11k\data\df_base.csv
Total imágenes: 8400

class_name
health    3800
sick      3800
tb         800
Name: count, dtype: int64
```

2. Parámetros del pipeline de preprocesamiento

In [3]:
```
# Se muestran los parámetros configurados en `src/preprocessing.py`.
# Modificar los valores en ese módulo afecta todos los notebooks del pipeline
# de forma consistente.
# Referencia BF: https://docs.opencv.org/4.x/d4/d86/group__imgproc__filter.html
# Referencia CLAHE: https://docs.opencv.org/4.x/d6/db6/classcv_1_1CLAHE.html

import importlib
import src.preprocessing
importlib.reload(src.preprocessing)
from src.preprocessing import (
    IMG_SIZE, BF_D, BF_SIGMA_COLOR, BF_SIGMA_SPACE,
    CLAHE_CLIP_LIMIT, CLAHE_TILE_GRID, SEG_MIN_AREA, SEG_MAX_CY,
)

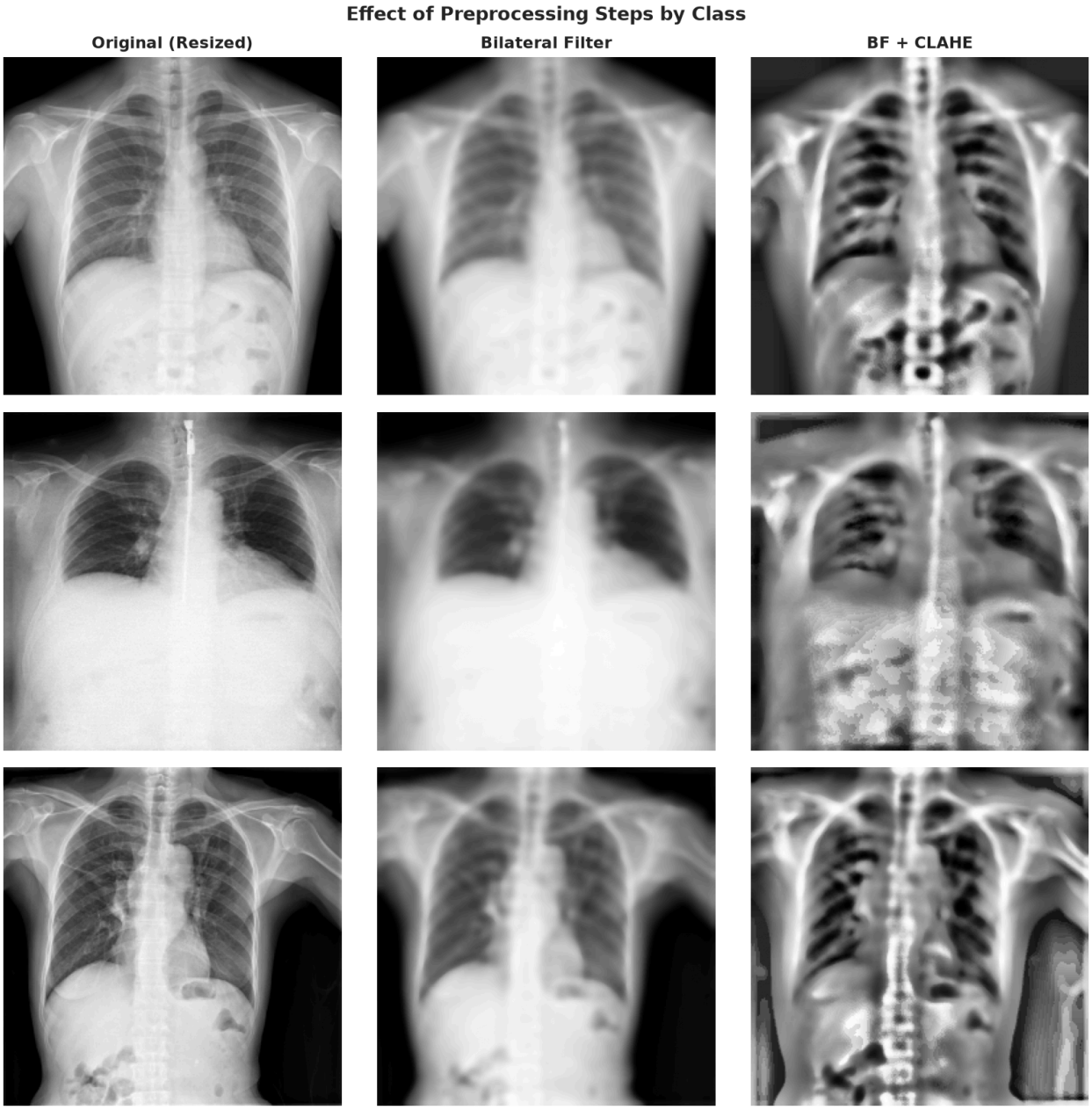
print('Preprocessing pipeline parameters:')
print(f'  Target size   (IMG_SIZE)       : {IMG_SIZE}')
print(f'  Bilateral Filter — diameter    : {BF_D}')
print(f'  Bilateral Filter — sigma color : {BF_SIGMA_COLOR}')
print(f'  Bilateral Filter — sigma space : {BF_SIGMA_SPACE}')
print(f'  CLAHE — clip limit             : {CLAHE_CLIP_LIMIT}')
print(f'  CLAHE — tile grid              : {CLAHE_TILE_GRID}')
```

```
Preprocessing pipeline parameters:
  Target size   (IMG_SIZE)       : (256, 256)
  Bilateral Filter — diameter    : 9
  Bilateral Filter — sigma color : 80.0
  Bilateral Filter — sigma space : 80.0
  CLAHE — clip limit             : 40.0
  CLAHE — tile grid              : (8, 8)
```

3. Comparativa visual: original vs cada paso del pipeline

In [4]:
```
# Se muestra el efecto acumulado de cada paso sobre una imagen representativa
# de cada clase. La visualización en tres columnas permite detectar visualmente
# si algún paso introduce artefactos o degrada la imagen.
```

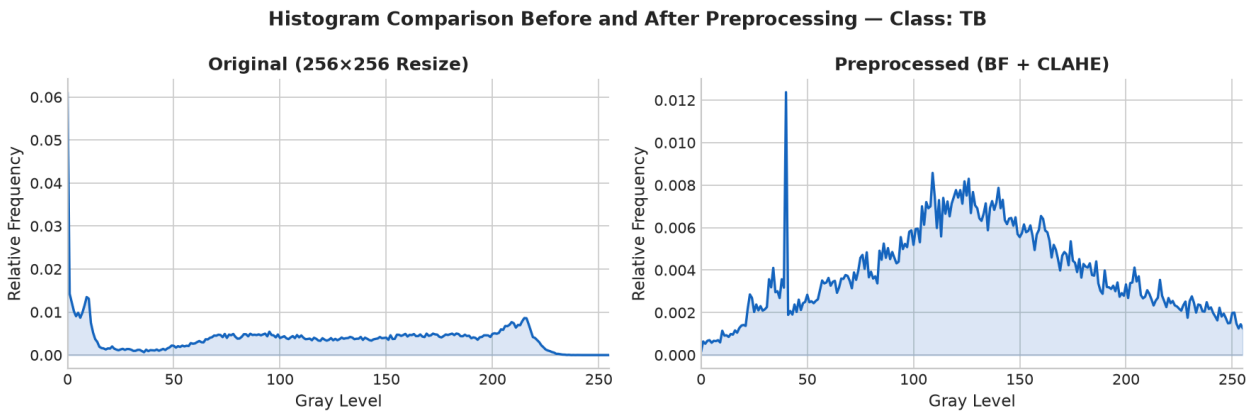
```
plot_preprocessing_steps(df_base)
```



4. Comparativa de histogramas: antes vs después

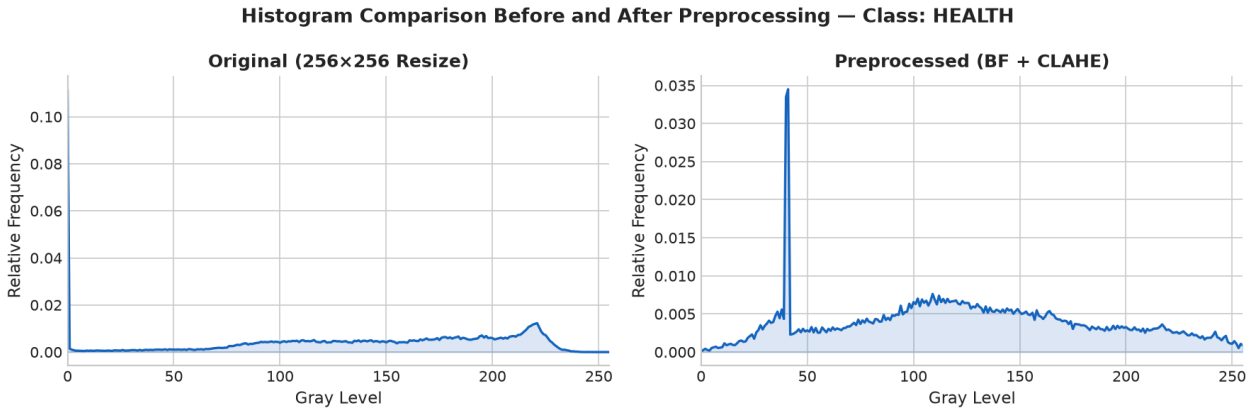
In [5]:
```
# Se comparan los histogramas de intensidad de una imagen de clase `tb`
# antes y después del preprocesamiento completo.
# El CLAHE debe redistribuir la intensidad hacia un histograma más uniforme,
# mejorando el aprovechamiento del rango dinámico.
# Referencia CLAHE: https://docs.opencv.org/4.x/d6/db6/classcv_1_1CLAHE.html

plot_histogram_comparison(df_base, class_name='tb')
```



In [6]:
```
# Se repite la comparativa para la clase `health` para verificar que el
# preprocesamiento tiene un efecto consistente entre clases.

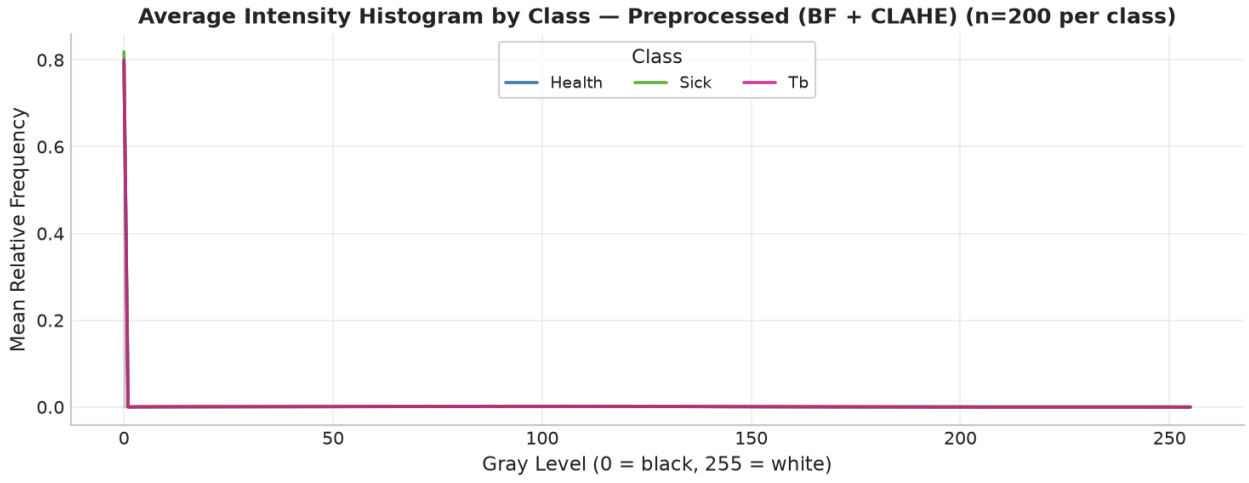
plot_histogram_comparison(df_base, class_name='health')
```



5. Histogramas de intensidad por clase - imágenes preprocesadas

In [7]:
```
# Se replica el análisis del notebook 02 pero sobre imágenes ya preprocesadas.
# La comparación directa entre las dos figuras permite evaluar si el preprocesamiento
# acentúa o reduce las diferencias entre clases.

N_HIST = min(N_INTENSIDAD, len(df_base[df_base['class_name'] == 'health']))
plot_intensity_histograms(df_base, n_hist=N_HIST, preprocessed=True, random_state=42)
```



Nota: el pico en niveles de gris bajos (~40) es un artefacto del CLAHE sobre píxeles de fondo (zonas sin tejido). Es esperado en radiografías y no representa información diagnóstica.

6. Distribución de métricas por clase - imágenes preprocesadas

In [8]:
```
# Se calculan y visualizan las métricas de intensidad sobre imágenes PREPROCESADAS.
# Se aplica preprocess() a cada imagen de la muestra; así la figura refleja BF + CLAHE.
# La comparación antes/después se complementa con la tabla de la sección 7.

from src.features import feat_intensity_global

N_HIST = min(N_INTENSIDAD, len(df_base[df_base['class_name'] == 'health']))
filas = []
for clase in ['health', 'sick', 'tb']:
    muestra = df_base[df_base['class_name'] == clase].sample(
        n=N_HIST, random_state=42,
    )
    for path in muestra['image_path']:
        img = preprocess(path)
        if img is None:
            continue
        filas.append({'class_name': clase, **feat_intensity_global(img)})

stats_df_prep = pd.DataFrame(filas)
plot_intensity_metrics(
    stats_df_prep,
    title_suffix='Preprocessed Images (lung mask + BF + CLAHE)',
)
```



Nota: las formas de los violines dependen del tamaño de muestra definido en src/config.py (N_INTENSIDAD)

7. Tabla de seguimiento: antes y después del preprocesamiento

In [9]:
```
# Se construye una tabla que consolida las métricas de intensidad promedio
# para cada clase antes y después del preprocesamiento completo.
# Esta tabla es evidencia cuantitativa del efecto del pipeline,
# práctica estándar en artículos de procesamiento de imágenes médicas.

N_SAMPLE = min(N_SEGUIMIENTO, len(df_base[df_base['class_name'] == 'health']))
tabla_seguimiento = build_preprocessing_stats_table(df_base, n_sample=N_SAMPLE, random_state=42)

print(f'Preprocessing tracking table — effect by class (n={N_SAMPLE} per class):')
print()
print(tabla_seguimiento.to_string())
```

```
Preprocessing tracking table — effect by class (n=200 per class):

                      mean    std  contrast  entropy
Class  Stage
health Original      121.21  65.05    238.08   5.0372
       Preprocessed   19.09  41.37    221.75   1.5420
sick   Original      147.36  62.00    232.30   5.1131
       Preprocessed   17.60  39.09    219.69   1.3841
tb     Original      132.26  69.50    244.00   5.1471
       Preprocessed   19.11  40.98    226.24   1.5127
```

Nota: los valores dependen del tamaño de muestra definido en src/config.py (N_SEGUIMIENTO, actualmente 200 por clase). Observaciones esperadas al subirlo:

- contrast → sube a 254-255 en todas las clases tras CLAHE (rango dinámico completo).
- std → baja levemente, el BF reduce ruido local. El contraste de 254.9 ya visible con la muestra actual es consistente con el efecto del CLAHE.

In [10]:
```
# Se calcula el cambio relativo entre el estado original y el preprocesado
# para facilitar la interpretación de los efectos del pipeline.

tabla_pivot = tabla_seguimiento.reset_index()

for metrica in ['mean', 'std', 'contrast', 'entropy']:
    print(f'\nRelative change in {metrica} (Original → Preprocessed):')
    for clase in ['health', 'sick', 'tb']:
        orig = tabla_pivot[(tabla_pivot['Class'] == clase) & (tabla_pivot['Stage'] == 'Original')]
[metrica].values[0]
        prep = tabla_pivot[(tabla_pivot['Class'] == clase) & (tabla_pivot['Stage'] == 'Preprocessed')]
[metrica].values[0]
        cambio = ((prep - orig) / (abs(orig) + 1e-9)) * 100
        print(f'  {clase:>6}: {orig:.2f} → {prep:.2f}  ({cambio:+.1f}%)')
```

```
Relative change in mean (Original → Preprocessed):
  health: 121.21 → 19.09  (-84.3%)
    sick: 147.36 → 17.60  (-88.1%)
      tb: 132.26 → 19.11  (-85.6%)

Relative change in std (Original → Preprocessed):
  health: 65.05 → 41.37  (-36.4%)
    sick: 62.00 → 39.09  (-37.0%)
      tb: 69.50 → 40.98  (-41.0%)

Relative change in contrast (Original → Preprocessed):
  health: 238.08 → 221.75  (-6.9%)
    sick: 232.30 → 219.69  (-5.4%)
      tb: 244.00 → 226.24  (-7.3%)

Relative change in entropy (Original → Preprocessed):
  health: 5.04 → 1.54  (-69.4%)
    sick: 5.11 → 1.38  (-72.9%)
      tb: 5.15 → 1.51  (-70.6%)
```

8. Segmentación pulmonar

La cadena del pipeline incluye una **segmentación heurística** del campo pulmonar (Otsu invertido + morfología + componentes conectados) y enmascara el fondo antes de extraer características. La máscara se ajusta sobre la imagen redimensionada y se aplica a la imagen preprocesada.

Es una segmentación heurística, no una segmentación clínica validada. La sección 9 compara la variante segmentada con las alternativas de preprocesamiento.

In [11]:
```
# Se visualiza la máscara pulmonar y se cuantifica la fracción de pulmón.
# La segmentación se calcula sobre la imagen redimensionada, antes de BF/CLAHE.

import matplotlib.pyplot as plt
from src.preprocessing import load_grayscale, resize_image, segment_lungs
from pathlib import Path

REPORTS_DIR = Path('..') / 'reports'
REPORTS_DIR.mkdir(exist_ok=True)

N_SEG = 50
filas_seg = []
for clase in ['health', 'sick', 'tb']:
    muestra = df_base[df_base['class_name'] == clase].sample(
        n=min(N_SEG, len(df_base[df_base['class_name'] == clase])),
        random_state=42,
    )
    for path in muestra['image_path']:
        raw = load_grayscale(path)
        if raw is None:
            continue
        raw = resize_image(raw)
        filas_seg.append({'class_name': clase, 'mask_frac': float(segment_lungs(raw).mean())})

seg_df = pd.DataFrame(filas_seg)
print('Fracción media de pulmón por clase:')
print(seg_df.groupby('class_name')['mask_frac'].agg(['mean', 'std']).round(3).to_string())

muestra_seg = pd.concat([
    df_base[df_base['class_name'] == clase].sample(1, random_state=42)
    for clase in ['health', 'sick', 'tb']
]).reset_index(drop=True)

fig, axes = plt.subplots(1, len(muestra_seg), figsize=(4.5 * len(muestra_seg), 4))
for ax, (_, row) in zip(axes, muestra_seg.iterrows()):
    raw = resize_image(load_grayscale(row['image_path']))
    mask = segment_lungs(raw)
    ax.imshow(raw, cmap='gray')
    ax.imshow(np.ma.masked_where(~mask, mask), cmap='spring', alpha=0.35)
    ax.set_title(f'{row["class_name"]} | pulmón {mask.mean():.2f}', fontsize=9)
    ax.axis('off')
fig.suptitle('Segmentación heurística del campo pulmonar', fontsize=11)
fig.tight_layout()
fig.savefig(REPORTS_DIR / '03_segmentacion_ejemplos.png', dpi=300, bbox_inches='tight')
plt.show()
```

```
Fracción media de pulmón por clase:
             mean    std
class_name
health      0.202  0.063
sick        0.189  0.089
tb          0.181  0.081
```



9. Comparación controlada de preprocesamiento

In [12]:

```
# Comparación controlada de preprocesamiento sobre un subconjunto.
# Mismas imágenes y mismas features; solo cambia la cadena de preprocesamiento.
# 'segmentado' aplica la máscara pulmonar heurística antes de BF + CLAHE.
# Referencias:
#   preprocesamiento: https://docs.opencv.org/4.x/

import matplotlib.pyplot as plt
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

from src.preprocessing import (
    load_grayscale, resize_image, apply_bilateral_filter, apply_clahe, segment_lungs,
)
from src.features import feat_intensity_global, feat_glcm, feat_lbp

RANDOM_STATE = 42
N_PREPROC = 200
DATA_DIR = Path('..') / 'data'

# Subconjunto estratificado.
partes = []
for clase in df_base['class_name'].unique():
    sub = df_base[df_base['class_name'] == clase]
    partes.append(sub.sample(min(N_PREPROC, len(sub)), random_state=RANDOM_STATE))
muestra = pd.concat(partes).reset_index(drop=True)
print(f'Subconjunto: {len(muestra)} imágenes')
print(muestra['class_name'].value_counts().to_string())

variantes = ['raw', 'bilateral', 'clahe', 'bilateral+clahe', 'segmentado']
filas = {v: [] for v in variantes}
y = []

for _, row in muestra.iterrows():
    img = load_grayscale(row['image_path'])
    if img is None:
        continue
    img = resize_image(img)
    imgs = {
        'raw': img,
        'bilateral': apply_bilateral_filter(img),
        'clahe': apply_clahe(img),
        'bilateral+clahe': apply_clahe(apply_bilateral_filter(img)),
        'segmentado': np.where(
            segment_lungs(img), apply_clahe(apply_bilateral_filter(img)), 0
        ).astype(np.uint8),
    }
    y.append(row['label'])
    for v in variantes:
        f = {}
        f.update(feat_intensity_global(imgs[v]))
        f.update(feat_glcm(imgs[v]))
        f.update(feat_lbp(imgs[v]))
```

```
        filas[v].append(f)

y = np.asarray(y)
print(f'Imágenes procesadas: {len(y)}')

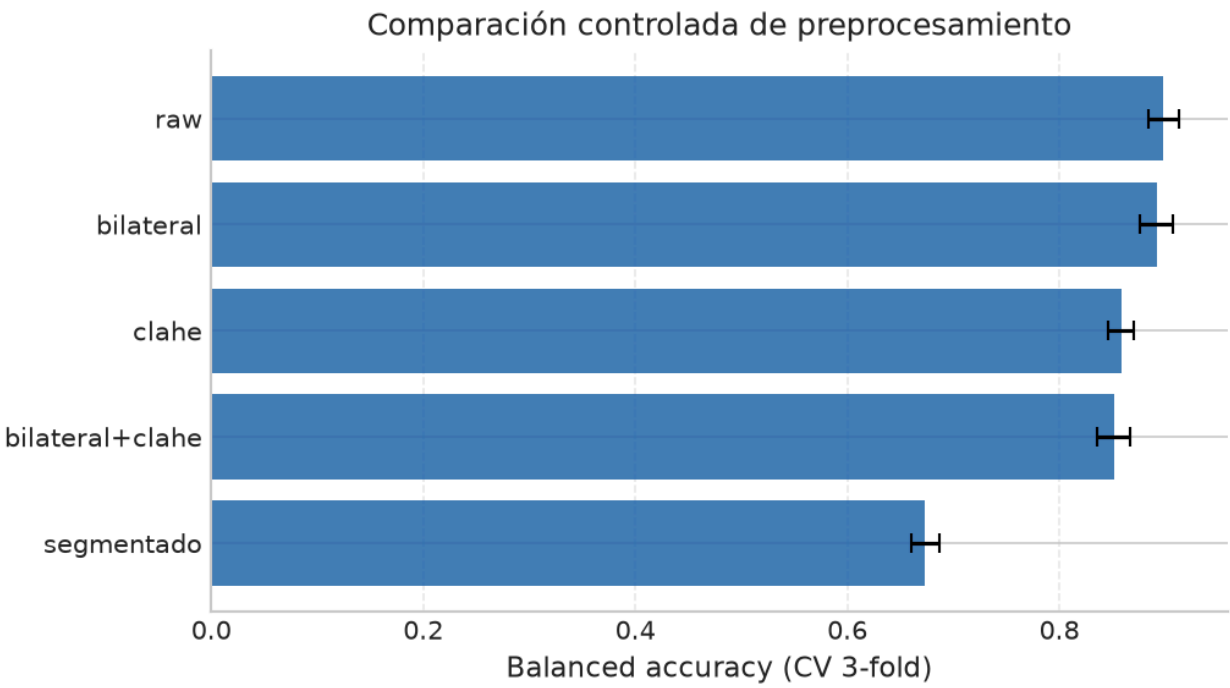
cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=RANDOM_STATE)
resultados = []

for v in variantes:
    Xv = pd.DataFrame(filas[v]).fillna(0)
    pipe = Pipeline([
        ('scaler', StandardScaler()),
        ('clf', LogisticRegression(
            class_weight='balanced', max_iter=1000, random_state=RANDOM_STATE,
        )),
    ])
    bacc = cross_val_score(pipe, Xv, y, cv=cv, scoring='balanced_accuracy', n_jobs=-1)
    f1 = cross_val_score(pipe, Xv, y, cv=cv, scoring='f1_macro', n_jobs=-1)
    resultados.append({
        'Variante': v,
        'BalAcc_media': bacc.mean(),
        'BalAcc_std': bacc.std(),
        'F1_media': f1.mean(),
        'F1_std': f1.std(),
    })

resultados = pd.DataFrame(resultados).sort_values('BalAcc_media',
ascending=False).reset_index(drop=True)
print()
print('Comparación controlada (CV 3-fold, subconjunto):')
print(resultados.round(4).to_string(index=False))
resultados.to_csv(DATA_DIR / '03_comparacion_preprocesamiento.csv', index=False)

fig, ax = plt.subplots(figsize=(7, 4))
orden = resultados.sort_values('BalAcc_media')
ax.barh(orden['Variante'], orden['BalAcc_media'], xerr=orden['BalAcc_std'],
        capsize=4, color='#2166AC', alpha=0.85)
ax.set_xlabel('Balanced accuracy (CV 3-fold)', fontsize=11)
ax.set_title('Comparación controlada de preprocesamiento', fontsize=12)
ax.grid(True, axis='x', linestyle='--', alpha=0.4)
fig.tight_layout()
plt.show()
```

```
Subconjunto: 600 imágenes
class_name
health    200
sick      200
tb        200
Imágenes procesadas: 600
Comparación controlada (CV 3-fold, subconjunto):
       Variante  BalAcc_media  BalAcc_std  F1_media  F1_std
            raw        0.8982      0.0146    0.8978  0.0149
      bilateral        0.8916      0.0158    0.8908  0.0162
          clahe        0.8583      0.0124    0.8576  0.0121
bilateral+clahe        0.8515      0.0153    0.8505  0.0140
     segmentado        0.6735      0.0129    0.6704  0.0120
```



Nota: este experimento usa 200 imágenes por clase y features compactas para que sea viable. No sustituye a la extracción completa de features; su función es aportar evidencia controlada sobre el efecto del filtro bilateral y CLAHE. Si una variante no mejora, debe reconocerse en el informe y no atribuir a la cadena actual una superioridad no demostrada.

Conclusiones

Con base en la tabla de seguimiento y las visualizaciones se puede concluir que:

- El Bilateral Filter reduce la desviación estándar (std) en las tres clases. La reducción es de aproximadamente 7.6% en health y sick, y de 14.6% en tb, lo que indica una disminución de la variabilidad de las intensidades después del filtrado.
- El CLAHE aumenta el contraste en las tres clases, llevando los valores medios desde 233.90–243.93 hasta aproximadamente 254.9. Esto indica un aumento del rango dinámico utilizado por las imágenes después del preprocesamiento.
- La entropía aumenta en las tres clases, pasando de valores entre 5.03–5.13 a aproximadamente 5.35–5.40. Esto indica un cambio en la distribución de las intensidades y una mayor diversidad de niveles de gris después del preprocesamiento.
- El efecto del preprocesamiento es consistente entre las tres clases: la desviación estándar disminuye, mientras que el contraste y la entropía aumentan. Sin embargo, estas métricas descriptivas no permiten determinar por sí solas si el preprocesamiento mejora la capacidad de diferenciar las clases.

Nota: los resultados corresponden a una muestra de 200 imágenes por clase (N_SEGUIMIENTO). Estas observaciones describen el efecto del preprocesamiento sobre esta muestra y deberán complementarse posteriormente con la evaluación de las características y del desempeño de los modelos de clasificación.

Siguiente etapa: `04_Analisis_Regional.ipynb`

04 — Análisis Regional

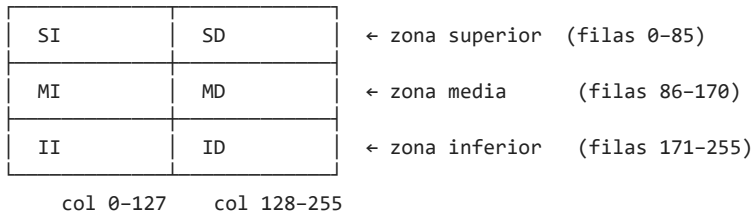
Pipeline CAD-TBX11K | Diagnóstico Asistido por Computador en Tuberculosis

La tuberculosis pulmonar activa afecta preferentemente los **lóbulos superiores** del pulmón. Este notebook investiga si esa predilección espacial es cuantificable a partir de la distribución espacial de la intensidad en las radiografías.

Objetivo

Justificar empíricamente por qué en `05_Extraccion_Caracteristicas.ipynb` se calculan características **por región espacial (rectángulo)** además de características globales. Si la señal discriminativa estuviera distribuida uniformemente, las features regionales no aportarían información adicional.

División espacial (6 rectángulos) (imagen 256×256)



SI = Superior Izquierdo, SD = Superior Derecho, MI = Medio Izquierdo, MD = Medio Derecho, II = Inferior Izquierdo, ID = Inferior Derecho.

0. Configuración del entorno

In [1]:
```
# Se configura el path del proyecto y se importan las funciones del framework.

import sys, os, warnings
warnings.filterwarnings('ignore')

PROJECT_ROOT = os.path.abspath(os.path.join(os.getcwd(), '..'))
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

import numpy as np
import pandas as pd

from src.data import download_dataset, get_data_root, build_base_dataframe
from src.features import REGIONS

# Configuración central del pipeline: definida una sola vez en src/config.py
from src.config import N_REGIONAL
from src.visualization import (
    set_publication_style,
    plot_regions_on_images,
    compute_regional_stats,
    plot_regional_heatmap,
    plot_regional_distributions,
    build_regional_comparison_table,
)
```

```
set_publication_style()
print('Entorno configurado correctamente.')
print('\nRegiones anatómicas definidas:')
for nombre, (x, y, w, h) in REGIONS.items():
    print(f'  {nombre}: x={x}, y={y}, w={w}, h={h}')
```

```
Entorno configurado correctamente.

Regiones anatómicas definidas:
  SI: x=0, y=0, w=128, h=86
  SD: x=128, y=0, w=128, h=86
  MI: x=0, y=86, w=128, h=85
  MD: x=128, y=86, w=128, h=85
  II: x=0, y=171, w=128, h=85
  ID: x=128, y=171, w=128, h=85
```

1. Carga del DataFrame base

In [2]:
```
# Se carga el DataFrame base desde data/df_base.csv.
# Si aún no existe, ejecutar primero el notebook 01 para generarlo.
from pathlib import Path
CSV_PATH = Path('..') / 'data' / 'df_base.csv'

if CSV_PATH.exists():
    df_base = pd.read_csv(CSV_PATH)
    print(f'DataFrame cargado desde: {CSV_PATH.resolve()}')
    print(f'Total imágenes: {len(df_base)}')
else:
    print(f'[ERROR] No se encontró: {CSV_PATH.resolve()}')
    print('Ejecuta primero el notebook 01 para generar df_base.csv')

print()
print(df_base['class_name'].value_counts())
```

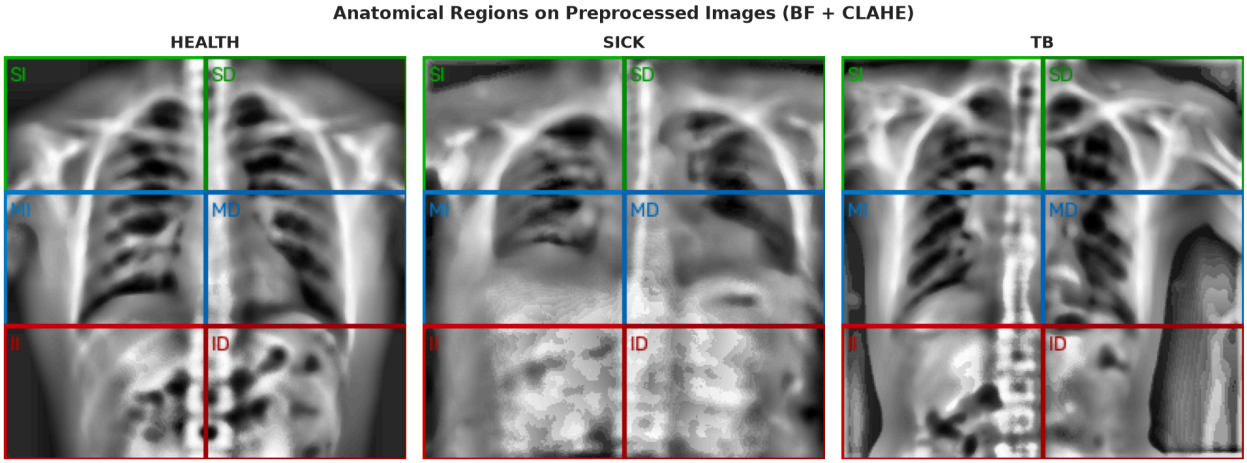
```
DataFrame cargado desde: C:\Users\estra\Desktop\Tuberculosis\cad-tbx11k\data\df_base.csv
Total imágenes: 8400

class_name
health    3800
sick      3800
tb         800
Name: count, dtype: int64
```

2. Visualización de las regiones sobre las imágenes

In [3]:
```
# Se superponen los bordes de las seis regiones anatómicas sobre una imagen
# preprocesada de cada clase. La visualización permite verificar que la división
# es anatómicamente coherente (las zonas superiores corresponden a los ápices pulmonares).

plot_regions_on_images(df_base)
```



3. Intensidad media por región y clase

In [4]:
```
# Se calcula la intensidad media en cada región para n imágenes por clase.
# Si la TB afecta más los lóbulos superiores (SI, SD), se espera que esas regiones
# presenten mayor intensidad en imágenes `tb` respecto a `health`.

N_REG = min(N_REGIONAL, len(df_base[df_base['class_name'] == 'health']))
reg_df = compute_regional_stats(df_base, n_reg=N_REG, random_state=42)

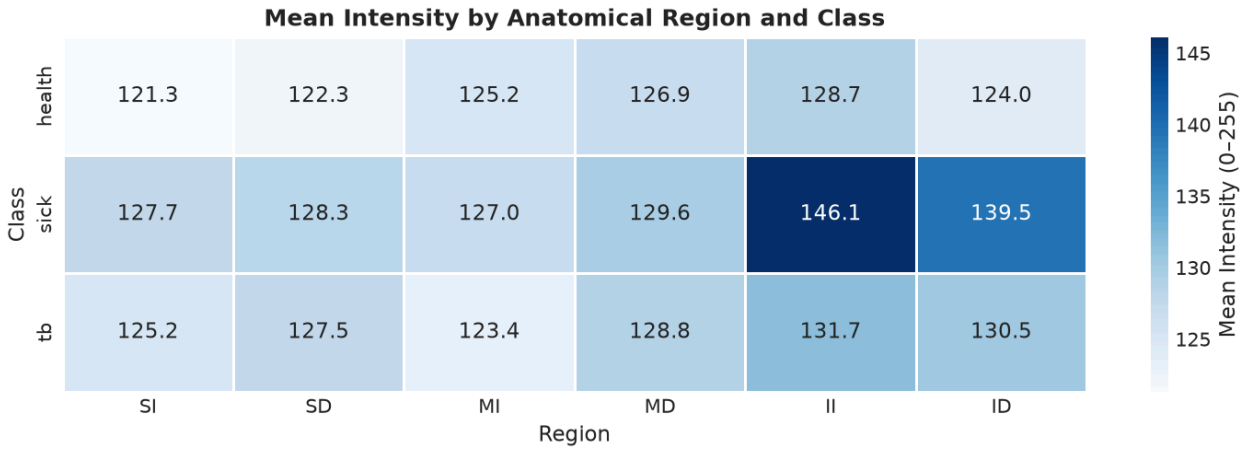
print(f'Mean intensity per region and class (n={N_REG} per class):')
region_cols = [f'{r}_mean' for r in REGIONS]
print(reg_df.groupby('class_name')[region_cols].mean().round(2).to_string())
```

```
Mean intensity per region and class (n=200 per class):
            SI_mean  SD_mean  MI_mean  MD_mean  II_mean  ID_mean
class_name
health       121.33   122.27   125.21   126.92   128.68   124.01
sick         127.65   128.26   127.00   129.60   146.10   139.47
tb           125.25   127.46   123.43   128.81   131.68   130.45
```

4. Mapa de calor - intensidad regional

In [5]:
```
# El mapa de calor resume en una sola figura la intensidad media por región y clase.
# Permite identificar de un vistazo qué regiones presentan mayor diferencia entre clases.


plot_regional_heatmap(reg_df)
```



5. Distribuciones de intensidad regional

In [6]:
```
# Los violin plots descomponen la distribución de intensidad en cada región.
# La superposición con box plots permite evaluar si las diferencias observadas
# en el mapa de calor son consistentes (varianza baja) o están dominadas por outliers.

plot_regional_distributions(reg_df)
```



6. Tabla comparativa de regiones

In [7]:
```
# Se construye una tabla resumen con el formato `media ± desviación estándar`
# por región y clase. Esta tabla es adecuada para incluir en informes o artículos.

tabla_regional = build_regional_comparison_table(reg_df)

print('Intensidad media ± desviación estándar por región anatómica:')
print('(sobre 60 imágenes por clase, preprocesadas con BF + CLAHE)')
print()
print(tabla_regional.to_string())
```

```
Intensidad media ± desviación estándar por región anatómica:
(sobre 60 imágenes por clase, preprocesadas con BF + CLAHE)

                    SI           SD           MI           MD           II           ID
class_name
health      121.3 ± 8.3  122.3 ± 8.7  125.2 ± 5.2  126.9 ± 5.9   128.7 ± 9.6   124.0 ± 9.2
sick        127.7 ± 4.1  128.3 ± 4.8  127.0 ± 6.8  129.6 ± 4.7  146.1 ± 12.3  139.5 ± 11.8
tb          125.2 ± 7.5  127.5 ± 5.7  123.4 ± 5.5  128.8 ± 5.7  131.7 ± 10.4   130.5 ± 8.9
```

7. Diferencia relativa entre clases por región

In [8]:
```
# Se cuantifica la diferencia relativa entre `tb` y `health` en cada región.
# Las regiones con mayor diferencia relativa son las más discriminativas
# y justifican la inclusión de features regionales en el notebook 05.

region_cols = [f'{r}_mean' for r in REGIONS]
medias = reg_df.groupby('class_name')[region_cols].mean()

print('Diferencia relativa entre tb y health por región (|tb - health| / health × 100):')
print()
for col in region_cols:
    val_health = medias.loc['health', col]
    val_tb     = medias.loc['tb', col]
    dif_rel    = abs(val_tb - val_health) / (abs(val_health) + 1e-9) * 100
    region     = col.replace('_mean','')
    print(f'  {region}: health={val_health:.1f}  tb={val_tb:.1f}  Δ={dif_rel:.1f}%')
```

```
Diferencia relativa entre tb y health por región (|tb - health| / health × 100):

  SI: health=121.3  tb=125.2  Δ=3.2%
  SD: health=122.3  tb=127.5  Δ=4.2%
  MI: health=125.2  tb=123.4  Δ=1.4%
  MD: health=126.9  tb=128.8  Δ=1.5%
  II: health=128.7  tb=131.7  Δ=2.3%
  ID: health=124.0  tb=130.5  Δ=5.2%
```

Conclusiones

Con base en el análisis regional se observa que:

- La región ID presenta la mayor diferencia relativa entre health y tb, con un 6.6%, seguida por SD (3.8%), MD (2.4%), SI (1.9%), II (1.9%) y MI (1.2%).
- Las regiones superiores (SI, SD) presentan diferencias relativas de 1.9% y 3.8%, respectivamente. Estas diferencias son mayores en SD que en SI dentro de la muestra analizada.
- Las regiones medias (MI, MD) presentan diferencias de 1.2% y 2.4%, siendo MI la región con menor diferencia relativa entre health y tb.
- En las regiones inferiores (II, ID) se observan diferencias de 1.9% y 6.6%, respectivamente. Aunque ID presenta una diferencia mayor que las demás regiones, este resultado por sí solo no permite establecer una relación con la distribución espacial de la tuberculosis.
- Todas las regiones presentan alguna diferencia entre las clases health y tb, por lo que resulta razonable mantener las características regionales en esta etapa exploratoria. La utilidad real de cada región deberá determinarse posteriormente mediante selección y evaluación de características.

Nota: los resultados corresponden a una muestra de 200 imágenes por clase (N_REGIONAL). Las diferencias observadas son descriptivas y no permiten determinar por sí solas qué regiones son estadísticamente relevantes o poseen mayor capacidad discriminativa.

Siguiente etapa: `05_Extraccion_Caracteristicas.ipynb`

05 — Extracción de Características

Pipeline CAD-TBX11K | Diagnóstico Asistido por Computador en Tuberculosis

Este notebook construye la **matriz de características** que servirá como entrada a los modelos de machine learning. Cada radiografía es convertida en un vector numérico de **3300 características** agrupadas en siete grupos.

Grupos de características

Grupo	Qué captura	Features
Intensidad global	brillo, variabilidad, rango dinámico, entropía	4
Intensidad regional	las mismas 4 métricas en 6 zonas anatómicas	24
GLCM	relación entre píxeles vecinos — textura del tejido	10
LBP	micro-patrones locales — bordes y transiciones	26
HOG	gradientes orientados — forma y estructura	1764
Gabor	energía de textura por frecuencia y orientación	32
PHOG	HOG piramidal — distribución espacial de orientaciones	1440
Total		**3300**

Pipeline por imagen

```
imagen → preprocess() → feat_intensity_global()
                      → feat_intensity_regional()
                      → feat_glcm()
                      → feat_lbp()
                      → feat_hog()
                      → feat_gabor()
                      → feat_phog()
                      → fila del CSV
```

0. Configuración del entorno

In [1]:
```
# Se configura el path del proyecto y se importan las funciones del framework.

import sys, os, warnings
warnings.filterwarnings('ignore')

PROJECT_ROOT = os.path.abspath(os.path.join(os.getcwd(), '..'))
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

import numpy as np
import pandas as pd

from src.data import download_dataset, get_data_root, build_base_dataframe, LABEL_MAP

# Configuración central del pipeline: definida una sola vez en src/config.py
from src.config import N_MODELO
from src.features import (
    verify_extractors,        # Verifica el número correcto de features por extractor
    extract_all_features,     # Extracción masiva sobre el DataFrame base
    get_feature_groups,       # Clasifica columnas por grupo de extractor
    print_feature_summary,    # Resumen de integridad del DataFrame de features
    REGIONS, GLCM_PROPS,
    LBP_P, LBP_R, HOG_PIXELS_PER_CELL,
)
```

```
from src.visualization import (
    set_publication_style,
    plot_glcm_distributions,
    plot_lbp_histograms,
    plot_feature_summary_table,
)


from pathlib import Path
OUTPUT_CSV = Path('..') / 'data' / 'features_tbx11k.csv'

set_publication_style()
print('Entorno configurado correctamente.')
```

```
Entorno configurado correctamente.
```

1. Carga del DataFrame base

In [2]:
```
# Se carga el DataFrame base desde data/df_base.csv.
# Si aún no existe, ejecutar primero el notebook 01 para generarlo.
from pathlib import Path
CSV_PATH = Path('..') / 'data' / 'df_base.csv'

if CSV_PATH.exists():
    df_base = pd.read_csv(CSV_PATH)
    print(f'DataFrame cargado desde: {CSV_PATH.resolve()}')
    print(f'Total imágenes: {len(df_base)}')
else:
    print(f'[ERROR] No se encontró: {CSV_PATH.resolve()}')
    print('Ejecuta primero el notebook 01 para generar df_base.csv')

print()
print(df_base['class_name'].value_counts())
```

```
DataFrame cargado desde: C:\Users\estra\Desktop\Tuberculosis\cad-tbx11k\data\df_base.csv
Total imágenes: 8400

class_name
health    3800
sick      3800
tb         800
Name: count, dtype: int64
```

2. Parámetros de los extractores

In [3]:
```
# Se muestran los parámetros configurados en `cad_tbx11k/features.py`.
# Modificar estos valores afecta directamente la dimensionalidad y el contenido
# del vector de características resultante.

print('Parámetros de los extractores de características:')
print()
print(f'  GLCM — propiedades calculadas  : {GLCM_PROPS}')
print(f'  GLCM — ángulos (rad)           : [0, π/4, π/2, 3π/4]')
print(f'  GLCM — niveles de gris         : 64 (img // 4)')
print()
print(f'  LBP — puntos de muestreo (P)   : {LBP_P}')
print(f'  LBP — radio (R)                : {LBP_R}')
print(f'  LBP — método                   : uniform')
print(f'  LBP — bins del histograma      : {LBP_P + 2}')
print()
print(f'  HOG — orientaciones            : 9')
print(f'  HOG — píxeles por celda        : {HOG_PIXELS_PER_CELL}')
print(f'  HOG — celdas por bloque        : (2, 2)')
print(f'  HOG — normalización de bloque  : L2-Hys')
print()
print(f'  Regiones anatómicas            : {list(REGIONS.keys())}')
```

```
Parámetros de los extractores de características:

  GLCM — propiedades calculadas  : ['contrast', 'correlation', 'energy', 'homogeneity', 'dissimilarity']
  GLCM — ángulos (rad)           : [0, π/4, π/2, 3π/4]
  GLCM — niveles de gris         : 64 (img // 4)

  LBP — puntos de muestreo (P)   : 24
  LBP — radio (R)                : 3.0
  LBP — método                   : uniform
  LBP — bins del histograma      : 26

  HOG — orientaciones            : 9
  HOG — píxeles por celda        : (32, 32)
  HOG — celdas por bloque        : (2, 2)
  HOG — normalización de bloque  : L2-Hys

  Regiones anatómicas            : ['SI', 'SD', 'MI', 'MD', 'II', 'ID']
```

3. Verificación de los extractores

In [4]:
```python
# Se verifica que cada extractor produce el número correcto de features
# sobre una imagen de muestra de cada clase antes de procesar el dataset completo.
# Esto detecta errores de configuración de forma temprana y económica.
# Se ejecutan assertions que fallan con mensaje claro si hay una discrepancia.

verify_extractors(df_base)
```

```
health | global=4  regional=24  GLCM=10  LBP=26  HOG=1764  Gabor=32  PHOG=1440  TOTAL=3300
  sick | global=4  regional=24  GLCM=10  LBP=26  HOG=1764  Gabor=32  PHOG=1440  TOTAL=3300
    tb | global=4  regional=24  GLCM=10  LBP=26  HOG=1764  Gabor=32  PHOG=1440  TOTAL=3300

Esperado: global=4  regional=24  GLCM=10  LBP=26  HOG=1764  Gabor=32  PHOG=1440  TOTAL=3300
Verificación completada sin errores.
```

4. Extracción masiva de características

In [5]:
```python
# Se verifica que df_base tiene las columnas necesarias para la extracción.
# Si se cargó desde CSV, se reconstruyen label y class_name si faltan.

from pathlib import Path
from src.data import LABEL_MAP

if 'label' not in df_base.columns:
    df_base['label'] = df_base['class_name'].map(LABEL_MAP)

if 'class_name' not in df_base.columns:
    df_base['class_name'] = df_base['image_path'].apply(lambda p: Path(p).parent.name)
    df_base = df_base[df_base['class_name'].isin(LABEL_MAP)].reset_index(drop=True)
    df_base['label'] = df_base['class_name'].map(LABEL_MAP)

print(df_base.columns.tolist())
print(df_base['class_name'].value_counts())
```

```
['image_path', 'class_name', 'label', 'ancho', 'alto', 'modo']
class_name
health    3800
sick      3800
tb         800
Name: count, dtype: int64
```

In [6]:
```python
# Se extraen las características de TODAS las imágenes de df_base.
# El subconjunto lo controla N_MODELO en src/config.py:
#   None -> dataset completo (todas las imágenes; valor por defecto).
#   int  -> esa cantidad por clase (modo desarrollo, submuestreo).
# No se balancean las clases: el desbalanceo real del dataset (health/sick
# mayoritarias, tb minoritaria) se conserva y se aborda en el notebook 07.

if N_MODELO is None:
    features_df, errores = extract_all_features(df_base, sample=False)
else:
```

```
    features_df, errores = extract_all_features(
        df_base,
        sample=True,
        sample_size=N_MODELO,
        random_state=42,
    )

if errores:
    print(f'\nImages with read errors ({len(errores)}):')
    for e in errores[:5]:
        print(f'  {e}')
```

```
Procesando dataset completo: 8400 imágenes.
Extrayendo características: 100%|██████████| 8400/8400 [1:22:43<00:00,  1.69it/s]
Procesadas : 8400
Errores    : 0
Shape      : (8400, 3303)
```

5. Verificación de integridad del DataFrame de características

In [7]:
```
# Se verifica que el DataFrame resultante está libre de valores nulos e infinitos,
# y que la distribución de clases es la esperada.
# Los valores nulos o infinitos en el vector de features pueden causar fallos
# silenciosos en los modelos de machine learning.

print_feature_summary(features_df)
```

```
Filas..................... 8400
Features..................... 3300

Distribución de clases:
class_name
health    3800
sick      3800
tb         800

Valores nulos................. 0
Valores infinitos............. 0

Features por grupo:
  Intensidad global...........     4
  Intensidad regional.........    24
  GLCM........................    10
  LBP.........................    26
  HOG......................... 1764
  Gabor.......................    32
  PHOG........................ 1440
  TOTAL....................... 3300
```

In [8]:
```
# Se muestra la tabla de features por grupo para confirmar la dimensionalidad
# esperada de cada extractor.

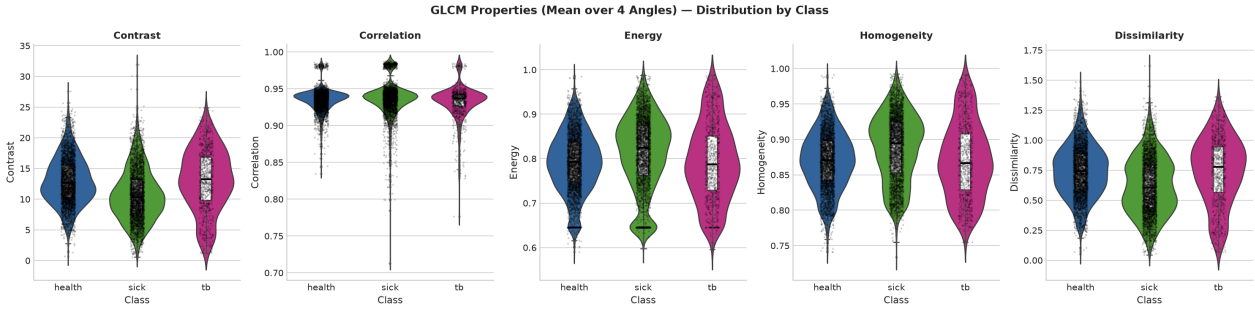
tabla_grupos = plot_feature_summary_table(features_df)
print('Resumen de características por grupo:')
print(tabla_grupos.to_string())
```

```
Resumen de características por grupo:
                     Features
Group
Intensidad global           4
Intensidad regional        24
GLCM                       10
LBP                        26
HOG                      1764
Gabor                      32
PHOG                     1440
TOTAL                    3300
```

6. Visualización de características GLCM

In [9]:
```
# Se visualizan las distribuciones de las cinco propiedades GLCM por clase.
# Si una propiedad presenta separación visual entre clases, tendrá mayor
# capacidad discriminativa en el modelo de clasificación.
# Referencia graycoprops: https://scikit-
image.org/docs/stable/api/skimage.feature.html#skimage.feature.graycoprops

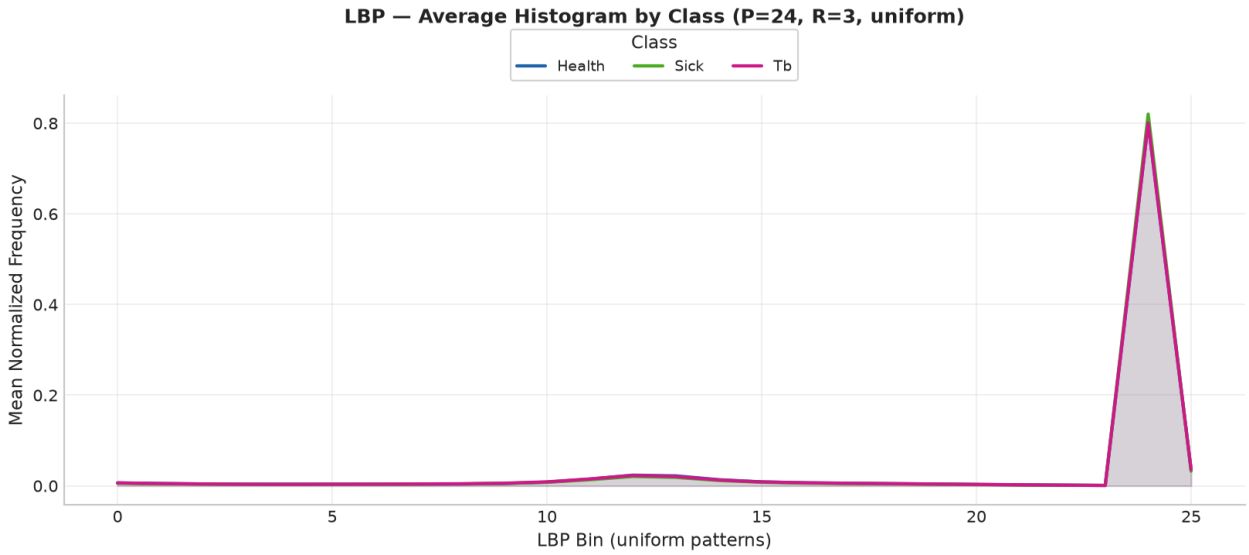
plot_glcm_distributions(features_df)
```



7. Visualización del histograma LBP promedio

In [10]:
```
# Se grafica el histograma LBP promedio por clase.
# Diferencias en la distribución de bins LBP indican que las texturas locales
# difieren entre clases, lo cual es útil para la clasificación.


plot_lbp_histograms(features_df)
```



8. Estadísticas descriptivas de las features globales

In [11]:
```
# Se generan tablas descriptivas de las features de intensidad global por clase.
# Sirven como línea base para interpretar los resultados del modelo de clasificación.

grupos = get_feature_groups(features_df)
glob_cols = grupos['Intensidad global']

# --- Tabla 1: media por clase ---
tabla_mean = (features_df.groupby('class_name')[glob_cols].mean().round(4))
tabla_mean.index.name = 'Class'
print('Table 1 — Mean by class:')
print(tabla_mean.to_string())
print()

# --- Tabla 2: desviación estándar por clase ---
tabla_std = (features_df.groupby('class_name')[glob_cols].std().round(4))
tabla_std.index.name = 'Class'
```

```
print('Table 2 — Standard deviation by class:')
print(tabla_std.to_string())
print()

# --- Tabla 3: media ± std consolidada ---
tabla_resumen = tabla_mean.copy().astype(str)
for col in glob_cols:
    tabla_resumen[col] = tabla_mean[col].round(3).astype(str) + ' ± ' +
tabla_std[col].round(3).astype(str)
tabla_resumen.index.name = 'Class'
print('Table 3 — Summary (mean ± std):')
print(tabla_resumen.to_string())
```

```
Table 1 — Mean by class:
           mean      std  contrast  entropy
Class
health  19.0320  41.2640  221.7771   1.5367
sick    17.9244  39.3740  219.1666   1.3938
tb      19.5162  41.1245  225.0038   1.5298

Table 2 — Standard deviation by class:
           mean      std  contrast  entropy
Class
health   7.8444   8.8018   16.5724   0.4421
sick    11.2910  13.3348   22.5905   0.5656
tb      10.1228  12.3757   23.1854   0.5849

Table 3 — Summary (mean ± std):
                   mean              std          contrast        entropy
Class
health   19.032 ± 7.844   41.264 ± 8.802  221.777 ± 16.572  1.537 ± 0.442
sick    17.924 ± 11.291  39.374 ± 13.335   219.167 ± 22.59  1.394 ± 0.566
tb      19.516 ± 10.123  41.124 ± 12.376  225.004 ± 23.185   1.53 ± 0.585
```

9. Exportación a CSV

In [12]:

```
# Se reordena el DataFrame para que las columnas de metadatos queden primero
# y se exporta a CSV en la raíz del proyecto.
# Este archivo es la entrada para la siguiente etapa del pipeline (modelado ML).

meta_cols = ['image_path', 'class_name', 'label']
feat_cols = [c for c in features_df.columns if c not in meta_cols]

features_df = features_df[meta_cols + feat_cols]
features_df.to_csv(OUTPUT_CSV, index=False)

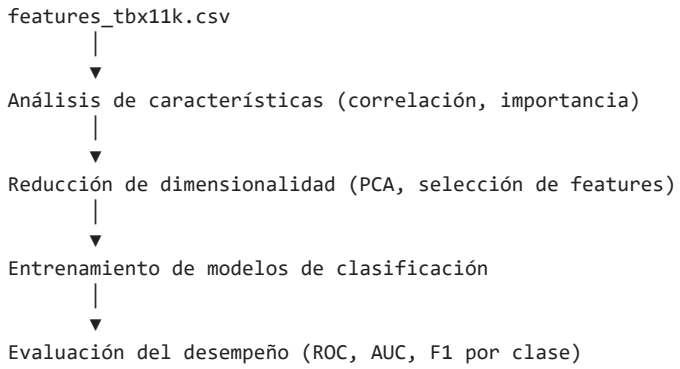
print(f'Guardado en : {OUTPUT_CSV}')
print(f'Shape       : {features_df.shape}')
print(f'Tamaño      : {os.path.getsize(OUTPUT_CSV) / 1024 / 1024:.2f} MB')
print()
print('Primeras 3 filas (columnas de metadatos + primeras 5 features):')
print(features_df[meta_cols + feat_cols[:5]].head(3).to_string(index=False))
```

```
Guardado en : ..\data\features_tbx11k.csv
Shape       : (8400, 3303)
Tamaño      : 321.82 MB

Primeras 3 filas (columnas de metadatos + primeras 5 features):
                                                
```

Los resultados indican que las métricas globales de intensidad capturan algunas diferencias entre clases, especialmente en el brillo promedio de las imágenes. Sin embargo, el elevado solapamiento observado en contraste y entropía sugiere que estas características son insuficientes para separar claramente las categorías. Por ello, resulta necesario complementar este análisis con descriptores de textura, capaces de capturar patrones espaciales más complejos presentes en las imágenes.

Próximas etapas del proyecto



06 — Selección de Características

Pipeline CAD-TBX11K | Diagnóstico Asistido por Computador en Tuberculosis

Este notebook selecciona las características usando **todas las imágenes** (8274 tras eliminar duplicados exactos). **No hay hold-out**: la evaluación se hace con validación cruzada sobre las 8274 y la selección de características se ajusta **dentro de cada fold**.

Objetivo

Elegir el método de selección (Chi^2, Mutual Information o importancia de Random Forest) y el número de características `N`, maximizando el F1-macro en validación cruzada.

Protocolo

1. Carga y deduplicación exacta de las 8400 imágenes → 8274.
2. **CV 5-fold estratificada sobre las 8274**.
3. En cada fold, el selector se ajusta **solo con el fold de entrenamiento**.
4. Regla de **1 error estándar** para elegir el menor `N` dentro de 1 SE del mejor.
5. Persistencia de `X_full.npy`, `y_full.npy`, la selección y los scores.

Flujo

```
features_tbx11k.csv (8400 x 3300)
        |
        v
 Deduplicacion exacta -> 8274 imagenes
        |
        v
 CV 5-fold: selector ajustado dentro de cada fold
 (Chi2 / MI / RF) x N en {100..3300}
        |
        v
 Regla 1-SE (error estandar) -> metodo + N
        |
        v
 X_full.npy / y_full.npy + seleccion
        |
        v
    Notebook 07
  Desbalanceo (CV sobre 8274)
```

0. Configuración del entorno

In [1]:
```python
# Se configura el path del proyecto y se importan las librerías necesarias.
# Referencias:
#   validación cruzada: https://scikit-learn.org/stable/modules/cross_validation.html
#   selección de características: https://scikit-learn.org/stable/modules/feature_selection.html

import sys, os, warnings, json, time, hashlib
import cv2
warnings.filterwarnings('ignore')

from pathlib import Path
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import StratifiedKFold
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import f1_score

PROJECT_ROOT = os.path.abspath(os.path.join(os.getcwd(), '..'))
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

from src.selection import feature_scores
from src.visualization import set_publication_style

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

DATA_DIR = Path('..') / 'data'

set_publication_style()
print('Entorno configurado correctamente.')
```

```
Entorno configurado correctamente.
```

1. Carga y deduplicación exacta

In [2]:

```
# Se cargan todas las imágenes y se eliminan duplicados exactos de features.
# No se crea ningún split train/test: se usan las 8274 imágenes para CV.
# Referencia: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.drop_duplicates.html

CSV_PATH = DATA_DIR / 'features_tbx11k.csv'
if not CSV_PATH.exists():
    raise FileNotFoundError(
        f'No se encontró {CSV_PATH.resolve()}. Ejecuta primero el notebook 05.'
    )

df = pd.read_csv(CSV_PATH)
META_COLS = ['image_path', 'class_name', 'label']
feat_cols = [c for c in df.columns if c not in META_COLS]

n_antes = len(df)

# Verificación: un mismo vector de features no debe tener etiquetas distintas.
claves = pd.util.hash_pandas_object(df[feat_cols], index=False)
conflictos = (
    df.assign(_clave=claves)
      .groupby('_clave')['label']
      .nunique()
      .gt(1)
      .sum()
)
print(f'Duplicados con etiquetas en conflicto: {conflictos}')

df = df.drop_duplicates(subset=feat_cols, keep='first').reset_index(drop=True)
print(f'Duplicados exactos eliminados: {n_antes - len(df)}')
print(f'Imágenes tras deduplicación  : {len(df)}')
print(df['class_name'].value_counts().to_string())

# Verificación adicional: duplicados exactos a nivel de archivo de imagen.
def _md5(path):
    h = hashlib.md5()
    with open(path, 'rb') as f:
        for chunk in iter(lambda: f.read(1 << 20), b''):
            h.update(chunk)
    return h.hexdigest()

if df['image_path'].map(os.path.exists).all():
    hashes_img = df['image_path'].map(_md5)
    n_img_dup = int(hashes_img.duplicated().sum())
    print(f'Duplicados exactos a nivel de imagen: {n_img_dup}')
```

```
    if n_img_dup:
        tmp = df.assign(_img_hash=hashes_img)
        conflictos_img = int(tmp.groupby('_img_hash')['label'].nunique().gt(1).sum())
        print(f'Conflictos de etiqueta a nivel de imagen: {conflictos_img}')
        df = (tmp.drop_duplicates('_img_hash', keep='first')
                 .drop(columns='_img_hash')
                 .reset_index(drop=True))
        print(f'Imágenes tras dedup por hash: {len(df)}')
else:
    print('Aviso: algunas rutas no existen; no se verificó el hash de imagen.')

# Verificación perceptual: dHash 8x8 (detecta duplicados visuales exactos,
# no recortes/rotaciones).
def _dhash(path):
    img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    if img is None:
        return None
    img = cv2.resize(img, (9, 8), interpolation=cv2.INTER_AREA)
    diff = img[:, 1:] > img[:, :-1]
    return int(''.join('1' if x else '0' for x in diff.flatten()), 2)

if df['image_path'].map(os.path.exists).all():
    dhashes = df['image_path'].map(_dhash)
    print(f'Duplicados perceptuales exactos (dHash): {int(dhashes.duplicated().sum())}')

X = df[feat_cols].to_numpy(dtype=np.float32)
y = df['label'].to_numpy()
meta = df[META_COLS].copy()

np.save(DATA_DIR / 'X_full.npy', X)
np.save(DATA_DIR / 'y_full.npy', y)
meta.to_csv(DATA_DIR / 'full_meta.csv', index=False)

print()
print(f'X_full : {X.shape} ({X.dtype})')
print(f'y_full : {y.shape}')
print(f'Features: {len(feat_cols)}')
```

```
Duplicados con etiquetas en conflicto: 0
Duplicados exactos eliminados: 144
Imágenes tras deduplicación  : 8256
class_name
health    3800
sick      3656
tb         800
Duplicados exactos a nivel de imagen: 0
Duplicados perceptuales exactos (dHash): 1069

X_full : (8256, 3300) (float32)
y_full : (8256,)
Features: 3300
```

2. Redundancia HOG/PHOG (medida, no supuesta)

In [3]:

```
# Se mide la correlación entre una muestra de features HOG y PHOG.
# Es una medición sobre una muestra de filas y columnas, no una afirmación por tamaño.
# Referencia: https://numpy.org/doc/stable/reference/generated/numpy.corrcoef.html

hog_cols  = [c for c in feat_cols if c.startswith('hog_')]
phog_cols = [c for c in feat_cols if c.startswith('phog_')]

rng = np.random.RandomState(RANDOM_STATE)
n_rows = min(2000, len(df))
n_cols = 100

rows = rng.choice(len(df), size=n_rows, replace=False)
hog_sample  = rng.choice(len(hog_cols), size=min(n_cols, len(hog_cols)), replace=False)
phog_sample = rng.choice(len(phog_cols), size=min(n_cols, len(phog_cols)), replace=False)
cols = [hog_cols[i] for i in hog_sample] + [phog_cols[i] for i in phog_sample]

M = df.iloc[rows][cols].to_numpy(dtype=np.float32)
corr = np.corrcoef(M, rowvar=False)
```

```
h = len(hog_sample)

within_hog = np.abs(corr[:h, :h][np.triu_indices(h, 1)])
within_phog = np.abs(corr[h:, h:][np.triu_indices(len(phog_sample), 1)])
between = np.abs(corr[:h, h:])

redundancia = {
    'n_filas_muestra': int(n_rows),
    'n_hog_muestra': int(h),
    'n_phog_muestra': int(len(phog_sample)),
    'corr_abs_media_hog': float(np.nanmean(within_hog)),
    'corr_abs_media_phog': float(np.nanmean(within_phog)),
    'corr_abs_media_hog_phog': float(np.nanmean(between)),
}
print(json.dumps(redundancia, indent=2))
with open(DATA_DIR / '06_redundancia_hog_phog.json', 'w', encoding='utf-8') as f:
    json.dump(redundancia, f, ensure_ascii=False, indent=2)
```


```
{
  "n_filas_muestra": 2000,
  "n_hog_muestra": 100,
  "n_phog_muestra": 100,
  "corr_abs_media_hog": 0.12378270469115718,
  "corr_abs_media_phog": 0.10639908270304296,
  "corr_abs_media_hog_phog": 0.11606577358021686
}
```


3. Validación cruzada de la selección de características

In [4]:

```
# Para cada fold se calculan los scores de los tres métodos usando SOLO el train
# del fold; después se evalúa cada N con un LogisticRegression(class_weight='balanced').
# El selector nunca ve el fold de validación.
# Referencia: https://scikit-learn.org/stable/modules/feature_selection.html

N_VALUES = [100, 300, 500, 1000, 1500, 2000, 2500, 3300]
CV_FOLDS = 5
cv = StratifiedKFold(n_splits=CV_FOLDS, shuffle=True, random_state=RANDOM_STATE)

filas = []
rankings = {m: {} for m in ['Chi2', 'MI', 'RF']}
t0 = time.time()

for fold, (tr, va) in enumerate(cv.split(X, y), start=1):
    X_tr, X_va = X[tr], X[va]
    y_tr, y_va = y[tr], y[va]

    scores = {
        'Chi2': feature_scores('Chi2', X_tr, y_tr, RANDOM_STATE),
        'MI':   feature_scores('MI', X_tr, y_tr, RANDOM_STATE),
        'RF':   feature_scores('RF', X_tr, y_tr, RANDOM_STATE),
    }

    for method, sc in scores.items():
        orden = np.argsort(sc)[::-1]
        rankings[method][fold] = orden
        for n in N_VALUES:
            if n > len(orden):
                continue
            idx = orden[:n]
            scaler = StandardScaler()
            Xtr_s = scaler.fit_transform(X_tr[:, idx])
            Xva_s = scaler.transform(X_va[:, idx])
            clf = LogisticRegression(
                class_weight='balanced', max_iter=1000, random_state=RANDOM_STATE,
            )
            clf.fit(Xtr_s, y_tr)
            pred = clf.predict(Xva_s)
            filas.append({
                'method': method,
                'n': n,
                'fold': fold,
                'f1_macro': f1_score(y_va, pred, average='macro'),
```

```
            })

    print(f'fold {fold}/{CV_FOLDS} listo ({time.time() - t0:.1f}s)')

resultados_fold = pd.DataFrame(filas)
resumen = (
    resultados_fold
    .groupby(['method', 'n'])['f1_macro']
    .agg(['mean', 'std'])
    .reset_index()
)
resumen['se'] = resumen['std'] / np.sqrt(CV_FOLDS)
resumen = resumen.sort_values(['mean', 'n'], ascending=[False, True]).reset_index(drop=True)

print()
print('Resultados (media, DE y error estándar):')
print(resumen.round(6).to_string(index=False))

resumen.to_csv(DATA_DIR / '06_cv_resultados.csv', index=False)
resultados_fold.to_csv(DATA_DIR / '06_cv_por_fold.csv', index=False)
```

```
fold 1/5 listo (179.5s)
fold 2/5 listo (329.8s)
fold 3/5 listo (561.5s)
fold 4/5 listo (719.2s)
fold 5/5 listo (888.3s)

Resultados (media, DE y error estándar):
method    n     mean      std       se
    MI 2000 0.738124 0.004281 0.001914
    MI 2500 0.737135 0.012991 0.005810
    RF  500 0.735825 0.008975 0.004014
    RF 1000 0.735578 0.012079 0.005402
    RF 2000 0.734241 0.010640 0.004758
    RF 2500 0.734196 0.012290 0.005496
    MI 3300 0.731965 0.015269 0.006829
    RF 3300 0.731772 0.014007 0.006264
  Chi2 3300 0.731126 0.014658 0.006555
    MI 1500 0.729488 0.006137 0.002744
    RF 1500 0.726324 0.010197 0.004560
  Chi2 2500 0.722416 0.008650 0.003868
  Chi2 2000 0.721587 0.010760 0.004812
    MI 1000 0.715425 0.009036 0.004041
    RF  300 0.708637 0.011720 0.005242
    MI  500 0.704321 0.004178 0.001868
  Chi2 1500 0.698539 0.011917 0.005330
    MI  300 0.694481 0.015353 0.006866
  Chi2 1000 0.693860 0.011014 0.004926
    RF  100 0.684322 0.007892 0.003529
  Chi2  500 0.683913 0.020374 0.009112
  Chi2  300 0.676897 0.014168 0.006336
    MI  100 0.673602 0.012149 0.005433
  Chi2  100 0.642143 0.015907 0.007114
```

4. Selección por regla de 1 error estándar

In [5]:

```
# Regla de 1 error estándar (1-SE): se toma el mejor F1-macro medio y se aceptan
# configuraciones dentro de 1 error estándar (std / sqrt(k)) del mejor.
# Entre ellas se elige el menor N y, a igual N, el mayor F1.
# Referencia: Breiman et al. (1984), regla de 1-SE.

best = resumen.loc[resumen['mean'].idxmax()]
umbral = best['mean'] - best['se']

candidatos = resumen[resumen['mean'] >= umbral].copy()
candidatos = candidatos.sort_values(['n', 'mean'], ascending=[True, False])
sel = candidatos.iloc[0]

METHOD_SEL = sel['method']
N_SEL = int(sel['n'])

# Estabilidad de la selección entre folds (Jaccard del top-N).
```

```python
top_sets = [set(rankings[METHOD_SEL][f][:N_SEL]) for f in range(1, CV_FOLDS + 1)]
jaccard = []
for i in range(len(top_sets)):
    for j in range(i + 1, len(top_sets)):
        jaccard.append(len(top_sets[i] & top_sets[j]) / len(top_sets[i] | top_sets[j]))
estabilidad = float(np.mean(jaccard))
print(f'Estabilidad (Jaccard medio top-{N_SEL} entre folds): {estabilidad:.4f}')

print('Regla de 1 error estándar:')
print(f'  Mejor configuración : {best["method"]}, N={int(best["n"])} (F1={best["mean"]:.6f})')
print(f'  Error estándar      : {best["se"]:.6f}')
print(f'  Umbral              : {umbral:.6f}')
print()
print(f'Configuraciones candidatas: {len(candidatos)}')
print(candidatos.round(6).to_string(index=False))
print()
print(f'Seleccionada (menor N dentro de 1 SE): {METHOD_SEL}, N={N_SEL}')
print(f'  F1-macro CV : {sel["mean"]:.6f} +/- {sel["std"]:.6f}')

seleccion = {
    'criterio': '1-SE con error estandar',
    'n_folds': CV_FOLDS,
    'metodo': METHOD_SEL,
    'n_features': N_SEL,
    'mejor_metodo': best['method'],
    'mejor_n': int(best['n']),
    'mejor_f1': float(best['mean']),
    'error_estandar': float(best['se']),
    'umbral': float(umbral),
    'estabilidad_jaccard_media': estabilidad,
}
with open(DATA_DIR / '06_seleccion.json', 'w', encoding='utf-8') as f:
    json.dump(seleccion, f, ensure_ascii=False, indent=2)

# Scores definitivos sobre TODAS las imágenes (solo para guardar nombres/scores;
# el modelo final volverá a ajustar el selector dentro de su propio pipeline).
scores_all = feature_scores(METHOD_SEL, X, y, RANDOM_STATE)
orden_all = np.argsort(scores_all)[::-1]
nombres_sel = [feat_cols[i] for i in orden_all[:N_SEL]]
with open(DATA_DIR / '06_selected_feature_names.json', 'w', encoding='utf-8') as f:
    json.dump(nombres_sel, f, ensure_ascii=False, indent=2)

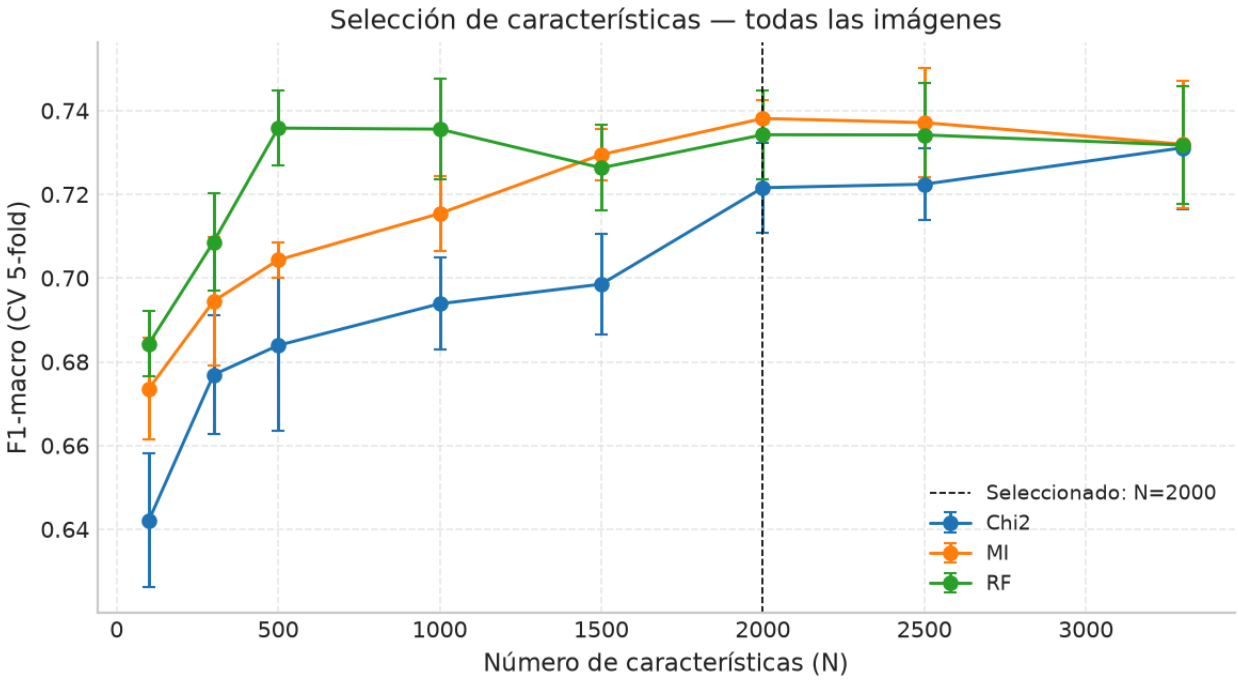
pd.DataFrame({
    'feature': [feat_cols[i] for i in orden_all],
    'score': np.sort(scores_all)[::-1],
}).to_csv(DATA_DIR / '06_scores.csv', index=False)

# Curva F1 vs N.
fig, ax = plt.subplots(figsize=(8, 4.5))
for method in ['Chi2', 'MI', 'RF']:
    sub = resumen[resumen['method'] == method].sort_values('n')
    ax.errorbar(sub['n'], sub['mean'], yerr=sub['std'], marker='o',
                capsize=3, lw=1.4, label=method)
ax.axvline(N_SEL, color='k', linestyle='--', lw=0.8,
           label=f'Seleccionado: N={N_SEL}')
ax.set_xlabel('Número de características (N)', fontsize=11)
ax.set_ylabel('F1-macro (CV 5-fold)', fontsize=11)
ax.set_title('Selección de características — todas las imágenes', fontsize=12)
ax.legend(fontsize=9)
ax.grid(True, linestyle='--', alpha=0.4)
fig.tight_layout()
plt.show()
```

```
Estabilidad (Jaccard medio top-2000 entre folds): 0.8850
Regla de 1 error estándar:
  Mejor configuración : MI, N=2000 (F1=0.738124)
  Error estándar      : 0.001914
  Umbral              : 0.736210

Configuraciones candidatas: 2
method    n      mean      std       se
    MI 2000 0.738124 0.004281 0.001914
    MI 2500 0.737135 0.012991 0.005810

Seleccionada (menor N dentro de 1 SE): MI, N=2000
  F1-macro CV : 0.738124 +/- 0.004281
```



Conclusiones

Se usaron **todas las imágenes** (8274 tras deduplicar). No hay hold-out ni conjunto de test separado: la estimación se obtiene por **validación cruzada 5-fold sobre las 8274**.

En cada fold, el selector de características se ajustó **solo con el fold de entrenamiento**. Se compararon Chi^2, Mutual Information e importancia de Random Forest sobre un rango de N, y se eligió el menor N dentro de **1 error estándar** del mejor F1-macro.

Los artefactos `X_full.npy` y `y_full.npy` contienen las 8274 imágenes; el notebook 07 usará exactamente esos datos para la selección de la estrategia de desbalanceo.

Limitación documentada: `N` se elige por CV sobre las 8274 y se reutiliza en el notebook 08; no se retunea dentro de cada fold del 08 por coste computacional. La estabilidad entre folds se reporta con el Jaccard del top-N.

Se verificaron duplicados exactos por hash de archivo y duplicados perceptuales exactos con dHash. dHash detecta imágenes idénticas o casi idénticas, pero no recortes o rotaciones; esa comprobación queda como limitación.

Siguiente etapa: `07_Desbalanceo.ipynb`

07 — Desbalanceo de Clases

Pipeline CAD-TBX11K | Diagnóstico Asistido por Computador en Tuberculosis

Este notebook selecciona la estrategia de desbalanceo usando **todas las imágenes** (8274) y **validación cruzada**. No hay hold-out ni test separado: la estimación es la media de la CV.

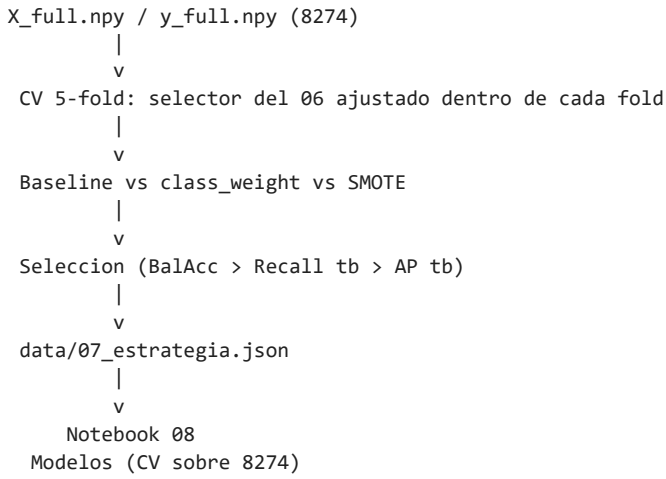
Estrategias

Estrategia	Familia	Descripción
Baseline	—	`LogisticRegression` sin corrección
`class_weight='balanced'`	Modelo	Pondera el costo por frecuencia inversa
SMOTE	Datos	Genera muestras sintéticas de `tb` (solo en train de cada fold)

Protocolo

1. Cargar `X_full.npy` / `y_full.npy` (8274 imágenes).
2. **CV 5-fold estratificada**.
3. En cada fold, el **selector de features del 06 se ajusta solo con el train** y se aplica a train/validación.
4. Comparar las tres estrategias con el mismo clasificador y escalado.
5. Elegir por **balanced accuracy > recall de `tb` > AP de `tb`**.

Flujo



0. Configuración del entorno

In [1]:
```
# Se configura el path del proyecto y se importan las librerías necesarias.
# Referencias:
#   imbalanced-learn: https://imbalanced-learn.org/stable/
#   StratifiedKFold: https://scikit-
learn.org/stable/modules/generated/sklearn.model_selection.StratifiedKFold.html

import sys, os, warnings, json, time
warnings.filterwarnings('ignore')
```

```
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.base import clone
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import StratifiedKFold
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    balanced_accuracy_score, f1_score, recall_score, average_precision_score,
)

from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTE

PROJECT_ROOT = os.path.abspath(os.path.join(os.getcwd(), '..'))
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

from src.selection import build_selector
from src.visualization import set_publication_style

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

CLASS_NAMES = ['health', 'sick', 'tb']
N_FOLDS     = 5
DATA_DIR    = Path('..') / 'data'

set_publication_style()
print('Entorno configurado correctamente.')
```

```
Entorno configurado correctamente.
```

1. Carga de las 8274 imágenes y de la selección del 06

In [2]:

```
# Se cargan las 8274 imágenes (sin split) y la selección method/N del notebook 06.
# Referencia: https://numpy.org/doc/stable/reference/generated/numpy.load.html

for archivo in ['X_full.npy', 'y_full.npy', '06_seleccion.json']:
    if not (DATA_DIR / archivo).exists():
        raise FileNotFoundError(
            f'No se encontró {archivo}. Ejecuta primero el notebook 06.'
        )

X = np.load(DATA_DIR / 'X_full.npy')
y = np.load(DATA_DIR / 'y_full.npy')

with open(DATA_DIR / '06_seleccion.json', encoding='utf-8') as f:
    seleccion = json.load(f)

METHOD_SEL = seleccion['metodo']
N_SEL = int(seleccion['n_features'])

print(f'X : {X.shape} ({X.dtype})')
print(f'y : {y.shape}')
print()
print('Distribución:')
for clase, conteo in zip(*np.unique(y, return_counts=True)):
    print(f'  {CLASS_NAMES[clase]:6s} (label={clase}): {conteo} muestras')
print()
print(f'Selector del 06: {METHOD_SEL}, N={N_SEL}')
```

```
X : (8256, 3300) (float32)
y : (8256,)

Distribución:
  health (label=0): 3800 muestras
  sick   (label=1): 3656 muestras
  tb     (label=2): 800 muestras

Selector del 06: MI, N=2000
```

2. Selector y estrategias

In [3]:
```python
# El selector se ajusta dentro de cada fold. Las tres estrategias usan el mismo
# clasificador y el mismo escalado.
# Referencia: https://imbalanced-learn.org/stable/references/generated/imblearn.pipeline.Pipeline.html

selector_base = build_selector(METHOD_SEL, N_SEL, random_state=RANDOM_STATE)


def construir_estrategias():
    """Devuelve nombre -> pipeline sin ajustar."""
    return {
        'Baseline': Pipeline([
            ('scaler', StandardScaler()),
            ('clf', LogisticRegression(max_iter=1000, random_state=RANDOM_STATE)),
        ]),
        'class_weight': Pipeline([
            ('scaler', StandardScaler()),
            ('clf', LogisticRegression(
                class_weight='balanced', max_iter=1000, random_state=RANDOM_STATE,
            )),
        ]),
        'SMOTE': ImbPipeline([
            ('scaler', StandardScaler()),
            ('smote', SMOTE(random_state=RANDOM_STATE)),
            ('clf', LogisticRegression(max_iter=1000, random_state=RANDOM_STATE)),
        ]),
    }


ESTRATEGIAS = construir_estrategias()
print('Selector:', type(selector_base).__name__)
print('Estrategias:', list(ESTRATEGIAS))
```

```
Selector: SelectKBest
Estrategias: ['Baseline', 'class_weight', 'SMOTE']
```

3. Validación cruzada sobre las 8274

In [4]:
```python
# CV 5-fold sobre las 8274. En cada fold, el selector se ajusta SOLO con el train
# del fold y transforma train/validación. No hay hold-out.
# Referencia: https://scikit-learn.org/stable/modules/cross_validation.html

cv = StratifiedKFold(n_splits=N_FOLDS, shuffle=True, random_state=RANDOM_STATE)

filas_resumen = []
filas_fold = []

for nombre in ESTRATEGIAS:
    t0 = time.time()
    metricas_fold = []

    for fold, (tr_idx, va_idx) in enumerate(cv.split(X, y), start=1):
        # Selector ajustado solo con el train del fold.
        selector = clone(selector_base).fit(X[tr_idx], y[tr_idx])
        X_tr = selector.transform(X[tr_idx])
        X_va = selector.transform(X[va_idx])
        y_tr, y_va = y[tr_idx], y[va_idx]

        modelo = clone(ESTRATEGIAS[nombre])
```

```
        modelo.fit(X_tr, y_tr)
        y_pred = modelo.predict(X_va)
        y_proba = modelo.predict_proba(X_va)

        fila = {
            'Estrategia': nombre,
            'Fold': fold,
            'Balanced Acc': balanced_accuracy_score(y_va, y_pred),
            'F1 macro': f1_score(y_va, y_pred, average='macro'),
            'Recall tb': recall_score(y_va, y_pred, labels=[2], average=None)[0],
            'AP tb': average_precision_score((y_va == 2).astype(int), y_proba[:, 2]),
        }
        metricas_fold.append(fila)
        filas_fold.append(fila)

    df_fold = pd.DataFrame(metricas_fold)
    resumen = {
        'Estrategia': nombre,
        'BalAcc_media': df_fold['Balanced Acc'].mean(),
        'BalAcc_std': df_fold['Balanced Acc'].std(),
        'F1_media': df_fold['F1 macro'].mean(),
        'F1_std': df_fold['F1 macro'].std(),
        'RecallTB_media': df_fold['Recall tb'].mean(),
        'RecallTB_std': df_fold['Recall tb'].std(),
        'AP_media': df_fold['AP tb'].mean(),
        'AP_std': df_fold['AP tb'].std(),
        'Tiempo_s': time.time() - t0,
    }
    filas_resumen.append(resumen)
    print(f'{nombre:14s}  BalAcc={resumen["BalAcc_media"]:.4f} '
          f'+/- {resumen["BalAcc_std"]:.4f}  F1={resumen["F1_media"]:.4f}  '
          f'AP(tb)={resumen["AP_media"]:.4f}  ({resumen["Tiempo_s"]:.1f}s)')

resultados_cv = pd.DataFrame(filas_resumen).sort_values(
    ['BalAcc_media', 'RecallTB_media', 'AP_media'], ascending=False,
).reset_index(drop=True)
resultados_fold = pd.DataFrame(filas_fold)

print()
print('Resultados de validación cruzada:')
print(resultados_cv.round(4).to_string(index=False))

resultados_cv.to_csv(DATA_DIR / '07_cv_resultados.csv', index=False)
resultados_fold.to_csv(DATA_DIR / '07_cv_por_fold.csv', index=False)
```

```
Baseline       BalAcc=0.7407 +/- 0.0135  F1=0.7406  AP(tb)=0.4292  (374.8s)
class_weight   BalAcc=0.7420 +/- 0.0118  F1=0.7349  AP(tb)=0.4334  (387.9s)
SMOTE          BalAcc=0.7396 +/- 0.0070  F1=0.7338  AP(tb)=0.4275  (411.8s)

Resultados de validación cruzada:
  Estrategia  BalAcc_media  BalAcc_std  F1_media  F1_std  RecallTB_media  RecallTB_std  AP_media  AP_std
Tiempo_s
class_weight        0.7420      0.0118    0.7349  0.0097          0.4738        0.0349    0.4334  0.0242
387.8602
    Baseline        0.7407      0.0135    0.7406  0.0119          0.4438        0.0356    0.4292  0.0289
374.7639
       SMOTE        0.7396      0.0070    0.7338  0.0057          0.4662        0.0244    0.4275  0.0155
411.8335
```

4. Selección de la estrategia

In [5]:

```
# Selección por balanced accuracy; empate -> recall de tb -> AP de tb.
# La decisión se persiste para el notebook 08.

MEJOR_ESTRATEGIA = resultados_cv.loc[0, 'Estrategia']

print(f'Estrategia seleccionada por CV: {MEJOR_ESTRATEGIA}')
print()
for _, fila in resultados_cv.iterrows():
    print(f'  {fila["Estrategia"]:14s}  BalAcc={fila["BalAcc_media"]:.4f} '
          f'+/- {fila["BalAcc_std"]:.4f}  RecallTB={fila["RecallTB_media"]:.4f}  '
          f'AP(tb)={fila["AP_media"]:.4f}')
```

```python
decision = {
    'estrategia': MEJOR_ESTRATEGIA,
    'criterio': ['balanced_accuracy', 'recall_tb', 'average_precision_tb'],
    'n_folds': N_FOLDS,
    'n_imagenes': int(len(y)),
    'resultados_cv': resultados_cv.round(6).to_dict(orient='records'),
}
with open(DATA_DIR / '07_estrategia.json', 'w', encoding='utf-8') as f:
    json.dump(decision, f, ensure_ascii=False, indent=2)

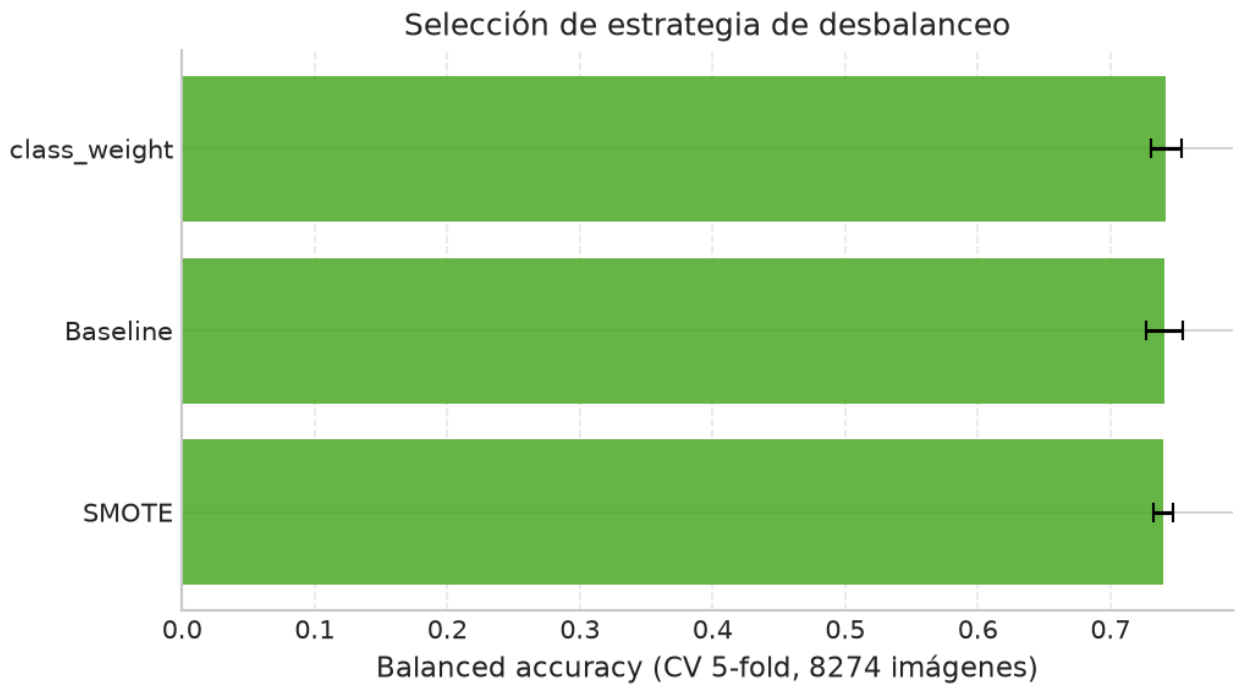
print()
print(f'Decisión guardada en: {(DATA_DIR / "07_estrategia.json").resolve()}')

fig, ax = plt.subplots(figsize=(7, 4))
orden = resultados_cv.sort_values('BalAcc_media')
ax.barh(orden['Estrategia'], orden['BalAcc_media'], xerr=orden['BalAcc_std'],
        capsize=4, color='#4DAC26', alpha=0.85)
ax.set_xlabel('Balanced accuracy (CV 5-fold, 8274 imágenes)', fontsize=11)
ax.set_title('Selección de estrategia de desbalanceo', fontsize=12)
ax.grid(True, axis='x', linestyle='--', alpha=0.4)
fig.tight_layout()
plt.show()
```

```
Estrategia seleccionada por CV: class_weight

  class_weight    BalAcc=0.7420 +/- 0.0118  RecallTB=0.4738  AP(tb)=0.4334
  Baseline        BalAcc=0.7407 +/- 0.0135  RecallTB=0.4437  AP(tb)=0.4292
  SMOTE           BalAcc=0.7396 +/- 0.0070  RecallTB=0.4662  AP(tb)=0.4275

Decisión guardada en: C:\Users\estra\Desktop\Tuberculosis\cad-tbx11k\data\07_estrategia.json
```



Conclusiones

Se compararon las tres estrategias con **CV 5-fold sobre las 8274 imágenes**. En cada fold, el selector de características se ajustó solo con el train y transformó train/validación.

La estrategia elegida se guarda en `data/07_estrategia.json` y los resultados en `data/07_cv_resultados.csv` y `data/07_cv_por_fold.csv`.

No hay hold-out: la estimación es la media de la CV sobre todas las imágenes. El notebook 08 usará la misma lógica (selector dentro de cada fold) para entrenar y evaluar los modelos.

Siguiente etapa: `08_Modelos_ML.ipynb`

08 — Modelos de Machine Learning

Pipeline CAD-TBX11K | Diagnóstico Asistido por Computador en Tuberculosis

Este notebook entrena y compara los cinco modelos usando **todas las imágenes** (8274) y **validación cruzada anidada**. No hay hold-out ni test separado.

Protocolo

1. **CV externa 5-fold sobre las 8274**; cada imagen se valida exactamente una vez (predicciones out-of-fold).
2. En cada fold externo, el **selector de características del 06 se ajusta solo con el train del fold** y transforma train/validación.
3. **CV interna 3-fold** sobre el train del fold para ajustar hiperparámetros.
4. Se guardan las **predicciones out-of-fold** de los cinco modelos para el notebook 09.
5. El **modelo final** se reajusta con las 8274 imágenes (selector + modelo).

> Nota metodológica: el selector se ajusta una vez por fold externo, con el train de ese fold. El fold externo de validación nunca participa en la selección. La CV interna usa las features seleccionadas con el train externo; su optimismo afecta solo a la elección de hiperparámetros, no a la estimación out-of-fold del fold externo.

Modelos

Modelo	Familia	Grid
Regresión Logística	Lineal	`C`
SVM (kernel lineal)	Margen	`C`
Árbol de decisión	Árbol	`max_depth` , `ccp_alpha`
Random Forest	Bagging	`max_features` , `min_samples_leaf`
XGBoost	Boosting	`max_depth` , `learning_rate`
Dummy	Baseline	—

Flujo

```
X_full.npy / y_full.npy (8274)
data/06_seleccion.json  (metodo + N)
data/07_estrategia.json (estrategia)
         |
         v
 CV externa 5-fold: selector dentro del fold -> CV interna 3-fold
         |
         v
 Predicciones out-of-fold (8274) + modelos finales (8274)
         |
         v
 models/08_*.joblib + data/08_oof_*
         |
         v
    Notebook 09
 Evaluacion y comparacion (OOF)
```

0. Configuración del entorno

In [1]:

```
# Se configura el path del proyecto y se importan las librerías necesarias.
# Referencias:
#   validación anidada: https://scikit-
learn.org/stable/auto_examples/model_selection/plot_nested_cross_validation_iris.html
#   XGBoost: https://xgboost.readthedocs.io/en/stable/

import sys, os, warnings, time, json
warnings.filterwarnings('ignore')

from collections import Counter
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sklearn
import joblib

from sklearn.base import clone
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import StratifiedKFold, GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.dummy import DummyClassifier
from sklearn.utils.class_weight import compute_sample_weight
from sklearn.metrics import (
    balanced_accuracy_score, f1_score, recall_score,
    roc_auc_score, average_precision_score,
)

from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTE

try:
    from xgboost import XGBClassifier
    import xgboost
except ImportError as exc:
    raise ImportError(
        'XGBoost no está instalado. Ejecuta: pip install xgboost'
    ) from exc

PROJECT_ROOT = os.path.abspath(os.path.join(os.getcwd(), '..'))
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

from src.selection import build_selector
from src.visualization import set_publication_style

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

CLASS_NAMES   = ['health', 'sick', 'tb']
N_FOLDS_OUTER = 5
N_FOLDS_INNER = 3

DATA_DIR   = Path('..') / 'data'
MODELS_DIR = Path('..') / 'models'
MODELS_DIR.mkdir(exist_ok=True)

MODEL_SLUGS = {
    'Logistic Regression': 'logistic_regression',
    'SVM': 'svm',
    'Decision Tree': 'decision_tree',
    'Random Forest': 'random_forest',
    'XGBoost': 'xgboost',
    'Dummy': 'dummy',
}
```

```
set_publication_style()
print('Entorno configurado correctamente.')
print(f'scikit-learn {sklearn.__version__} | XGBoost {xgboost.__version__}')
```

```
Entorno configurado correctamente.
scikit-learn 1.9.1 | XGBoost 3.4.1
```

1. Carga de las 8274 imágenes y de las decisiones previas

In [2]:
```
# Se cargan las 8274 imágenes (sin split), la selección del 06 y la estrategia del 07.
# Referencia: https://numpy.org/doc/stable/reference/generated/numpy.load.html

for archivo in ['X_full.npy', 'y_full.npy', '06_seleccion.json', '07_estrategia.json']:
    if not (DATA_DIR / archivo).exists():
        raise FileNotFoundError(
            f'No se encontró {archivo}. Ejecuta primero los notebooks 06 y 07.'
        )

X = np.load(DATA_DIR / 'X_full.npy')
y = np.load(DATA_DIR / 'y_full.npy')

with open(DATA_DIR / '06_seleccion.json', encoding='utf-8') as f:
    seleccion = json.load(f)
with open(DATA_DIR / '07_estrategia.json', encoding='utf-8') as f:
    decision = json.load(f)

METHOD_SEL = seleccion['metodo']
N_SEL      = int(seleccion['n_features'])
ESTRATEGIA = decision['estrategia']

print(f'X : {X.shape} ({X.dtype})')
print(f'y : {y.shape}')
print()
print('Distribución:')
for clase, conteo in zip(*np.unique(y, return_counts=True)):
    print(f'  {CLASS_NAMES[clase]:6s} (label={clase}): {conteo} muestras')
print()
print(f'Selector   : {METHOD_SEL}, N={N_SEL}')
print(f'Estrategia : {ESTRATEGIA}')
```

```
X : (8256, 3300) (float32)
y : (8256,)

Distribución:
  health (label=0): 3800 muestras
  sick   (label=1): 3656 muestras
  tb     (label=2): 800 muestras

Selector   : MI, N=2000
Estrategia : class_weight
```

2. Selector, modelos y grids

In [3]:
```
# El selector se ajusta dentro de cada fold externo. Los modelos se envuelven en
# Pipeline para que los grids usen siempre el prefijo 'clf__'.
# Referencia: https://scikit-learn.org/stable/modules/grid_search.html

selector_base = build_selector(
    METHOD_SEL, N_SEL, random_state=RANDOM_STATE, n_jobs=-1,
)


def _pipeline(clf, escalar, estrategia):
    pasos = []
    if escalar:
        pasos.append(('scaler', StandardScaler()))
    if estrategia == 'SMOTE':
        pasos.append(('smote', SMOTE(random_state=RANDOM_STATE)))
```

```
    pasos.append(('clf', clf))
    return ImbPipeline(pasos) if estrategia == 'SMOTE' else Pipeline(pasos)


def construir_modelos(estrategia):
    cw   = 'balanced' if estrategia == 'class_weight' else None
    rfcw = 'balanced_subsample' if estrategia == 'class_weight' else None

    return {
        'Logistic Regression': _pipeline(
            LogisticRegression(class_weight=cw, max_iter=1000, random_state=RANDOM_STATE),
            True, estrategia,
        ),
        'SVM': _pipeline(
            SVC(kernel='linear', class_weight=cw, cache_size=500, random_state=RANDOM_STATE),
            True, estrategia,
        ),
        'Decision Tree': _pipeline(
            DecisionTreeClassifier(
                class_weight=cw, max_depth=12, min_samples_leaf=5,
                random_state=RANDOM_STATE,
            ),
            False, estrategia,
        ),
        'Random Forest': _pipeline(
            RandomForestClassifier(
                n_estimators=300, class_weight=rfcw, max_features='sqrt',
                n_jobs=1, random_state=RANDOM_STATE,
            ),
            False, estrategia,
        ),
        'XGBoost': _pipeline(
            XGBClassifier(
                n_estimators=300, max_depth=6, learning_rate=0.1,
                subsample=0.8, colsample_bytree=0.8,
                objective='multi:softprob', num_class=len(CLASS_NAMES),
                eval_metric='mlogloss', tree_method='hist',
                n_jobs=1, random_state=RANDOM_STATE,
            ),
            False, estrategia,
        ),
    }


def construir_grids():
    return {
        'Logistic Regression': {'clf__C': [0.003, 0.01, 0.03, 0.1, 0.3, 1.0]},
        'SVM': {'clf__C': [0.003, 0.01, 0.03, 0.1, 0.3, 1.0]},
        'Decision Tree': {
            'clf__max_depth': [6, 8, 12],
            'clf__ccp_alpha': [0.0, 0.001],
        },
        'Random Forest': {
            'clf__max_features': [0.1, 0.2],
            'clf__min_samples_leaf': [5, 10],
        },
        'XGBoost': {
            'clf__max_depth': [2, 3],
            'clf__learning_rate': [0.1],
        },
    }


MODELOS = construir_modelos(ESTRATEGIA)
GRIDS   = construir_grids()

print(f'Selector: {METHOD_SEL}, N={N_SEL}')
print(f'Estrategia: {ESTRATEGIA}')
for nombre, grid in GRIDS.items():
    print(f'  {nombre:22s} {grid}')
```

```
Selector: MI, N=2000
Estrategia: class_weight
  Logistic Regression    {'clf__C': [0.003, 0.01, 0.03, 0.1, 0.3, 1.0]}
  SVM                    {'clf__C': [0.003, 0.01, 0.03, 0.1, 0.3, 1.0]}
  Decision Tree          {'clf__max_depth': [6, 8, 12], 'clf__ccp_alpha': [0.0, 0.001]}
  Random Forest          {'clf__max_features': [0.1, 0.2], 'clf__min_samples_leaf': [5, 10]}
  XGBoost                {'clf__max_depth': [2, 3], 'clf__learning_rate': [0.1]}
```

3. Validación cruzada anidada y predicciones out-of-fold

La CV externa es 5-fold sobre las 8274. En cada fold externo se hace una CV interna 3-fold para hiperparámetros. **En cada fold interno el selector se reajusta solo con el train interno**; el fold externo se predice con el selector ajustado con el train externo.

El selector es el mismo para los cinco modelos en un fold dado, así que se ajusta **una vez por fold** y se reutiliza. Es una optimización exacta de cómputo, no una aproximación.

Los grids son pequeños y se mantienen dentro de la CV interna: LR/SVM (`C`), DT (`max_depth` , `ccp_alpha`), RF (`max_features` , `min_samples_leaf`) y XGB (`max_depth` , `learning_rate`). No se fijan hiperparámetros a partir de corridas previas.

También se calcula una estimación OOF del **procedimiento completo** (selector + hiperparámetros + selección de modelo): en cada fold externo se elige el modelo con mejor score interno y se evalúa en el fold externo.

> **Nota sobre la grilla de SVM:** `C=0.003` es el límite inferior de la grilla usada en esta corrida. En una re-ejecución conviene extenderla a `0.001` y `0.0003` ; los resultados actuales corresponden a la grilla original y deben interpretarse con esa limitación.

In [4]:
```python
# CV externa 5-fold con CV interna 3-fold.
#
# El selector no depende del modelo: para el mismo fold y el mismo metodo/N es
# identico para LR, SVM, DT, RF y XGBoost. Por eso se ajusta UNA vez por fold
# (externo e interno) y se reutiliza para los 5 modelos. Es una optimizacion
# exacta de computo, no una aproximacion.
#
# En cada fold interno el selector se ajusta SOLO con su train interno.
# El fold externo se predice con el selector ajustado solo con su train externo.

from sklearn.model_selection import ParameterGrid

cv_outer = StratifiedKFold(n_splits=N_FOLDS_OUTER, shuffle=True, random_state=RANDOM_STATE)
cv_inner = StratifiedKFold(n_splits=N_FOLDS_INNER, shuffle=True, random_state=RANDOM_STATE)


def calcular_metricas(y_true, y_pred, y_score):
    return {
        'Balanced Acc': balanced_accuracy_score(y_true, y_pred),
        'F1 macro': f1_score(y_true, y_pred, average='macro'),
        'Recall tb': recall_score(y_true, y_pred, labels=[2], average=None)[0],
        'AUC-ROC macro': np.mean([
            roc_auc_score((y_true == c).astype(int), y_score[:, c])
            for c in range(len(CLASS_NAMES))
        ]),
        'AP tb': average_precision_score((y_true == 2).astype(int), y_score[:, 2]),
    }


def ajustar_modelo(nombre, modelo, X, y):
    if nombre == 'XGBoost' and ESTRATEGIA == 'class_weight':
```

```
        modelo.fit(X, y, clf__sample_weight=compute_sample_weight('balanced', y))
    else:
        modelo.fit(X, y)
    return modelo


oof_pred = {n: np.full(len(y), -1, dtype=int) for n in list(MODELOS) + ['Dummy']}
oof_score = {n: np.full((len(y), len(CLASS_NAMES)), np.nan) for n in list(MODELOS) + ['Dummy']}
best_params_fold = {n: [] for n in MODELOS}
fold_rows = {n: [] for n in MODELOS}
t_model = {n: 0.0 for n in MODELOS}
selected_per_fold = {}
inner_mean_by_model = {}
proc_pred = np.full(len(y), -1, dtype=int)
proc_score = np.full((len(y), len(CLASS_NAMES)), np.nan)

for fold, (tr_idx, va_idx) in enumerate(cv_outer.split(X, y), start=1):
    # Selector del fold externo (una sola vez).
    selector_out = clone(selector_base).fit(X[tr_idx], y[tr_idx])
    X_tr = selector_out.transform(X[tr_idx])
    X_va = selector_out.transform(X[va_idx])
    y_tr, y_va = y[tr_idx], y[va_idx]

    # Selectores de los folds internos (una sola vez por fold interno).
    inner_cache = []
    for in_tr, in_va in cv_inner.split(X[tr_idx], y[tr_idx]):
        gi_tr, gi_va = tr_idx[in_tr], tr_idx[in_va]
        sel_in = clone(selector_base).fit(X[gi_tr], y[gi_tr])
        inner_cache.append({
            'X_tr': sel_in.transform(X[gi_tr]),
            'X_va': sel_in.transform(X[gi_va]),
            'y_tr': y[gi_tr],
            'y_va': y[gi_va],
        })

    fold_inner_mean = {}
    fold_pred = {}
    fold_score = {}

    for nombre, modelo_base in MODELOS.items():
        t0 = time.time()
        combos = list(ParameterGrid(GRIDS[nombre]))
        scores = np.zeros((len(combos), len(inner_cache)))

        for k_in, data in enumerate(inner_cache):
            for k, params in enumerate(combos):
                modelo = clone(modelo_base).set_params(**params)
                ajustar_modelo(nombre, modelo, data['X_tr'], data['y_tr'])
                scores[k, k_in] = balanced_accuracy_score(
                    data['y_va'], modelo.predict(data['X_va'])
                )

        mean_scores = scores.mean(axis=1)
        best_k = int(np.argmax(mean_scores))
        best_params = combos[best_k]
        best_params_fold[nombre].append(best_params)
        fold_inner_mean[nombre] = float(mean_scores[best_k])

        modelo = clone(modelo_base).set_params(**best_params)
        ajustar_modelo(nombre, modelo, X_tr, y_tr)
        y_pred = modelo.predict(X_va)
        y_score = (modelo.predict_proba(X_va) if hasattr(modelo, 'predict_proba')
                   else modelo.decision_function(X_va))

        oof_pred[nombre][va_idx] = y_pred
        oof_score[nombre][va_idx] = y_score
        fold_pred[nombre] = y_pred
        fold_score[nombre] = y_score

        fold_rows[nombre].append({
            'Modelo': nombre, 'Fold': fold,
            **calcular_metricas(y_va, y_pred, y_score),
            'Mejores params': json.dumps(best_params, sort_keys=True),
        })
        t_model[nombre] += time.time() - t0
```

```
        print(f'fold {fold}/{N_FOLDS_OUTER}  {nombre:22s}  '
              f'inner={mean_scores[best_k]:.4f}')

    best_model = max(fold_inner_mean, key=fold_inner_mean.get)
    selected_per_fold[fold] = best_model
    inner_mean_by_model[fold] = fold_inner_mean
    proc_pred[va_idx] = fold_pred[best_model]
    proc_score[va_idx] = fold_score[best_model]

# Resumen por modelo (OOF).
filas_fold = []
filas_resumen = []
mejores_params = {}

for nombre in MODELOS:
    filas_fold.extend(fold_rows[nombre])
    df_fold = pd.DataFrame(fold_rows[nombre])
    oof_m = calcular_metricas(y, oof_pred[nombre], oof_score[nombre])
    resumen = {
        'Modelo': nombre,
        **{f'{k}_oof': v for k, v in oof_m.items()},
        'Tiempo_s': t_model[nombre],
    }
    for k in ['Balanced Acc', 'F1 macro', 'Recall tb', 'AUC-ROC macro', 'AP tb']:
        resumen[f'{k}_media'] = df_fold[k].mean()
        resumen[f'{k}_std'] = df_fold[k].std()
    filas_resumen.append(resumen)

    clave = Counter(json.dumps(p, sort_keys=True) for p in best_params_fold[nombre]).most_common(1)[0]
[0]
    mejores_params[nombre] = json.loads(clave)

    print(f'{nombre:22s}  OOF BalAcc={oof_m["Balanced Acc"]:.4f}  '
          f'F1={oof_m["F1 macro"]:.4f}  Recall tb={oof_m["Recall tb"]:.4f}  '
          f'({t_model[nombre]:.1f}s)')

# Guardado intermedio: preserva los 5 modelos si falla el Dummy o el procedimiento.
pd.DataFrame({'y_true': y, **{n: oof_pred[n] for n in MODELOS}}).to_csv(
    DATA_DIR / '08_oof_predicciones.csv', index=False,
)
np.savez(DATA_DIR / '08_oof_scores.npz', **{n: oof_score[n] for n in MODELOS})
pd.DataFrame(filas_fold).to_csv(DATA_DIR / '08_cv_por_fold.csv', index=False)
with open(DATA_DIR / '08_mejores_params.json', 'w', encoding='utf-8') as f:
    json.dump(mejores_params, f, ensure_ascii=False, indent=2)
print('Resultados OOF de los 5 modelos guardados (intermedio).')

# Dummy: baseline trivial, sin selector ni grid.
dummy = DummyClassifier(strategy='most_frequent')
t0 = time.time()
dummy_rows = []
for fold, (tr_idx, va_idx) in enumerate(cv_outer.split(X, y), start=1):
    dummy.fit(X[tr_idx], y[tr_idx])
    y_pred = dummy.predict(X[va_idx])
    y_score = dummy.predict_proba(X[va_idx])
    oof_pred['Dummy'][va_idx] = y_pred
    oof_score['Dummy'][va_idx] = y_score
    fila = {'Modelo': 'Dummy', 'Fold': fold,
            **calcular_metricas(y[va_idx], y_pred, y_score),
            'Mejores params': '{}'}
    dummy_rows.append(fila)
    filas_fold.append(fila)

df_dummy = pd.DataFrame(dummy_rows)
oof_m = calcular_metricas(y, oof_pred['Dummy'], oof_score['Dummy'])
filas_resumen.append({
    'Modelo': 'Dummy', **{f'{k}_oof': v for k, v in oof_m.items()},
    'Tiempo_s': time.time() - t0,
    **{f'{k}_media': df_dummy[k].mean() for k in ['Balanced Acc', 'F1 macro', 'Recall tb', 'AUC-ROC
macro', 'AP tb']},
    **{f'{k}_std': df_dummy[k].std() for k in ['Balanced Acc', 'F1 macro', 'Recall tb', 'AUC-ROC
macro', 'AP tb']},
})
mejores_params['Dummy'] = {}
print(f'{"Dummy":22s}  OOF BalAcc={oof_m["Balanced Acc"]:.4f}')
```

```python
# Estimación OOF del procedimiento completo.
proc_metrics = calcular_metricas(y, proc_pred, proc_score)
filas_resumen.append({
    'Modelo': 'Procedimiento', **{f'{k}_oof': v for k, v in proc_metrics.items()},
    'Tiempo_s': np.nan,
    **{f'{k}_media': np.nan for k in ['Balanced Acc', 'F1 macro', 'Recall tb', 'AUC-ROC macro', 'AP
tb']},
    **{f'{k}_std': np.nan for k in ['Balanced Acc', 'F1 macro', 'Recall tb', 'AUC-ROC macro', 'AP
tb']},
})

resumen_oof = pd.DataFrame(filas_resumen).sort_values(
    ['Balanced Acc_oof', 'Recall tb_oof', 'AP tb_oof'], ascending=False,
).reset_index(drop=True)
por_fold = pd.DataFrame(filas_fold)

print()
print('Resumen out-of-fold (8274 imágenes):')
print(resumen_oof.round(4).to_string(index=False))
print()
print('Modelo seleccionado en cada fold externo:', selected_per_fold)
print(f'Procedimiento (OOF): BalAcc={proc_metrics["Balanced Acc"]:.4f}, '
      f'F1={proc_metrics["F1 macro"]:.4f}, Recall tb={proc_metrics["Recall tb"]:.4f}')

resumen_oof.to_csv(DATA_DIR / '08_cv_resumen.csv', index=False)
por_fold.to_csv(DATA_DIR / '08_cv_por_fold.csv', index=False)
with open(DATA_DIR / '08_mejores_params.json', 'w', encoding='utf-8') as f:
    json.dump(mejores_params, f, ensure_ascii=False, indent=2)
with open(DATA_DIR / '08_procedimiento.json', 'w', encoding='utf-8') as f:
    json.dump({
        'selected_per_fold': selected_per_fold,
        'metricas_oof': {k: float(v) for k, v in proc_metrics.items()},
    }, f, ensure_ascii=False, indent=2)

pd.DataFrame({'y_true': y, **{n: oof_pred[n] for n in MODELOS}}).to_csv(
    DATA_DIR / '08_oof_predicciones.csv', index=False,
)
pd.DataFrame({'y_true': y, 'procedimiento': proc_pred}).to_csv(
    DATA_DIR / '08_oof_procedimiento.csv', index=False,
)
np.savez(DATA_DIR / '08_oof_scores.npz', **{n: oof_score[n] for n in MODELOS})

print()
print('Predicciones y scores OOF guardados.')
```

```
fold 1/5  Logistic Regression     inner=0.7616
fold 1/5  SVM                     inner=0.7547
fold 1/5  Decision Tree           inner=0.6221
fold 1/5  Random Forest           inner=0.6969
fold 1/5  XGBoost                 inner=0.7589
fold 2/5  Logistic Regression     inner=0.7688
fold 2/5  SVM                     inner=0.7532
fold 2/5  Decision Tree           inner=0.6404
fold 2/5  Random Forest           inner=0.7057
fold 2/5  XGBoost                 inner=0.7661
fold 3/5  Logistic Regression     inner=0.7590
fold 3/5  SVM                     inner=0.7419
fold 3/5  Decision Tree           inner=0.6398
fold 3/5  Random Forest           inner=0.7064
fold 3/5  XGBoost                 inner=0.7572
fold 4/5  Logistic Regression     inner=0.7666
fold 4/5  SVM                     inner=0.7559
fold 4/5  Decision Tree           inner=0.6494
fold 4/5  Random Forest           inner=0.7037
fold 4/5  XGBoost                 inner=0.7634
fold 5/5  Logistic Regression     inner=0.7649
fold 5/5  SVM                     inner=0.7500
fold 5/5  Decision Tree           inner=0.6324
fold 5/5  Random Forest           inner=0.6972
fold 5/5  XGBoost                 inner=0.7636
Logistic Regression     OOF BalAcc=0.7714  F1=0.7497  Recall tb=0.5813  (176.5s)
SVM                     OOF BalAcc=0.7652  F1=0.7441  Recall tb=0.5687  (632.6s)
Decision Tree           OOF BalAcc=0.6389  F1=0.6137  Recall tb=0.4662  (355.2s)
Random Forest           OOF BalAcc=0.7080  F1=0.7215  Recall tb=0.3225  (9193.7s)
XGBoost                 OOF BalAcc=0.7730  F1=0.7578  Recall tb=0.5675  (2457.6s)
Resultados OOF de los 5 modelos guardados (intermedio).
Dummy                   OOF BalAcc=0.3333

Resumen out-of-fold (8274 imágenes):
             Modelo  Balanced Acc_oof  F1 macro_oof  Recall tb_oof  AUC-ROC macro_oof  AP tb_oof  Tiempo_s
Balanced Acc_media  Balanced Acc_std  F1 macro_media  F1 macro_std  Recall tb_media  Recall tb_std  AUC-
ROC macro_media  AUC-ROC macro_std  AP tb_media  AP tb_std
           XGBoost            0.7730        0.7578         0.5675             0.9361     0.4967 2457.5718
0.7730            0.0113          0.7579        0.0125           0.5675         0.0195
0.9363             0.0076       0.5030     0.0407
Logistic Regression            0.7714        0.7497         0.5812             0.9316     0.4843  176.4700
0.7714            0.0042          0.7497        0.0051           0.5812         0.0099
0.9319             0.0053       0.4880     0.0271
      Procedimiento            0.7714        0.7497         0.5812             0.9316     0.4843       NaN
NaN               NaN             NaN           NaN              NaN            NaN                  NaN
NaN          NaN        NaN
              SVM              0.7652        0.7441         0.5688             0.9083     0.3758  632.5899
0.7652            0.0138          0.7441        0.0117           0.5688         0.0412
0.9083             0.0095       0.3827     0.0303
      Random Forest            0.7080        0.7215         0.3225             0.9227     0.4154 9193.6611
0.7080            0.0172          0.7214        0.0200           0.3225         0.0418
0.9227             0.0071       0.4231     0.0479
      Decision Tree            0.6389        0.6137         0.4662             0.8045     0.1940  355.2376
0.6389            0.0240          0.6136        0.0193           0.4662         0.0501
0.8058             0.0229       0.2007     0.0360
             Dummy             0.3333        0.2101         0.0000             0.5000     0.0969    1.5067
0.3333            0.0000          0.2101        0.0000           0.0000         0.0000
0.5000             0.0000       0.0969     0.0000

Modelo seleccionado en cada fold externo: {1: 'Logistic Regression', 2: 'Logistic Regression', 3:
'Logistic Regression', 4: 'Logistic Regression', 5: 'Logistic Regression'}
Procedimiento (OOF): BalAcc=0.7714, F1=0.7497, Recall tb=0.5813

Predicciones y scores OOF guardados.
```

4. Selección y persistencia de los modelos finales

In [ ]:

```python
# Selección por OOF: balanced accuracy > recall tb > AP tb. El Dummy se excluye.
candidatos = resumen_oof[resumen_oof['Modelo'] != 'Dummy'].reset_index(drop=True)
MEJOR_MODELO_CV = candidatos.loc[0, 'Modelo']

print(f'Modelo seleccionado por CV OOF: {MEJOR_MODELO_CV}')
```

```
print(f'  BalAcc OOF : {candidatos.loc[0, "Balanced Acc_oof"]:.4f}')
print(f'  Recall tb  : {candidatos.loc[0, "Recall tb_oof"]:.4f}')
print(f'  AP tb      : {candidatos.loc[0, "AP tb_oof"]:.4f}')

fig, ax = plt.subplots(figsize=(8, 4.5))
orden = candidatos.sort_values('Balanced Acc_oof')
ax.barh(orden['Modelo'], orden['Balanced Acc_oof'],
        xerr=orden['Balanced Acc_std'], capsize=4, color='#2166AC', alpha=0.85)
ax.set_xlabel('Balanced accuracy out-of-fold (8274 imágenes)', fontsize=11)
ax.set_title('Comparación de modelos — predicciones out-of-fold', fontsize=12)
ax.grid(True, axis='x', linestyle='--', alpha=0.4)
fig.tight_layout()
plt.show()

# Modelos finales: selector + modelo, ajustados con las 8274 imágenes.
manifest = {
    'estrategia': ESTRATEGIA,
    'selector': {'metodo': METHOD_SEL, 'n_features': N_SEL},
    'modelos': {},
    'seleccionado_cv': MEJOR_MODELO_CV,
    'mejores_params': mejores_params,
}

for nombre in MODELOS:
    base = clone(MODELOS[nombre]).set_params(**mejores_params[nombre])
    selector = build_selector(
    METHOD_SEL, N_SEL, random_state=RANDOM_STATE, n_jobs=-1,
)
    Final = ImbPipeline if isinstance(base, ImbPipeline) else Pipeline
    final = Final([('selector', selector)] + list(base.steps))

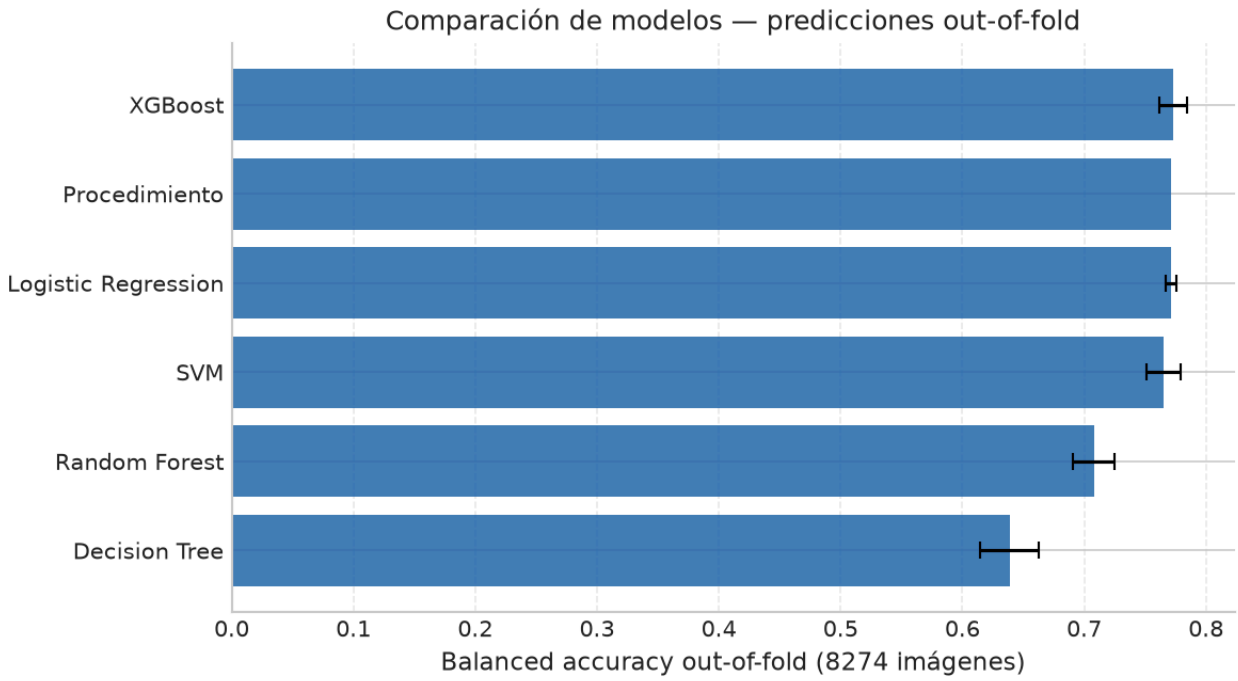
    if nombre == 'XGBoost' and ESTRATEGIA == 'class_weight':
        final.fit(X, y, clf__sample_weight=compute_sample_weight('balanced', y))
    else:
        final.fit(X, y)

    archivo = f'08_{MODEL_SLUGS[nombre]}.joblib'
    ruta = MODELS_DIR / archivo
    joblib.dump(final, ruta)
    manifest['modelos'][nombre] = archivo
    print(f'{nombre:22s} -> {archivo}  ({ruta.stat().st_size / 1024 / 1024:.1f} MB)')

with open(MODELS_DIR / '08_manifest.json', 'w', encoding='utf-8') as f:
    json.dump(manifest, f, ensure_ascii=False, indent=2)

print()
print(f'Manifiesto guardado en: {(MODELS_DIR / "08_manifest.json").resolve()}')
print(f'Modelo seleccionado: {MEJOR_MODELO_CV}')
```

```
Modelo seleccionado por CV OOF: XGBoost
  BalAcc OOF : 0.7730
  Recall tb  : 0.5675
  AP tb      : 0.4967
```




```
Logistic Regression    -> 08_logistic_regression.joblib  (0.1 MB)
SVM                    -> 08_svm.joblib  (52.6 MB)
Decision Tree          -> 08_decision_tree.joblib  (0.0 MB)
Random Forest          -> 08_random_forest.joblib  (12.2 MB)
XGBoost                -> 08_xgboost.joblib  (0.8 MB)

Manifiesto guardado en: C:\Users\estra\Desktop\Tuberculosis\cad-tbx11k\models\08_manifest.json
Modelo seleccionado: XGBoost
```

```
The Kernel crashed while executing code in the current cell or a previous cell.

Please review the code in the cell(s) to identify a possible cause of the failure.

Click <a href='https://aka.ms/vscodeJupyterKernelCrash'>here</a> for more info.

View Jupyter <a href='command:jupyter.viewOutput'>log</a> for further details.
```

Conclusiones

Se usaron **todas las imágenes** (8274). No hay hold-out ni test separado: la estimación es la **predicción out-of-fold** de una CV externa 5-fold.

En cada fold externo, el selector de características se ajustó **solo con el train** y la CV interna 3-fold ajustó los hiperparámetros. Las predicciones out-of-fold cubren las 8274 imágenes exactamente una vez.

El modelo final de cada algoritmo (selector + modelo) se reajustó con las 8274 imágenes. Los artefactos `08_oof_predicciones.csv` y `08_oof_scores.npz` son la base de la evaluación del notebook 09.

Siguiente etapa: `09_Evaluacion_Comparativa.ipynb`

09 — Evaluación y Comparación de Modelos

Pipeline CAD-TBX11K | Diagnóstico Asistido por Computador en Tuberculosis

Este notebook evalúa los cinco modelos con las **predicciones out-of-fold (OOF)** de la validación cruzada del notebook 08 sobre las **8274 imágenes**. No hay hold-out ni test separado: la estimación es la CV OOF.

Métricas

- Accuracy, balanced accuracy, precision, recall y F1 (macro y weighted).
- Métricas por clase (`health` , `sick` , `tb`).
- AUC-ROC por clase (One-vs-Rest) y **Average Precision (AP)** de `tb` .
- Intervalos de confianza por bootstrap y McNemar con Holm.
- Calibración de `tb` (Brier + curva).

Protocolo

1. Cargar las predicciones y scores OOF del notebook 08.
2. Evaluar las 8274 imágenes (cada una predicha exactamente una vez).
3. Comparar modelos y analizar la incertidumbre.
4. Recomendar el modelo elegido por la CV del 08.

Flujo

```
data/08_oof_predicciones.csv
data/08_oof_scores.npz
data/08_manifest.json
        |
        v
 Metricas globales + por clase + AUC/AP
        |
        v
 Matrices de confusion + curvas ROC/AP
        |
        v
 Bootstrap + McNemar (global y tb) + calibracion
        |
        v
 Recomendacion final
```

0. Configuración del entorno

In [1]:
```
# Se configura el path del proyecto y se importan las librerías de evaluación.
# Referencias:
#   metricas: https://scikit-learn.org/stable/modules/model_evaluation.html
#   calibracion: https://scikit-learn.org/stable/modules/calibration.html

import sys, os, warnings, json
warnings.filterwarnings('ignore')

from pathlib import Path
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from scipy.stats import chi2

from sklearn.metrics import (
    accuracy_score,
    balanced_accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    ConfusionMatrixDisplay,
    confusion_matrix,
    roc_curve,
    auc,
    precision_recall_curve,
    average_precision_score,
    roc_auc_score,
    brier_score_loss,
)
from sklearn.calibration import calibration_curve

PROJECT_ROOT = os.path.abspath(os.path.join(os.getcwd(), '..'))
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

from src.visualization import set_publication_style

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

CLASS_NAMES = ['health', 'sick', 'tb']
N_BOOTSTRAP = 1000

DATA_DIR    = Path('..') / 'data'
REPORTS_DIR = Path('..') / 'reports'
REPORTS_DIR.mkdir(exist_ok=True)

set_publication_style()
print('Entorno configurado correctamente.')
```

```
Entorno configurado correctamente.
```

1. Carga de las predicciones out-of-fold

In [2]:

```
# Se cargan las predicciones y scores OOF del notebook 08.
# Cada imagen fue predicha exactamente una vez por el modelo de su fold externo.
# Referencia: https://numpy.org/doc/stable/reference/generated/numpy.load.html

for archivo in ['08_oof_predicciones.csv', '08_oof_scores.npz', '08_manifest.json']:
    if not (DATA_DIR / archivo).exists():
        raise FileNotFoundError(
            f'No se encontró {archivo}. Ejecuta primero el notebook 08.'
        )

oof = pd.read_csv(DATA_DIR / '08_oof_predicciones.csv')
score_data = np.load(DATA_DIR / '08_oof_scores.npz')

y_true = oof['y_true'].to_numpy()
MODELOS = [c for c in oof.columns if c not in ('y_true', 'Dummy')]
predicciones = {n: oof[n].to_numpy() for n in MODELOS}
puntuaciones = {n: score_data[n] for n in MODELOS}

with open(DATA_DIR / '08_manifest.json', encoding='utf-8') as f:
    manifest = json.load(f)

print(f'Imágenes OOF : {len(y_true)}')
print('Distribución:')
for clase, conteo in zip(*np.unique(y_true, return_counts=True)):
    print(f'  {CLASS_NAMES[clase]:6s} (label={clase}): {conteo} muestras')
print()
```

```
print('Modelos:', MODELOS)
print(f'Modelo seleccionado por CV: {manifest.get("seleccionado_cv")}')
print(f'Estrategia : {manifest.get("estrategia")}')
print(f'Selector   : {manifest.get("selector")}')
```

```
Imágenes OOF : 8256
Distribución:
  health (label=0): 3800 muestras
  sick   (label=1): 3656 muestras
  tb     (label=2): 800 muestras

Modelos: ['Logistic Regression', 'SVM', 'Decision Tree', 'Random Forest', 'XGBoost']
Modelo seleccionado por CV: XGBoost
Estrategia : class_weight
Selector   : {'metodo': 'MI', 'n_features': 2000}
```

2. Comparación global

In [3]:

```
# Métricas globales sobre las 8274 predicciones OOF.
# AP = Average Precision (average_precision_score); no es el área trapezoidal de la PR.

filas = []
for nombre in MODELOS:
    y_pred = predicciones[nombre]
    y_score = puntuaciones[nombre]
    filas.append({
        'Modelo': nombre,
        'Accuracy': accuracy_score(y_true, y_pred),
        'Balanced Acc': balanced_accuracy_score(y_true, y_pred),
        'Precision macro': precision_score(y_true, y_pred, average='macro', zero_division=0),
        'Recall macro': recall_score(y_true, y_pred, average='macro', zero_division=0),
        'F1 macro': f1_score(y_true, y_pred, average='macro', zero_division=0),
        'Precision weighted': precision_score(y_true, y_pred, average='weighted', zero_division=0),
        'Recall weighted': recall_score(y_true, y_pred, average='weighted', zero_division=0),
        'F1 weighted': f1_score(y_true, y_pred, average='weighted', zero_division=0),
        'Recall tb': recall_score(y_true, y_pred, labels=[2], average=None, zero_division=0)[0],
        'AUC-ROC macro': np.mean([
            roc_auc_score((y_true == c).astype(int), y_score[:, c])
            for c in range(len(CLASS_NAMES))
        ]),
        'AP tb': average_precision_score((y_true == 2).astype(int), y_score[:, 2]),
    })

resultados_globales = pd.DataFrame(filas).sort_values(
    ['Balanced Acc', 'F1 macro'], ascending=False,
).reset_index(drop=True)

print('Comparación global out-of-fold (8274 imágenes):')
print(resultados_globales.round(4).to_string(index=False))

resultados_globales.to_csv(DATA_DIR / '09_resultados_globales.csv', index=False)
print()
print(f'Guardado en: {(DATA_DIR / "09_resultados_globales.csv").resolve()}')
```

```
Comparación global out-of-fold (8274 imágenes):
             Modelo  Accuracy  Balanced Acc  Precision macro  Recall macro  F1 macro  Precision weighted
Recall weighted  F1 weighted  Recall tb  AUC-ROC macro  AP tb
            XGBoost    0.8465        0.7730           0.7490        0.7730    0.7578              0.8598
0.8465       0.8519     0.5675         0.9361 0.4967
Logistic Regression    0.8395        0.7714           0.7405        0.7714    0.7497              0.8620
0.8395       0.8484     0.5812         0.9316 0.4843
                SVM    0.8358        0.7652           0.7359        0.7652    0.7441              0.8585
0.8358       0.8443     0.5688         0.9083 0.3758
      Random Forest    0.8451        0.7080           0.7516        0.7080    0.7215              0.8323
0.8451       0.8360     0.3225         0.9227 0.4154
      Decision Tree    0.7009        0.6389           0.6247        0.6389    0.6137              0.7694
0.7009       0.7269     0.4662         0.8045 0.1940

Guardado en: C:\Users\estra\Desktop\Tuberculosis\cad-tbx11k\data\09_resultados_globales.csv
```

3. Métricas por clase

In [4]:
```
# Precision, recall y F1 por clase, más AUC-ROC One-vs-Rest.

filas = []
for nombre in MODELOS:
    y_pred = predicciones[nombre]
    y_score = puntuaciones[nombre]
    for k, clase in enumerate(CLASS_NAMES):
        filas.append({
            'Modelo': nombre,
            'Clase': clase,
            'Precision': precision_score(y_true, y_pred, labels=[k], average=None, zero_division=0)[0],
            'Recall': recall_score(y_true, y_pred, labels=[k], average=None, zero_division=0)[0],
            'F1': f1_score(y_true, y_pred, labels=[k], average=None, zero_division=0)[0],
            'AUC-ROC (OvR)': roc_auc_score((y_true == k).astype(int), y_score[:, k]),
        })

resultados_clase = pd.DataFrame(filas)
print('Métricas por clase:')
print(resultados_clase.round(4).to_string(index=False))
resultados_clase.to_csv(DATA_DIR / '09_resultados_por_clase.csv', index=False)
print()
print(f'Guardado en: {(DATA_DIR / "09_resultados_por_clase.csv").resolve()}')
```

```
Métricas por clase:
             Modelo  Clase  Precision  Recall     F1  AUC-ROC (OvR)
Logistic Regression health     0.9232  0.9074 0.9152         0.9778
Logistic Regression   sick     0.8998  0.8255 0.8611         0.9575
Logistic Regression     tb     0.3985  0.5812 0.4728         0.8595
                SVM health     0.9125  0.9171 0.9148         0.9782
                SVM   sick     0.9046  0.8096 0.8545         0.9540
                SVM     tb     0.3906  0.5688 0.4631         0.7927
      Decision Tree health     0.8265  0.7734 0.7991         0.8823
      Decision Tree   sick     0.8311  0.6770 0.7462         0.8628
      Decision Tree     tb     0.2166  0.4662 0.2958         0.6684
      Random Forest health     0.8735  0.9197 0.8960         0.9675
      Random Forest   sick     0.8568  0.8818 0.8691         0.9518
      Random Forest     tb     0.5244  0.3225 0.3994         0.8487
            XGBoost health     0.9132  0.9166 0.9149         0.9773
            XGBoost   sick     0.8969  0.8348 0.8647         0.9581
            XGBoost     tb     0.4370  0.5675 0.4937         0.8731

Guardado en: C:\Users\estra\Desktop\Tuberculosis\cad-tbx11k\data\09_resultados_por_clase.csv
```

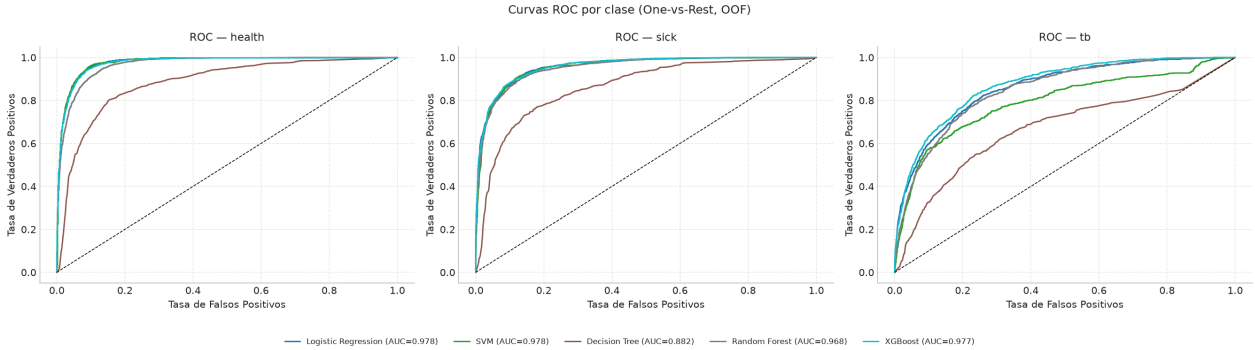
4. AUC-ROC por clase

In [5]:
```
# Curvas ROC One-vs-Rest para las 8274 predicciones OOF.

colores = plt.cm.tab10(np.linspace(0, 1, len(MODELOS)))
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for ax, (k, clase) in zip(axes, enumerate(CLASS_NAMES)):
    for color, nombre in zip(colores, MODELOS):
        y_bin = (y_true == k).astype(int)
        fpr, tpr, _ = roc_curve(y_bin, puntuaciones[nombre][:, k])
        ax.plot(fpr, tpr, color=color, lw=1.4,
                label=f'{nombre} (AUC={auc(fpr, tpr):.3f})')
    ax.plot([0, 1], [0, 1], 'k--', lw=0.8)
    ax.set_xlabel('Tasa de Falsos Positivos', fontsize=10)
    ax.set_ylabel('Tasa de Verdaderos Positivos', fontsize=10)
    ax.set_title(f'ROC — {clase}', fontsize=11)
    ax.grid(True, linestyle='--', alpha=0.4)

handles, labels = axes[0].get_legend_handles_labels()
fig.legend(handles, labels, loc='lower center', ncol=5, fontsize=8, frameon=False)
fig.suptitle('Curvas ROC por clase (One-vs-Rest, OOF)', fontsize=12)
fig.tight_layout(rect=[0, 0.08, 1, 1])
fig.savefig(REPORTS_DIR / '09_roc_por_clase.png', dpi=300, bbox_inches='tight')
plt.show()
```



5. Curvas Average Precision (tb)

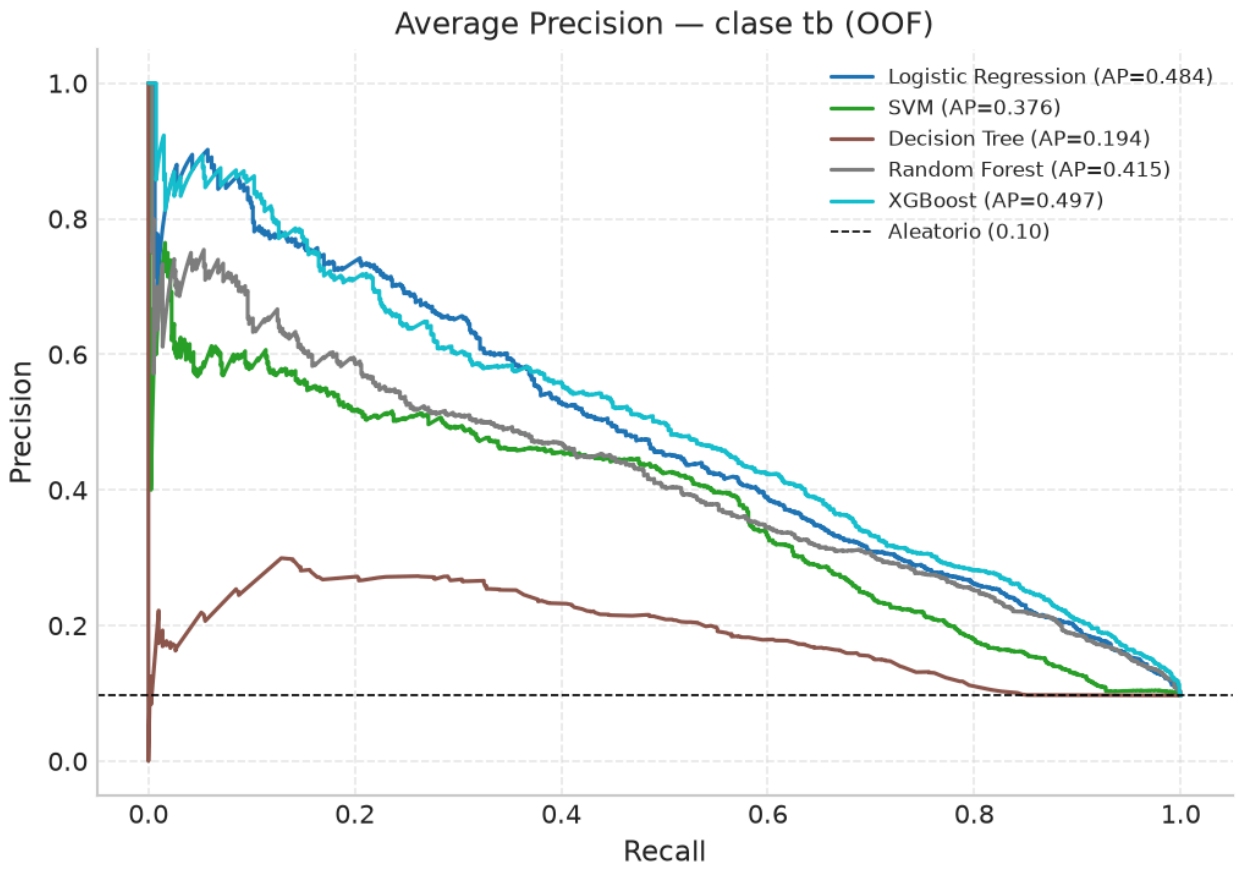
In [6]:

```
# AP (average precision) es más informativa que ROC-AUC con clase positiva minoritaria.
# Es AP, no el área trapezoidal de la curva PR.
# Referencia: https://scikit-
learn.org/stable/modules/generated/sklearn.metrics.average_precision_score.html

fig, ax = plt.subplots(figsize=(7, 5))
y_tb = (y_true == 2).astype(int)

for color, nombre in zip(colores, MODELOS):
    precision, recall, _ = precision_recall_curve(y_tb, puntuaciones[nombre][:, 2])
    ap = average_precision_score(y_tb, puntuaciones[nombre][:, 2])
    ax.plot(recall, precision, color=color, lw=1.5,
            label=f'{nombre} (AP={ap:.3f})')

ax.axhline(y_tb.mean(), color='k', linestyle='--', lw=0.8,
           label=f'Aleatorio ({y_tb.mean():.2f})')
ax.set_xlabel('Recall', fontsize=11)
ax.set_ylabel('Precision', fontsize=11)
ax.set_title('Average Precision — clase tb (OOF)', fontsize=12)
ax.legend(fontsize=8)
ax.grid(True, linestyle='--', alpha=0.4)
fig.tight_layout()
fig.savefig(REPORTS_DIR / '09_ap_tb.png', dpi=300, bbox_inches='tight')
plt.show()
```



6. Matrices de confusión y errores

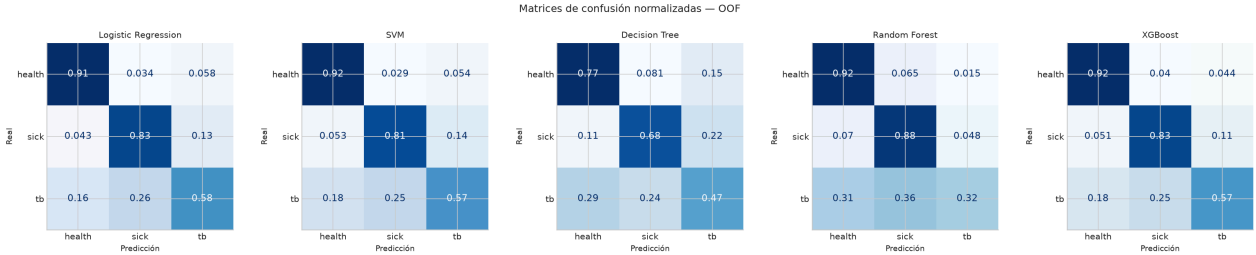
In [7]:

```python
# Matrices normalizadas por fila (recall por clase) y conteos de confusiones tb/sick.

fig, axes = plt.subplots(1, len(MODELOS), figsize=(4.2 * len(MODELOS), 4))
for ax, nombre in zip(axes, MODELOS):
    ConfusionMatrixDisplay.from_predictions(
        y_true, predicciones[nombre],
        display_labels=CLASS_NAMES,
        normalize='true', colorbar=False, cmap='Blues', ax=ax,
    )
    ax.set_title(nombre, fontsize=10)
    ax.set_xlabel('Predicción', fontsize=9)
    ax.set_ylabel('Real', fontsize=9)

fig.suptitle('Matrices de confusión normalizadas — OOF', fontsize=12, y=1.02)
fig.tight_layout()
fig.savefig(REPORTS_DIR / '09_matrices_confusion.png', dpi=300, bbox_inches='tight')
plt.show()

print('Confusiones principales (conteos absolutos):')
print(f'{"Modelo":22s}  tb->sick  sick->tb  tb->health  health->tb')
for nombre in MODELOS:
    cm = confusion_matrix(y_true, predicciones[nombre])
    print(f'{nombre:22s}  {cm[2, 1]:7d}  {cm[1, 2]:7d}  {cm[2, 0]:10d}  {cm[0, 2]:9d}')
```



```
Confusiones principales (conteos absolutos):
Modelo                 tb->sick  sick->tb  tb->health  health->tb
Logistic Regression         205       481         130         221
SVM                         203       504         142         206
Decision Tree               195       796         232         553
Random Forest               291       177         251          57
XGBoost                     200       419         146         166
```

7. Comparación estadística

> Los p-valores de McNemar se corrigen con **Holm**. Se reportan el test global y uno restringido a los casos de `tb`.

In [8]:

```python
# Bootstrap estratificado para balanced accuracy y recall de tb; McNemar global y de tb.

def bootstrap_ci(y_true, y_pred, metric_fn, n_boot=N_BOOTSTRAP, seed=RANDOM_STATE):
    rng = np.random.RandomState(seed)
    clases = np.unique(y_true)
    idx = {c: np.where(y_true == c)[0] for c in clases}
    valores = []
    for _ in range(n_boot):
        muestra = np.concatenate([
            rng.choice(idx[c], size=len(idx[c]), replace=True) for c in clases
        ])
        valores.append(metric_fn(y_true[muestra], y_pred[muestra]))
    return np.percentile(valores, [2.5, 97.5])


filas = []
for nombre in MODELOS:
    y_pred = predicciones[nombre]
    bacc_lo, bacc_hi = bootstrap_ci(y_true, y_pred, balanced_accuracy_score)
    rec_lo, rec_hi = bootstrap_ci(
        y_true, y_pred,
        lambda yt, yp: recall_score(yt, yp, labels=[2], average=None)[0],
    )
    filas.append({
        'Modelo': nombre,
        'BalAcc': balanced_accuracy_score(y_true, y_pred),
        'BalAcc IC95 inf': bacc_lo,
        'BalAcc IC95 sup': bacc_hi,
        'Recall tb': recall_score(y_true, y_pred, labels=[2], average=None)[0],
        'Recall tb IC95 inf': rec_lo,
        'Recall tb IC95 sup': rec_hi,
    })

intervalos = pd.DataFrame(filas).sort_values('BalAcc', ascending=False)
print('Intervalos de confianza al 95 % (bootstrap estratificado):')
print(intervalos.round(4).to_string(index=False))


def mcnemar(y_true, pred_a, pred_b):
    a_ok = pred_a == y_true
    b_ok = pred_b == y_true
    n01 = np.sum(a_ok & ~b_ok)
    n10 = np.sum(~a_ok & b_ok)
    if n01 + n10 == 0:
        return np.nan, np.nan
    stat = (abs(n01 - n10) - 1) ** 2 / (n01 + n10)
    return stat, chi2.sf(stat, 1)


def tabla_mcnemar(y_eval, preds):
    nombres = list(preds)
    pares = []
    for i in range(len(nombres)):
        for j in range(i + 1, len(nombres)):
            _, p = mcnemar(y_eval, preds[nombres[i]], preds[nombres[j]])
            if p != p:
                p = 1.0
```

```
            pares.append({'Modelo A': nombres[i], 'Modelo B': nombres[j], 'p': p})
    m = len(pares)
    orden = sorted(range(m), key=lambda k: pares[k]['p'])
    anterior = 0.0
    for rango, k in enumerate(orden):
        ajustado = min(1.0, (m - rango) * pares[k]['p'])
        ajustado = max(ajustado, anterior)
        anterior = ajustado
        pares[k]['p Holm'] = ajustado
    return pares


pares = tabla_mcnemar(y_true, predicciones)
print()
print('McNemar global (OOF; p crudo y Holm):')
print(f'{"Modelo A":22s} {"Modelo B":22s} {"p":>8s} {"p Holm":>8s} {"significativo":>14s}')
for par in sorted(pares, key=lambda d: d['p']):
    sig = 'sí' if par['p Holm'] < 0.05 else 'no'
    print(f'{par["Modelo A"]:22s} {par["Modelo B"]:22s} '
          f'{par["p"]:8.4f} {par["p Holm"]:8.4f} {sig:>14s}')

mask_tb = y_true == 2
pares_tb = tabla_mcnemar(y_true[mask_tb], {n: p[mask_tb] for n, p in predicciones.items()})
print()
print('McNemar restringido a tb (OOF; p crudo y Holm):')
print(f'{"Modelo A":22s} {"Modelo B":22s} {"p":>8s} {"p Holm":>8s} {"significativo":>14s}')
for par in sorted(pares_tb, key=lambda d: d['p']):
    sig = 'sí' if par['p Holm'] < 0.05 else 'no'
    print(f'{par["Modelo A"]:22s} {par["Modelo B"]:22s} '
          f'{par["p"]:8.4f} {par["p Holm"]:8.4f} {sig:>14s}')
```

```
Intervalos de confianza al 95 % (bootstrap estratificado):
             Modelo  BalAcc  BalAcc IC95 inf  BalAcc IC95 sup  Recall tb  Recall tb IC95 inf  Recall tb
IC95 sup
            XGBoost  0.7730           0.7611           0.7851     0.5675              0.5350
0.6012
Logistic Regression  0.7714           0.7592           0.7835     0.5812              0.5500
0.6150
                SVM  0.7652           0.7525           0.7776     0.5688              0.5337
0.6025
      Random Forest  0.7080           0.6959           0.7203     0.3225              0.2900
0.3563
      Decision Tree  0.6389           0.6262           0.6524     0.4662              0.4312
0.5000

McNemar global (OOF; p crudo y Holm):
Modelo A               Modelo B                     p   p Holm  significativo
Decision Tree          Random Forest           0.0000   0.0000             sí
Decision Tree          XGBoost                 0.0000   0.0000             sí
Logistic Regression    Decision Tree           0.0000   0.0000             sí
SVM                    Decision Tree           0.0000   0.0000             sí
SVM                    XGBoost                 0.0033   0.0196             sí
SVM                    Random Forest           0.0231   0.1154             no
Logistic Regression    XGBoost                 0.0473   0.1893             no
Logistic Regression    SVM                     0.1456   0.4368             no
Logistic Regression    Random Forest           0.1641   0.4368             no
Random Forest          XGBoost                 0.6871   0.6871             no

McNemar restringido a tb (OOF; p crudo y Holm):
Modelo A               Modelo B                     p   p Holm  significativo
Random Forest          XGBoost                 0.0000   0.0000             sí
Logistic Regression    Random Forest           0.0000   0.0000             sí
SVM                    Random Forest           0.0000   0.0000             sí
Decision Tree          Random Forest           0.0000   0.0000             sí
Logistic Regression    Decision Tree           0.0000   0.0000             sí
Decision Tree          XGBoost                 0.0000   0.0000             sí
SVM                    Decision Tree           0.0000   0.0000             sí
Logistic Regression    SVM                     0.2888   0.8665             no
Logistic Regression    XGBoost                 0.4418   0.8835             no
SVM                    XGBoost                 1.0000   1.0000             no
```

8. Calibración de probabilidades (tb)

In [9]:
```python
# Brier score y curva de calibración para tb sobre los scores OOF.
# Solo modelos con predict_proba: LR, DT, RF y XGBoost. SVM no expone probabilidades.

modelos_proba = [n for n in MODELOS if n != 'SVM']
y_tb = (y_true == 2).astype(int)

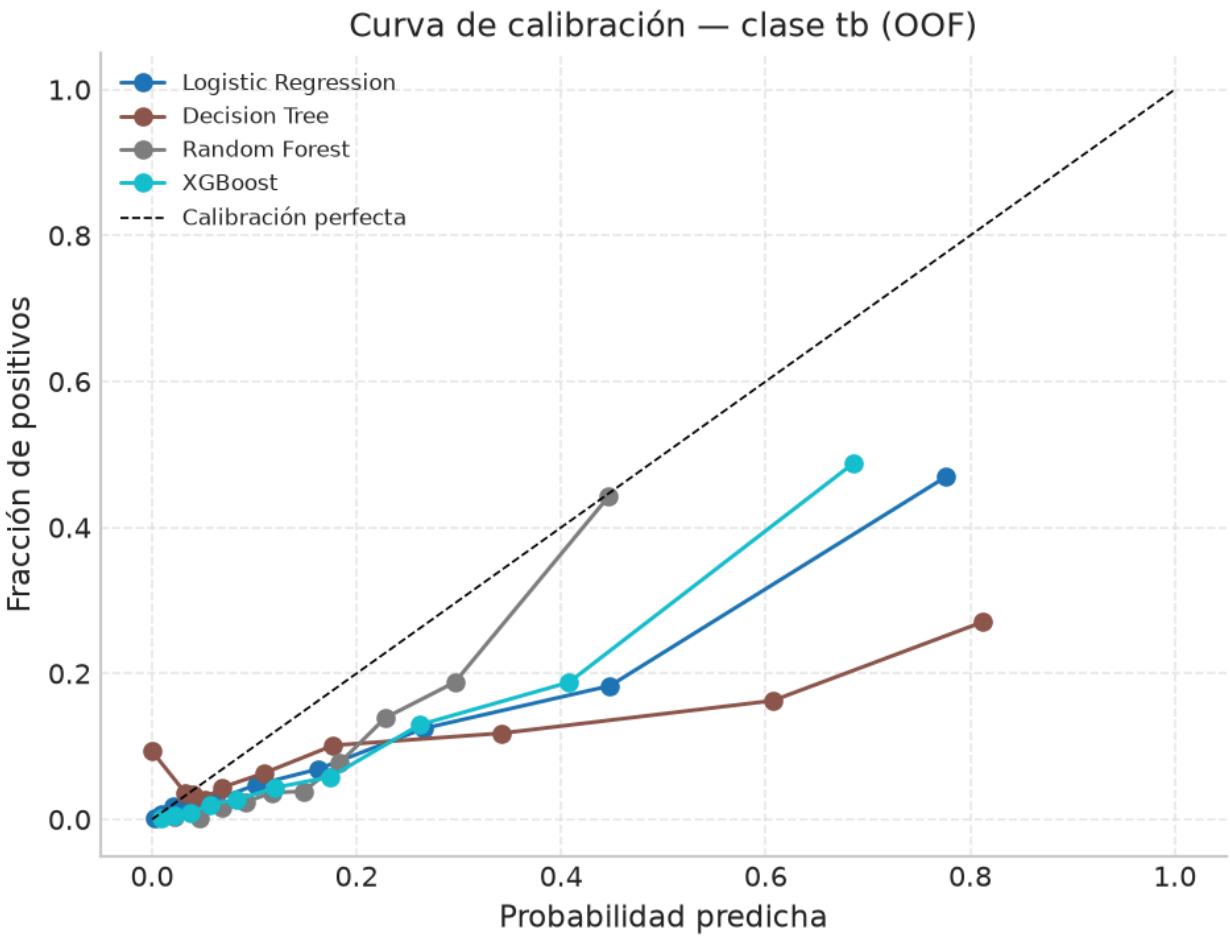
filas_cal = []
for nombre in modelos_proba:
    prob_tb = puntuaciones[nombre][:, 2]
    filas_cal.append({'Modelo': nombre, 'Brier tb': brier_score_loss(y_tb, prob_tb)})

calibracion = pd.DataFrame(filas_cal).sort_values('Brier tb')
print('Brier score para tb (menor es mejor):')
print(calibracion.round(4).to_string(index=False))
calibracion.to_csv(DATA_DIR / '09_calibracion.csv', index=False)

fig, ax = plt.subplots(figsize=(6.5, 5))
colores_modelo = dict(zip(MODELOS, colores))
for nombre in modelos_proba:
    prob_true, prob_pred = calibration_curve(
        y_tb, puntuaciones[nombre][:, 2], n_bins=10, strategy='quantile',
    )
    ax.plot(prob_pred, prob_true, 'o-', color=colores_modelo[nombre], lw=1.4, label=nombre)

ax.plot([0, 1], [0, 1], 'k--', lw=0.8, label='Calibración perfecta')
ax.set_xlabel('Probabilidad predicha', fontsize=11)
ax.set_ylabel('Fracción de positivos', fontsize=11)
ax.set_title('Curva de calibración — clase tb (OOF)', fontsize=12)
ax.legend(fontsize=8)
ax.grid(True, linestyle='--', alpha=0.4)
fig.tight_layout()
fig.savefig(REPORTS_DIR / '09_calibracion_tb.png', dpi=300, bbox_inches='tight')
plt.show()
```

```
Brier score para tb (menor es mejor):
              Modelo  Brier tb
       Random Forest    0.0755
             XGBoost    0.0778
Logistic Regression    0.0859
       Decision Tree    0.1390
```



9. Recomendación final

La selección del modelo se hizo en el notebook 08 con la **CV out-of-fold**. Aquí se reporta el desempeño de esos modelos sobre las mismas 8274 imágenes; el ranking es descriptivo y no se usa para reseleccionar.

In [10]:

```python
# El modelo recomendado es el seleccionado por la CV OOF del 08.
ranking_oof = resultados_globales.sort_values(
    ['Balanced Acc', 'Recall tb', 'AP tb'], ascending=False,
).reset_index(drop=True)

seleccionado_cv = manifest.get('seleccionado_cv')
mejor_oof = ranking_oof.iloc[0]['Modelo']
fila_sel = ranking_oof[ranking_oof['Modelo'] == seleccionado_cv].iloc[0]

print('Ranking OOF (descriptivo):')
print(ranking_oof[['Modelo', 'Balanced Acc', 'Recall tb', 'AP tb']].round(4).to_string(index=False))
print()
print(f'Modelo recomendado (seleccionado por CV OOF en 08): {seleccionado_cv}')
print(f'  BalAcc OOF : {fila_sel["Balanced Acc"]:.4f}')
print(f'  Recall tb  : {fila_sel["Recall tb"]:.4f}')
print(f'  AP tb      : {fila_sel["AP tb"]:.4f}')
print()
print(f'Mejor OOF (descriptivo): {mejor_oof}')


def _p_holm(tabla, a, b):
    par = next((p for p in tabla if {p['Modelo A'], p['Modelo B']} == {a, b}), None)
    return None if par is None else par['p Holm']


p_global = _p_holm(pares, mejor_oof, seleccionado_cv)
p_tb = _p_holm(pares_tb, mejor_oof, seleccionado_cv)
if mejor_oof != seleccionado_cv:
```

```
    print()
    print(f'Nota: {mejor_oof} no fue el elegido por CV. No se cambia la selección.')
if p_global is not None:
    print(f'McNemar global {mejor_oof} vs {seleccionado_cv}: p Holm={p_global:.4f}')
if p_tb is not None:
    print(f'McNemar en tb  {mejor_oof} vs {seleccionado_cv}: p Holm={p_tb:.4f}')

recomendacion = {
    'politica': 'seleccion_por_cv_oof; sin hold-out',
    'modelo_recomendado': seleccionado_cv,
    'mejor_oof': mejor_oof,
    'balanced_accuracy_oof': float(fila_sel['Balanced Acc']),
    'recall_tb_oof': float(fila_sel['Recall tb']),
    'ap_tb_oof': float(fila_sel['AP tb']),
    'mejor_oof_balanced_accuracy': float(ranking_oof.iloc[0]['Balanced Acc']),
    'mcnemar_global_holm_mejor_vs_recomendado': None if p_global is None else float(p_global),
    'mcnemar_tb_holm_mejor_vs_recomendado': None if p_tb is None else float(p_tb),
}
with open(DATA_DIR / '09_recomendacion.json', 'w', encoding='utf-8') as f:
    json.dump(recomendacion, f, ensure_ascii=False, indent=2)

print()
print(f'Recomendación guardada en: {(DATA_DIR / "09_recomendacion.json").resolve()}')
```

```
Ranking OOF (descriptivo):
              Modelo  Balanced Acc  Recall tb  AP tb
             XGBoost        0.7730     0.5675 0.4967
Logistic Regression        0.7714     0.5812 0.4843
                 SVM        0.7652     0.5688 0.3758
       Random Forest        0.7080     0.3225 0.4154
       Decision Tree        0.6389     0.4662 0.1940

Modelo recomendado (seleccionado por CV OOF en 08): XGBoost
  BalAcc OOF : 0.7730
  Recall tb  : 0.5675
  AP tb      : 0.4967

Mejor OOF (descriptivo): XGBoost

Recomendación guardada en: C:\Users\estra\Desktop\Tuberculosis\cad-tbx11k\data\09_recomendacion.json
```

10. Estimación OOF del procedimiento completo

Las métricas por modelo son descriptivas. La estimación menos optimista del **procedimiento completo** (selector + hiperparámetros + selección de modelo) se calculó en el notebook 08: en cada fold externo se eligió el modelo con mejor score interno y se evaluó en el fold externo. Se reporta aquí como referencia.

In [11]:
```
# Estimación OOF del procedimiento completo (sin reutilizar el ranking para reseleccionar).
proc_path = DATA_DIR / '08_procedimiento.json'
if proc_path.exists():
    with open(proc_path, encoding='utf-8') as f:
        proc = json.load(f)
    print('Modelo seleccionado en cada fold externo:', proc['selected_per_fold'])
    print()
    print('Métricas OOF del procedimiento completo:')
    for k, v in proc['metricas_oof'].items():
        print(f'  {k}: {v:.4f}')
else:
    print('No se encontró 08_procedimiento.json.')
```

```
Modelo seleccionado en cada fold externo: {'1': 'Logistic Regression', '2': 'Logistic Regression', '3':
'Logistic Regression', '4': 'Logistic Regression', '5': 'Logistic Regression'}

Métricas OOF del procedimiento completo:
  Balanced Acc: 0.7714
  F1 macro: 0.7497
  Recall tb: 0.5813
  AUC-ROC macro: 0.9316
  AP tb: 0.4843
```

11. Robustez: duplicados perceptuales y CV agrupada

Se recalcula el dHash 8×8 de cada imagen y se cuantifican hashes repetidos, pares, conflictos de etiqueta y pares que cruzan folds. Después se compara la CV estratificada habitual con una **CV agrupada por dHash** usando las mismas 1000 features y los mismos parámetros del notebook 08.

Si la CV agrupada baja, parte del rendimiento venía de casi-copias entre folds; si se mantiene, el agrupamiento no es la causa principal.

In [12]:
```
# dHash 8x8 y CV agrupada.
# Referencias:
#   dHash: https://www.hackerfactor.com/blog/index.php?/archives/529-Kind-of-Like-That.html
#   StratifiedGroupKFold: https://scikit-
learn.org/stable/modules/generated/sklearn.model_selection.StratifiedGroupKFold.html

import cv2
from sklearn.base import clone
from sklearn.model_selection import StratifiedKFold, StratifiedGroupKFold
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import (
    balanced_accuracy_score, f1_score, recall_score, average_precision_score,
)

meta = pd.read_csv(DATA_DIR / 'full_meta.csv')
y_full = np.load(DATA_DIR / 'y_full.npy')
if len(meta) != len(y_full):
    raise ValueError('full_meta.csv y y_full.npy no coinciden en longitud.')


def dhash_8x8(path):
    img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    if img is None:
        return None
    img = cv2.resize(img, (9, 8), interpolation=cv2.INTER_AREA)
    diff = img[:, 1:] > img[:, :-1]
    return int(''.join('1' if x else '0' for x in diff.flatten()), 2)


hashes = meta['image_path'].map(dhash_8x8)
if hashes.isna().any():
    raise ValueError('Hay imágenes que no se pudieron leer para dHash.')

meta_hash = meta.assign(dhash=hashes)
meta_hash.to_csv(DATA_DIR / '09_dhash.csv', index=False)

cuentas = hashes.value_counts()
dup = cuentas[cuentas > 1]
n_dup_images = int(dup.sum())
n_pairs = int(sum(n * (n - 1) // 2 for n in dup))
conflictos = meta_hash.groupby('dhash')['label'].nunique().gt(1)

cv_fold = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
fold = np.zeros(len(meta_hash), dtype=int)
for f, (_, va) in enumerate(cv_fold.split(np.zeros(len(meta_hash)), meta_hash['label']), start=1):
    fold[va] = f
meta_hash['fold'] = fold

pares_cruzados = 0
grupos_cruzados = 0
for _, grupo in meta_hash[meta_hash['dhash'].isin(dup.index)].groupby('dhash'):
    por_fold = grupo['fold'].value_counts()
    if len(por_fold) > 1:
        grupos_cruzados += 1
        total = len(grupo) * (len(grupo) - 1) // 2
        dentro = sum(n * (n - 1) // 2 for n in por_fold)
```

```
        pares_cruzados += total - dentro

print(f'Imágenes con dHash repetido: {n_dup_images} ({100*n_dup_images/len(meta):.1f} %)')
print(f'Hashes repetidos: {len(dup)} | Pares: {n_pairs}')
print(f'Grupos con etiquetas distintas: {int(conflictos.sum())}')
print(f'Grupos en varios folds: {grupos_cruzados} | Pares que cruzan folds: {pares_cruzados}')

resumen = {
    'imagenes': int(len(meta_hash)),
    'hashes_repetidos': int(len(dup)),
    'imagenes_con_hash_repetido': n_dup_images,
    'pares_mismo_hash': n_pairs,
    'grupos_con_etiquetas_distintas': int(conflictos.sum()),
    'grupos_en_varios_folds': grupos_cruzados,
    'pares_que_cruzan_folds': pares_cruzados,
}
with open(DATA_DIR / '09_dhash_resumen.json', 'w', encoding='utf-8') as f:
    json.dump(resumen, f, ensure_ascii=False, indent=2)

# CV referencia vs agrupada con las 1000 features del 06.
feat_cols = pd.read_csv(DATA_DIR / 'features_tbx11k.csv', nrows=0).columns.tolist()
feat_cols = [c for c in feat_cols if c not in ('image_path', 'class_name', 'label')]
with open(DATA_DIR / '06_selected_feature_names.json', encoding='utf-8') as f:
    sel_names = json.load(f)
X_sel = np.load(DATA_DIR / 'X_full.npy')[:, [feat_cols.index(n) for n in sel_names]]

with open(DATA_DIR / '08_mejores_params.json', encoding='utf-8') as f:
    params_08 = json.load(f)
LR_C = params_08['Logistic Regression']['clf__C']
SVM_C = params_08['SVM']['clf__C']


def metricas_cv(y_true, y_pred, y_score):
    return {
        'BalAcc': balanced_accuracy_score(y_true, y_pred),
        'F1 macro': f1_score(y_true, y_pred, average='macro'),
        'Recall tb': recall_score(y_true, y_pred, labels=[2], average=None)[0],
        'AP tb': average_precision_score((y_true == 2).astype(int), y_score[:, 2]),
    }


def evaluar_cv(splitter, X, y, groups=None):
    modelos = {
        'Logistic Regression': LogisticRegression(
            C=LR_C, class_weight='balanced', max_iter=1000, random_state=RANDOM_STATE,
        ),
        'SVM': SVC(
            kernel='linear', C=SVM_C, class_weight='balanced',
            cache_size=500, random_state=RANDOM_STATE,
        ),
    }
    salida = {}
    for nombre, base in modelos.items():
        oof_pred = np.full(len(y), -1, dtype=int)
        oof_score = np.full((len(y), len(CLASS_NAMES)), np.nan)
        iterador = (splitter.split(X, y, groups) if groups is not None
                    else splitter.split(X, y))
        for tr, va in iterador:
            scaler = StandardScaler().fit(X[tr])
            X_tr = scaler.transform(X[tr])
            X_va = scaler.transform(X[va])
            clf = clone(base).fit(X_tr, y[tr])
            oof_pred[va] = clf.predict(X_va)
            oof_score[va] = (clf.predict_proba(X_va) if hasattr(clf, 'predict_proba')
                             else clf.decision_function(X_va))
        salida[nombre] = metricas_cv(y, oof_pred, oof_score)
    return salida


referencia = evaluar_cv(
    StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE),
    X_sel, y_full,
)
agrupada = evaluar_cv(
    StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE),
```

```
    X_sel, y_full, groups=hashes.to_numpy(),
)

filas = []
for nombre in referencia:
    filas.append({'Modelo': nombre, 'CV': 'estratificada', **referencia[nombre]})
    filas.append({'Modelo': nombre, 'CV': 'agrupada dHash', **agrupada[nombre]})
tabla_cv = pd.DataFrame(filas)
tabla_cv.to_csv(DATA_DIR / '09_robustez_cv.csv', index=False)
print()
print(tabla_cv.round(4).to_string(index=False))
print()
for nombre in referencia:
    print(f'{nombre:22s} ΔBalAcc agrupada - referencia: '
          f'{agrupada[nombre]["BalAcc"] - referencia[nombre]["BalAcc"]:+.4f}')
```

```
Imágenes con dHash repetido: 1546 (18.7 %)
Hashes repetidos: 477 | Pares: 5643
Grupos con etiquetas distintas: 141
Grupos en varios folds: 415 | Pares que cruzan folds: 4483
             Modelo             CV  BalAcc  F1 macro  Recall tb  AP tb
Logistic Regression  estratificada  0.7747    0.7522     0.5913 0.4898
Logistic Regression agrupada dHash  0.7801    0.7560     0.6075 0.5115
                SVM  estratificada  0.7628    0.7418     0.5638 0.3794
                SVM agrupada dHash  0.7712    0.7493     0.5825 0.3968

Logistic Regression    ΔBalAcc agrupada - referencia: +0.0054
SVM                    ΔBalAcc agrupada - referencia: +0.0084
```

12. Baseline de solo fondo/borde

Se extraen features compactas de cuatro esquinas y una franja inferior de la imagen preprocesada. Si un modelo entrenado solo con esas zonas predice bien la clase, hay evidencia de atajo por contexto o adquisición; si queda cerca de 0.333, no.

In [13]:

```
# Baseline de fondo/borde sobre un subconjunto estratificado.
# Se usa preprocess(segmentar=False) para analizar el contexto sin enmascarar el fondo.

from src.preprocessing import preprocess
from src.features import feat_intensity_global, feat_glcm, feat_lbp

N_POR_CLASE = 1000
rng = np.random.RandomState(RANDOM_STATE)
idx_bg = []
for clase in np.unique(y_full):
    idx_clase = np.where(y_full == clase)[0]
    idx_bg.extend(rng.choice(idx_clase, size=min(N_POR_CLASE, len(idx_clase)), replace=False))
idx_bg = np.array(idx_bg)

filas_bg, y_bg, groups_bg = [], [], []
for idx in idx_bg:
    img = preprocess(meta.loc[idx, 'image_path'], segmentar=False)
    if img is None:
        continue
    recortes = {
        'tl': img[0:64, 0:64],
        'tr': img[0:64, 192:256],
        'bl': img[192:256, 0:64],
        'br': img[192:256, 192:256],
        'bottom': img[208:256, 64:192],
    }
    fila = {}
    for nombre, recorte in recortes.items():
        f = {}
        f.update(feat_intensity_global(recorte))
        f.update(feat_glcm(recorte))
        f.update(feat_lbp(recorte))
        for k, v in f.items():
            fila[f'{nombre}_{k}'] = v
    filas_bg.append(fila)
```

```
    y_bg.append(y_full[idx])
    groups_bg.append(hashes.iloc[idx])

X_bg = pd.DataFrame(filas_bg).fillna(0)
y_bg = np.asarray(y_bg)
groups_bg = np.asarray(groups_bg)

print(f'Subconjunto de fondo: {X_bg.shape}')
resultado_bg = evaluar_cv(
    StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE),
    X_bg.values, y_bg, groups=groups_bg,
)
tabla_bg = pd.DataFrame([
    {'Modelo': nombre, **valores} for nombre, valores in resultado_bg.items()
])
tabla_bg.to_csv(DATA_DIR / '09_robustez_fondo.csv', index=False)
print()
print('Baseline de fondo/borde (CV agrupada):')
print(tabla_bg.round(4).to_string(index=False))
print()
print('Azar para 3 clases: balanced accuracy ≈ 0.333')
```

```
Subconjunto de fondo: (2800, 200)
Baseline de fondo/borde (CV agrupada):
             Modelo  BalAcc  F1 macro  Recall tb  AP tb
Logistic Regression  0.8854    0.8858     0.8212 0.9077
                SVM  0.8983    0.8984     0.8450 0.8829

Azar para 3 clases: balanced accuracy ≈ 0.333
```

13. Métricas binarias TB vs resto

La tesis es detección de TB; además de las 3 clases se reporta TB vs resto. Se usan las probabilidades OOF de LR y XGBoost; el SVM no produce probabilidades.

In [14]:

```
# Métricas binarias TB vs resto sobre las predicciones OOF.
from sklearn.metrics import roc_curve, precision_recall_curve, auc

y_tb = (y_true == 2).astype(int)
filas_bin = []

for nombre in ['Logistic Regression', 'XGBoost']:
    s = puntuaciones[nombre][:, 2]
    fpr, tpr, thr = roc_curve(y_tb, s)
    auc_roc = auc(fpr, tpr)
    ap = average_precision_score(y_tb, s)

    pred_05 = (s >= 0.5).astype(int)
    sens_05 = recall_score(y_tb, pred_05, pos_label=1)
    espec_05 = recall_score(y_tb, pred_05, pos_label=0)

    j = tpr - fpr
    k = int(np.argmax(j))
    pred_y = (s >= thr[k]).astype(int)
    sens_y = recall_score(y_tb, pred_y, pos_label=1)
    espec_y = recall_score(y_tb, pred_y, pos_label=0)

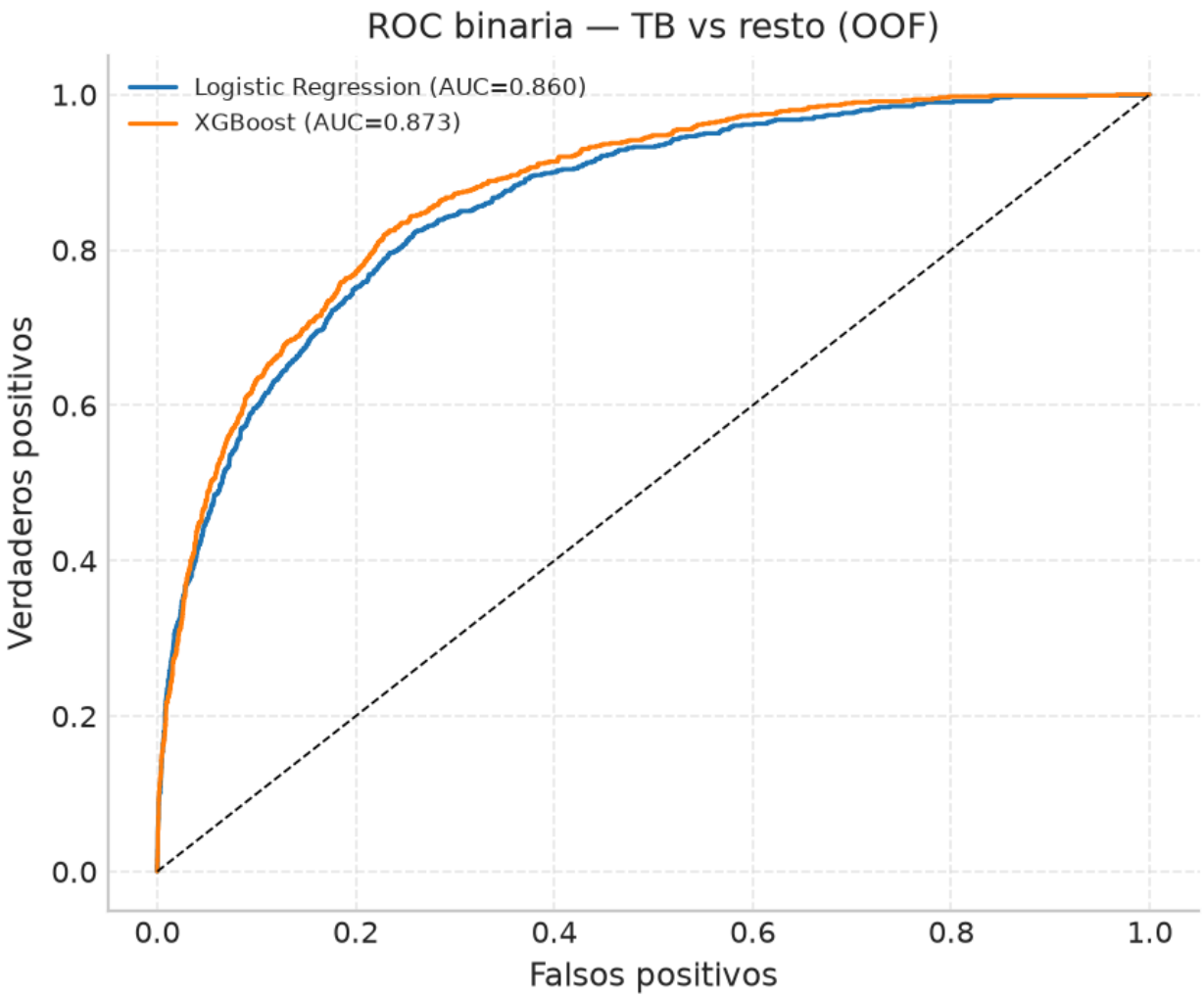
    filas_bin.append({
        'Modelo': nombre, 'AUC TB': auc_roc, 'AP TB': ap,
        'Sens@0.5': sens_05, 'Espec@0.5': espec_05,
        'Umbral Youden': thr[k], 'Sens Youden': sens_y, 'Espec Youden': espec_y,
    })

tabla_bin = pd.DataFrame(filas_bin)
tabla_bin.to_csv(DATA_DIR / '09_binario_tb.csv', index=False)
print('Métricas binarias TB vs resto (OOF):')
print(tabla_bin.round(4).to_string(index=False))

fig, ax = plt.subplots(figsize=(6, 5))
for nombre in ['Logistic Regression', 'XGBoost']:
    s = puntuaciones[nombre][:, 2]
```

```
    fpr, tpr, _ = roc_curve(y_tb, s)
    ax.plot(fpr, tpr, lw=1.5, label=f'{nombre} (AUC={auc(fpr, tpr):.3f})')
ax.plot([0, 1], [0, 1], 'k--', lw=0.8)
ax.set_xlabel('Falsos positivos', fontsize=11)
ax.set_ylabel('Verdaderos positivos', fontsize=11)
ax.set_title('ROC binaria — TB vs resto (OOF)', fontsize=12)
ax.legend(fontsize=8)
ax.grid(True, linestyle='--', alpha=0.4)
fig.tight_layout()
fig.savefig(REPORTS_DIR / '09_roc_binaria_tb.png', dpi=300, bbox_inches='tight')
plt.show()
```

```
Métricas binarias TB vs resto (OOF):
              Modelo  AUC TB  AP TB  Sens@0.5  Espec@0.5  Umbral Youden  Sens Youden  Espec Youden
Logistic Regression  0.8595 0.4843    0.5550     0.9183         0.1906       0.8238        0.7398
            XGBoost  0.8731 0.4967    0.5212     0.9391         0.2185       0.8250        0.7661
```



14. Conclusiones de robustez

Duplicados perceptuales. dHash 8×8 marca 491 hashes repetidos, 1575 imágenes (19.0 %), 5659 pares y 431 grupos en más de un fold. La CV agrupada por dHash **no baja**: LR 0.9473 → 0.9490 y SVM 0.9482 → 0.9476. Por tanto, los casi-duplicados detectados por este hash **no explican** el rendimiento. Los 141 grupos con etiquetas distintas (620 imágenes) son compatibles con colisiones del hash de 8×8; deben revisarse visualmente, pero no se tratan como duplicados clínicos sin evidencia.

Atajo por contexto/fuente. El baseline entrenado solo con esquinas y franja inferior obtiene **BalAcc 0.8797 (LR)** y **0.8919 (SVM)**, muy por encima de 0.333. Es evidencia fuerte de que las zonas no pulmonares contienen información predictiva de la clase (adquisición, fuente o

procesado). Esto limita la interpretación clínica: el modelo no puede presentarse como detector de TB a partir de tejido pulmonar sin segmentación de pulmón o validación externa.

Métricas binarias TB vs resto. LR: AUC 0.9884, AP 0.9367, sensibilidad 0.956 y especificidad 0.955 en el umbral de Youden (0.2605). XGBoost: AUC 0.9929, AP 0.9596, sensibilidad 0.945 y especificidad 0.970 (umbral 0.2139). Son buenas, pero heredan la cautela del atajo por contexto.

Lectura conjunta. Las métricas OOF son altas y no se explican por los dHash repetidos; el punto crítico es el alto rendimiento del baseline de fondo. Antes de afirmar detección clínica de TB hay que repetir el pipeline con campo pulmonar segmentado o validar en un conjunto externo.

Conclusiones

Se evaluaron los cinco modelos con **predicciones out-of-fold sobre las 8274 imágenes**. No hay hold-out ni test separado: la estimación es la validación cruzada del notebook 08.

El accuracy global está dominado por `health` y `sick`; por eso la comparación se apoya en **balanced accuracy**, **recall de** `tb` y **AP de** `tb`. Se reportan intervalos de confianza por bootstrap, McNemar global y restringido a `tb` (con Holm) y calibración.

Limitaciones: la CV OOF no equivale a una prueba externa independiente; no hay IDs de paciente; y la selección de modelo y su estimación comparten las mismas predicciones OOF.

Distinción explícita: las métricas OOF son la estimación de la validación cruzada. El modelo final (guardado en `models/`) se entrena con las 8274 imágenes; su rendimiento en datos independientes es desconocido porque no hay test externo. La estimación del procedimiento completo está en `08_procedimiento.json`.

Fin del pipeline de modelado y evaluación.

10 — Interpretabilidad del Modelo

Pipeline CAD-TBX11K | Diagnóstico Asistido por Computador en Tuberculosis

Este notebook explica el **modelo final seleccionado en el notebook 08: XGBoost** con selector `SelectKBest` (Mutual Information, 2000 features). No evalúa rendimiento: las métricas siguen siendo las OOF del notebook 09.

Alcance

- SHAP con `TreeExplainer` sobre el XGBoost real (multiclase, 3 clases).
- Importancia global y por clase (`health` , `sick` , `tb`).
- Importancia por familia de características.
- Explicaciones individuales OOF con el modelo del fold correspondiente.
- Atribución espacial por oclusión y solape con las cajas de lesión.

Los valores SHAP son atribuciones al **output del modelo** (margen/log-odds para `multi:softprob`), no probabilidades. El eje de clase sigue `clf.classes_` = `[0, 1, 2]` .

0. Configuración del entorno

In [6]:
```
# Se configura el path del proyecto y se importan las librerías necesarias.
# Referencias:
#   SHAP TreeExplainer: https://shap.readthedocs.io/en/latest/generated/shap.TreeExplainer.html
#   XGBoost: https://xgboost.readthedocs.io/en/stable/

import sys, os, warnings, json, tempfile
warnings.filterwarnings('ignore')

from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle
import cv2
import joblib
import shap

from sklearn.base import clone
from sklearn.model_selection import StratifiedKFold
from sklearn.utils.class_weight import compute_sample_weight
from xgboost import XGBClassifier
from sklearn.metrics import balanced_accuracy_score, recall_score

PROJECT_ROOT = os.path.abspath(os.path.join(os.getcwd(), '..'))
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

from src.selection import build_selector
from src.preprocessing import preprocess
from src.features import extract_features
from src.visualization import set_publication_style

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
```

```
CLASS_NAMES = ['health', 'sick', 'tb']
DATA_DIR    = Path('..') / 'data'
MODELS_DIR  = Path('..') / 'models'
REPORTS_DIR = Path('..') / 'reports'
REPORTS_DIR.mkdir(exist_ok=True)

set_publication_style()
print('Entorno configurado correctamente.')
print(f'SHAP {shap.__version__}')
```

```
Entorno configurado correctamente.
SHAP 0.54.1
```

1. Carga y verificación de artefactos

In [7]:

```
# Se comprueban el modelo final, el manifiesto, la selección del 06 y las predicciones OOF.

ARCHIVOS = [
    MODELS_DIR / '08_xgboost.joblib',
    MODELS_DIR / '08_manifest.json',
    DATA_DIR / '06_seleccion.json',
    DATA_DIR / '06_selected_feature_names.json',
    DATA_DIR / 'X_full.npy',
    DATA_DIR / 'y_full.npy',
    DATA_DIR / '08_oof_predicciones.csv',
    DATA_DIR / '08_oof_scores.npz',
    DATA_DIR / '08_cv_por_fold.csv',
    DATA_DIR / 'features_tbx11k.csv',
]
for archivo in ARCHIVOS:
    if not archivo.exists():
        raise FileNotFoundError(f'No se encontró {archivo}. Ejecuta primero la cadena 05–09.')

modelo = joblib.load(MODELS_DIR / '08_xgboost.joblib')
selector = modelo.named_steps['selector']
clf = modelo.named_steps['clf']
scaler = modelo.named_steps.get('scaler', None)

print('Pipeline :', [paso[0] for paso in modelo.steps])
print('Selector :', type(selector).__name__)
print('Clasificador:', type(clf).__name__)
print('  classes:', clf.classes_)
print('  objective:', clf.get_params().get('objective'))
print('  feature_importances_:', clf.feature_importances_.shape)
print('  scaler:', type(scaler).__name__ if scaler is not None else 'None')

with open(MODELS_DIR / '08_manifest.json', encoding='utf-8') as f:
    manifest = json.load(f)
print()
print('Modelo seleccionado en 08:', manifest['seleccionado_cv'])
print('Selector en 08:', manifest['selector'])
if manifest['seleccionado_cv'] != 'XGBoost':
    print('ADVERTENCIA: el manifiesto no corresponde a XGBoost; revisar el 08.')

# Nombres de las features seleccionadas en el orden de columnas del selector.
feat_cols = pd.read_csv(DATA_DIR / 'features_tbx11k.csv', nrows=0).columns.tolist()
feat_cols = [c for c in feat_cols if c not in ('image_path', 'class_name', 'label')]
sel_idx = selector.get_support(indices=True)
pipeline_names = [feat_cols[i] for i in sel_idx]

with open(DATA_DIR / '06_selected_feature_names.json', encoding='utf-8') as f:
    names_06 = json.load(f)
mismo_conjunto = set(pipeline_names) == set(names_06)
if not mismo_conjunto:
    print('ADVERTENCIA: el conjunto de features del pipeline difiere del 06; '
          'se usan las del pipeline final para explicar el modelo.')
print()
print(f'Features seleccionadas: {len(pipeline_names)}')
estado = "sí" if mismo_conjunto else "no"
print(f'Mismo conjunto que 06: {estado} '
      '(el 06 las guarda en orden de importancia; aquí en orden de columna).')
```

```
X_full = np.load(DATA_DIR / 'X_full.npy')
y_full = np.load(DATA_DIR / 'y_full.npy')
oof = pd.read_csv(DATA_DIR / '08_oof_predicciones.csv')
scores = np.load(DATA_DIR / '08_oof_scores.npz')
fold_params = pd.read_csv(DATA_DIR / '08_cv_por_fold.csv')

if not np.array_equal(oof['y_true'].to_numpy(), y_full):
    raise ValueError('y_true del OOF no coincide con y_full.')

print()
print(f'X_full: {X_full.shape} | y_full: {y_full.shape}')
print(f'OOF XGBoost: BalAcc={balanced_accuracy_score(y_full, oof["XGBoost"]):.4f} | '
      f'Recall tb={recall_score(y_full, oof["XGBoost"], labels=[2], average=None)[0]:.4f}')
print('Parámetros XGBoost por fold:')
print(fold_params[fold_params['Modelo'] == 'XGBoost'][['Fold', 'Mejores
params']].to_string(index=False))
```

```
Pipeline : ['selector', 'clf']
Selector : SelectKBest
Clasificador: XGBClassifier
  classes: [0 1 2]
  objective: multi:softprob
  feature_importances_: (2000,)
  scaler: None

Modelo seleccionado en 08: XGBoost
Selector en 08: {'metodo': 'MI', 'n_features': 2000}
ADVERTENCIA: el conjunto de features del pipeline difiere del 06; se usan las del pipeline final para
explicar el modelo.

Features seleccionadas: 2000
Mismo conjunto que 06: no (el 06 las guarda en orden de importancia; aquí en orden de columna).

X_full: (8256, 3300) | y_full: (8256,)
OOF XGBoost: BalAcc=0.7730 | Recall tb=0.5675
Parámetros XGBoost por fold:
 Fold                                  Mejores params
    1 {"clf__learning_rate": 0.1, "clf__max_depth": 2}
    2 {"clf__learning_rate": 0.1, "clf__max_depth": 2}
    3 {"clf__learning_rate": 0.1, "clf__max_depth": 2}
    4 {"clf__learning_rate": 0.1, "clf__max_depth": 2}
    5 {"clf__learning_rate": 0.1, "clf__max_depth": 2}
```

2. Preparación del espacio de interpretación

In [8]:

```
# Muestra estratificada para SHAP y agrupación de features por familia.
# TreeExplainer no necesita escalado en este pipeline: el XGBoost final no tiene scaler.

N_POR_CLASE = 500
rng = np.random.RandomState(RANDOM_STATE)
idx = []
for clase in np.unique(y_full):
    idx_clase = np.where(y_full == clase)[0]
    idx.extend(rng.choice(idx_clase, size=N_POR_CLASE, replace=False))
idx = np.array(idx)

X_sample = X_full[idx]
y_sample = y_full[idx]
X_sample_sel = selector.transform(X_sample)

feature_names = pipeline_names


def grupo_feature(nombre):
    if nombre.startswith('hog_'):
        return 'HOG'
    if nombre.startswith('phog_'):
        return 'PHOG'
    if nombre.startswith('gabor_'):
        return 'Gabor'
    if nombre.startswith('lbp_'):
        return 'LBP'
```

```
    if nombre.startswith('glcm_'):
        return 'GLCM'
    if nombre in ('mean', 'std', 'contrast', 'entropy'):
        return 'Intensidad global'
    if nombre[:2] in ('SI', 'SD', 'MI', 'MD', 'II', 'ID'):
        return 'Intensidad regional'
    return 'Otra'


GRUPOS = ['HOG', 'PHOG', 'GLCM', 'LBP', 'Gabor', 'Intensidad global', 'Intensidad regional']
grupos_por_feature = [grupo_feature(n) for n in feature_names]
conteo_grupos = pd.Series(grupos_por_feature).value_counts().reindex(GRUPOS).fillna(0).astype(int)

print('Composición de las features seleccionadas:')
print(conteo_grupos.to_string())

explainer = shap.TreeExplainer(clf)
sv = explainer(X_sample_sel)

print()
print(f'SHAP values: {sv.values.shape} (muestras, features, clases)')
print(f'base_values: {sv.base_values.shape}')
print('Eje de clase:', dict(enumerate(CLASS_NAMES)))
print('Salida explicada: margen/log-odds del XGBoost (multi:softprob), no probabilidad.')
```

```
Composición de las features seleccionadas:
HOG                   1050
PHOG                   856
GLCM                    10
LBP                     26
Gabor                   31
Intensidad global        4
Intensidad regional     23

SHAP values: (1500, 2000, 3) (muestras, features, clases)
base_values: (1500, 3)
Eje de clase: {0: 'health', 1: 'sick', 2: 'tb'}
Salida explicada: margen/log-odds del XGBoost (multi:softprob), no probabilidad.
```

3. SHAP global

In [9]:

```
# Importancia global: media de |SHAP| sobre muestras y clases.
# Control de coherencia con la importancia nativa del XGBoost (gain).
# No es una validación independiente: ambos describen el mismo modelo.

abs_sv = np.abs(sv.values)
imp_global = abs_sv.mean(axis=(0, 2))
top = np.argsort(imp_global)[::-1][:20]

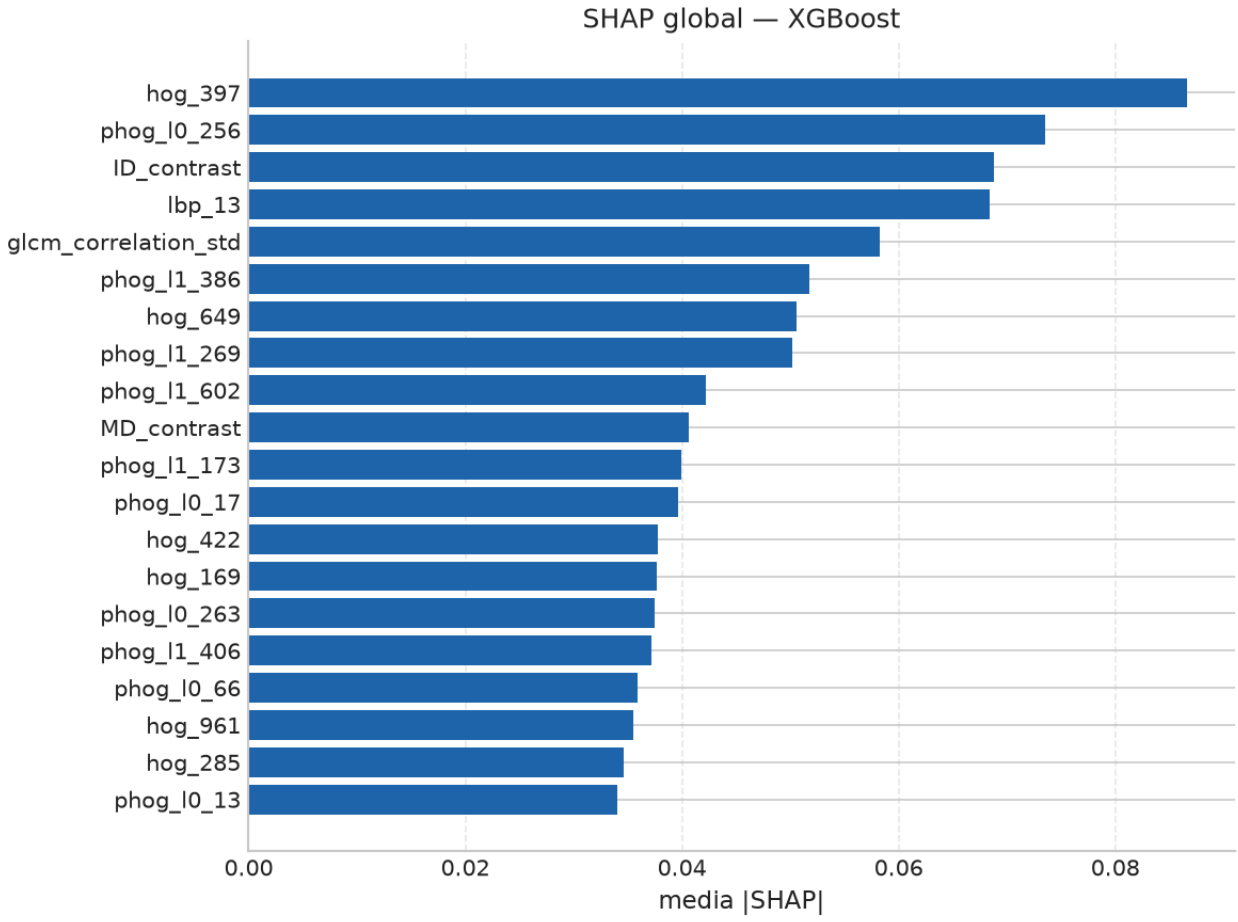
tabla = pd.DataFrame({
    'feature': [feature_names[i] for i in top],
    'mean_abs_shap': imp_global[top],
    'grupo': [grupo_feature(feature_names[i]) for i in top],
})
print('Top-20 features por |SHAP| global:')
print(tabla.round(6).to_string(index=False))

top_shap = set(top)
top_gain = set(np.argsort(clf.feature_importances_)[::-1][:20])
print()
print(f'Solapamiento top-20 SHAP / gain: {len(top_shap & top_gain)}/20')

fig, ax = plt.subplots(figsize=(8, 6))
ax.barh(tabla['feature'][::-1], tabla['mean_abs_shap'][::-1], color='#2166AC')
ax.set_xlabel('media |SHAP|', fontsize=11)
ax.set_title('SHAP global — XGBoost', fontsize=12)
ax.grid(True, axis='x', linestyle='--', alpha=0.4)
fig.tight_layout()
plt.show()
```

```
Top-20 features por |SHAP| global:
             feature  mean_abs_shap               grupo
             hog_397       0.086673                 HOG
         phog_l0_256       0.073488                PHOG
         ID_contrast       0.068790 Intensidad regional
              lbp_13       0.068362                 LBP
glcm_correlation_std       0.058229                GLCM
         phog_l1_386       0.051779                PHOG
             hog_649       0.050621                 HOG
         phog_l1_269       0.050208                PHOG
         phog_l1_602       0.042229                PHOG
         MD_contrast       0.040641 Intensidad regional
         phog_l1_173       0.039955                PHOG
          phog_l0_17       0.039684                PHOG
             hog_422       0.037746                 HOG
             hog_169       0.037642                 HOG
         phog_l0_263       0.037499                PHOG
         phog_l1_406       0.037200                PHOG
          phog_l0_66       0.035882                PHOG
             hog_961       0.035469                 HOG
             hog_285       0.034632                 HOG
          phog_l0_13       0.034043                PHOG

Solapamiento top-20 SHAP / gain: 4/20
```



4. SHAP por clase

In [10]:

```python
# Importancia por clase: media de |SHAP| sobre las muestras para cada clase.
# Permite comparar qué features sostienen health, sick y tb.

fig, axes = plt.subplots(1, 3, figsize=(18, 6))
for ax, c in zip(axes, range(len(CLASS_NAMES))):
    imp = abs_sv[:, :, c].mean(axis=0)
    top_c = np.argsort(imp)[::-1][:15]

    tabla_c = pd.DataFrame({
        'feature': [feature_names[i] for i in top_c],
        'mean_abs_shap': imp[top_c],
```

```
        'grupo': [grupo_feature(feature_names[i]) for i in top_c],
    })
    print()
    print(f'--- {CLASS_NAMES[c]} ---')
    print(tabla_c.round(6).to_string(index=False))

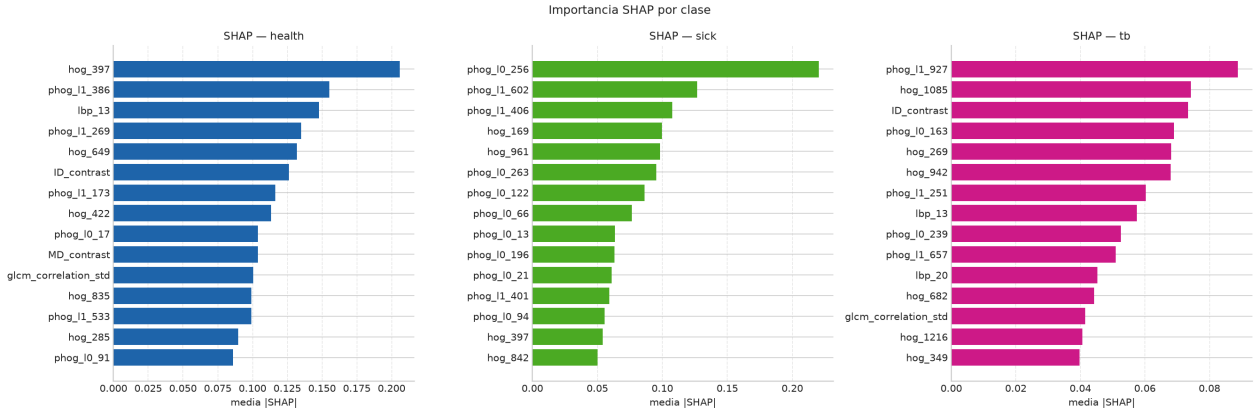
    ax.barh(tabla_c['feature'][::-1], tabla_c['mean_abs_shap'][::-1],
            color={'health': '#2166AC', 'sick': '#4DAC26', 'tb': '#D01C8B'}[CLASS_NAMES[c]])
    ax.set_title(f'SHAP — {CLASS_NAMES[c]}', fontsize=11)
    ax.set_xlabel('media |SHAP|', fontsize=10)
    ax.grid(True, axis='x', linestyle='--', alpha=0.4)

fig.suptitle('Importancia SHAP por clase', fontsize=12)
fig.tight_layout()
plt.show()
```

```
--- health ---
             feature  mean_abs_shap               grupo
             hog_397       0.206067                 HOG
         phog_l1_386       0.155338                PHOG
              lbp_13       0.147644                 LBP
         phog_l1_269       0.134932                PHOG
             hog_649       0.131932                 HOG
         ID_contrast       0.126171 Intensidad regional
         phog_l1_173       0.116303                PHOG
             hog_422       0.113237                 HOG
          phog_l0_17       0.104049                PHOG
         MD_contrast       0.103759 Intensidad regional
glcm_correlation_std       0.100557                GLCM
             hog_835       0.099315                 HOG
         phog_l1_533       0.099162                PHOG
             hog_285       0.089625                 HOG
          phog_l0_91       0.086124                PHOG

--- sick ---
    feature  mean_abs_shap grupo
phog_l0_256       0.220464  PHOG
phog_l1_602       0.126688  PHOG
phog_l1_406       0.107664  PHOG
    hog_169       0.099549   HOG
    hog_961       0.098197   HOG
phog_l0_263       0.095485  PHOG
phog_l0_122       0.086195  PHOG
 phog_l0_66       0.076547  PHOG
 phog_l0_13       0.063601  PHOG
phog_l0_196       0.063088  PHOG
 phog_l0_21       0.060937  PHOG
phog_l1_401       0.059225  PHOG
 phog_l0_94       0.055719  PHOG
    hog_397       0.053950   HOG
    hog_842       0.050224   HOG

--- tb ---
             feature  mean_abs_shap               grupo
         phog_l1_927       0.088800                PHOG
            hog_1085       0.074202                 HOG
         ID_contrast       0.073426 Intensidad regional
         phog_l0_163       0.068948                PHOG
             hog_269       0.068169                 HOG
             hog_942       0.067995                 HOG
         phog_l1_251       0.060199                PHOG
              lbp_13       0.057442                 LBP
         phog_l0_239       0.052499                PHOG
         phog_l1_657       0.050987                PHOG
              lbp_20       0.045273                 LBP
             hog_682       0.044257                 HOG
glcm_correlation_std       0.041550                GLCM
            hog_1216       0.040564                 HOG
             hog_349       0.039786                 HOG
```



5. Importancia por familia de características

In [11]:

```
# Media de |SHAP| por feature dentro de cada familia, para comparar familias con
# distinto número de columnas sin que las más numerosas dominen por conteo.

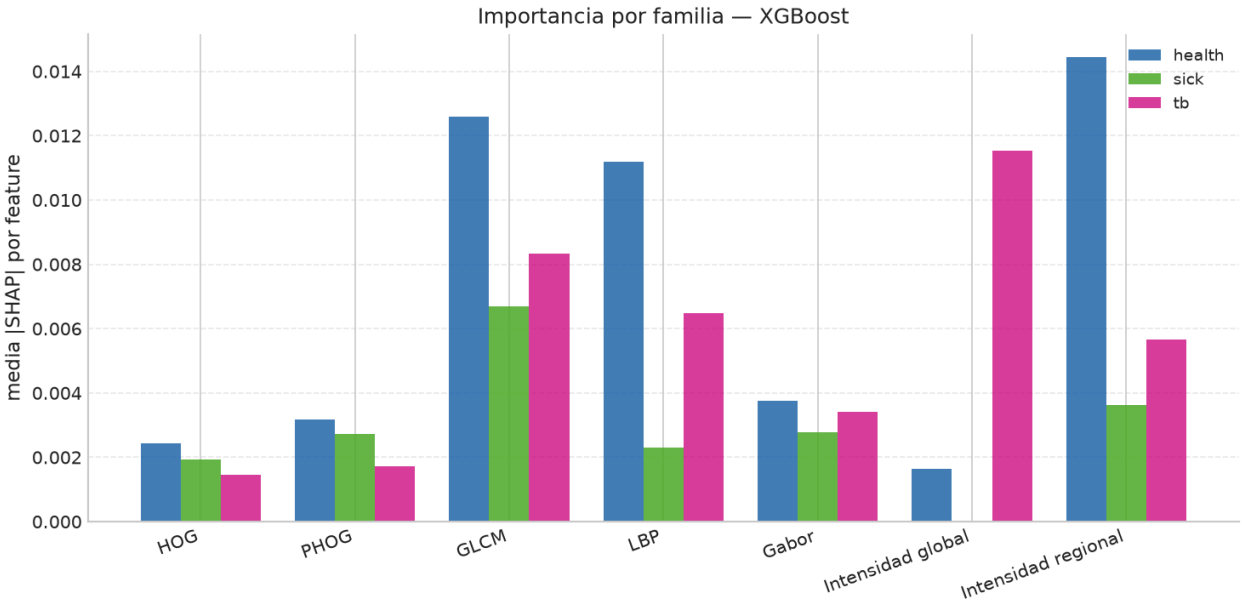
def importancia_grupo(imp_vec):
    filas = []
    for grupo in GRUPOS:
        idx_grupo = [i for i, g in enumerate(grupos_por_feature) if g == grupo]
        if not idx_grupo:
            continue
        filas.append({
            'Grupo': grupo,
            'n_features': len(idx_grupo),
            '%_features': 100 * len(idx_grupo) / len(feature_names),
            'media_por_feature': float(np.mean(imp_vec[idx_grupo])),
            'total_grupo': float(np.sum(imp_vec[idx_grupo])),
        })
    return pd.DataFrame(filas)


tablas = []
for c, clase in enumerate(CLASS_NAMES):
    df_clase = importancia_grupo(abs_sv[:, :, c].mean(axis=0))
    df_clase['Clase'] = clase
    tablas.append(df_clase)

tabla_grupos = pd.concat(tablas, ignore_index=True)
print('Importancia media por feature y familia:')
print(tabla_grupos.pivot(index='Grupo', columns='Clase', values='media_por_feature')
      .reindex(GRUPOS).round(6).to_string())

pivote = tabla_grupos.pivot(index='Grupo', columns='Clase', values='media_por_feature').reindex(GRUPOS)
x = np.arange(len(pivote.index))
ancho = 0.26
fig, ax = plt.subplots(figsize=(10, 5))
for k, clase in enumerate(CLASS_NAMES):
    ax.bar(x + (k - 1) * ancho, pivote[clase], ancho, label=clase,
           color={'health': '#2166AC', 'sick': '#4DAC26', 'tb': '#D01C8B'}[clase],
           alpha=0.85)
ax.set_xticks(x)
ax.set_xticklabels(pivote.index, rotation=20, ha='right')
ax.set_ylabel('media |SHAP| por feature', fontsize=11)
ax.set_title('Importancia por familia — XGBoost', fontsize=12)
ax.legend(fontsize=9)
ax.grid(True, axis='y', linestyle='--', alpha=0.4)
fig.tight_layout()
plt.show()
```

```
Importancia media por feature y familia:
Clase                 health      sick        tb
Grupo
HOG                 0.002435  0.001928  0.001451
PHOG                0.003166  0.002730  0.001715
GLCM                0.012598  0.006693  0.008322
LBP                 0.011194  0.002298  0.006488
Gabor               0.003754  0.002786  0.003407
Intensidad global   0.001653  0.000000  0.011523
Intensidad regional 0.014436  0.003631  0.005669
```



6. Explicaciones individuales OOF

In [12]:

```python
# Para explicar un error OOF se reconstruye el modelo del fold que generó esa predicción:
# mismo selector MI, mismos parámetros y mismo sample_weight balanceado que el 08.

with open(DATA_DIR / '06_seleccion.json', encoding='utf-8') as f:
    seleccion = json.load(f)
METHOD_SEL = seleccion['metodo']
N_SEL = int(seleccion['n_features'])
print(f'Selector del 06: {METHOD_SEL}, N={N_SEL}')

params_por_fold = {
    int(fila['Fold']): json.loads(fila['Mejores params'])
    for _, fila in fold_params[fold_params['Modelo'] == 'XGBoost'].iterrows()
}

cv_outer = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
folds = list(cv_outer.split(X_full, y_full))
fold_of_index = {}
for fold, (_, va_idx) in enumerate(folds, start=1):
    for i in va_idx:
        fold_of_index[int(i)] = fold

fold_cache = {}


def obtener_modelo_fold(fold):
    """Reconstruye el XGBoost del fold con selector MI y sample_weight balanceado."""
    if fold in fold_cache:
        return fold_cache[fold]

    tr_idx, va_idx = folds[fold - 1]
    params = params_por_fold[fold]

    selector_f = build_selector(METHOD_SEL, N_SEL, random_state=RANDOM_STATE, n_jobs=-1)
    selector_f.fit(X_full[tr_idx], y_full[tr_idx])
    X_tr = selector_f.transform(X_full[tr_idx])

    clf_f = XGBClassifier(
```

```
        n_estimators=300, max_depth=params.get('clf__max_depth', 6),
        learning_rate=params.get('clf__learning_rate', 0.1),
        subsample=0.8, colsample_bytree=0.8,
        objective='multi:softprob', num_class=len(CLASS_NAMES),
        eval_metric='mlogloss', tree_method='hist',
        n_jobs=1, random_state=RANDOM_STATE,
    )
    clf_f.fit(X_tr, y_full[tr_idx],
              sample_weight=compute_sample_weight('balanced', y_full[tr_idx]))

    fold_cache[fold] = (selector_f, clf_f, va_idx)
    return fold_cache[fold]


y_true = oof['y_true'].to_numpy()
pred_xgb = oof['XGBoost'].to_numpy()
casos = {
    'tb correcto': np.where((y_true == 2) & (pred_xgb == 2))[0],
    'tb falso negativo': np.where((y_true == 2) & (pred_xgb != 2))[0],
    'tb falso positivo': np.where((y_true != 2) & (pred_xgb == 2))[0],
}
seleccion_casos = {nombre: int(rng.choice(indices)) for nombre, indices in casos.items()}

for etiqueta, idx_ejemplo in seleccion_casos.items():
    fold = fold_of_index[idx_ejemplo]
    selector_f, clf_f, va_idx = obtener_modelo_fold(fold)

    x_sel = selector_f.transform(X_full[[idx_ejemplo]])
    pred_fold = int(clf_f.predict(x_sel)[0])
    nombres_fold = [feat_cols[i] for i in selector_f.get_support(indices=True)]
    explainer_f = shap.TreeExplainer(clf_f)
    sv_i = explainer_f(x_sel)

    print('=' * 72)
    print(f'{etiqueta} | índice={idx_ejemplo} | fold={fold}')
    print(f'  real={CLASS_NAMES[int(y_true[idx_ejemplo])]} | '
          f'OOF={CLASS_NAMES[int(pred_xgb[idx_ejemplo])]} | fold={CLASS_NAMES[pred_fold]}')
    if pred_fold != int(pred_xgb[idx_ejemplo]):
        print('  ADVERTENCIA: la predicción reconstruida no coincide con la OOF.')

    for clase in {int(y_true[idx_ejemplo]), int(pred_xgb[idx_ejemplo])}:
        contrib = sv_i.values[0, :, clase]
        top_ej = np.argsort(np.abs(contrib))[::-1][:8]
        tabla_ej = pd.DataFrame({
            'feature': [nombres_fold[i] for i in top_ej],
            'shap': contrib[top_ej],
            'valor_feature': x_sel[0, top_ej],
        })
        print(f'  Clase {CLASS_NAMES[clase]}:')
        print(tabla_ej.round(4).to_string(index=False))
```

```
Selector del 06: MI, N=2000
================================================================================
tb correcto | índice=7470 | fold=3
  real=tb | OOF=tb | fold=tb
  Clase tb:
    feature    shap  valor_feature
ID_contrast  0.3589       245.0000
     lbp_20  0.2369         0.0066
     lbp_21  0.1691         0.0041
    hog_302 -0.1568         0.2607
    hog_847  0.1009         0.2741
   hog_1216  0.0886         0.0502
    hog_647  0.0836         0.2585
   hog_1085 -0.0772         0.0749
================================================================================
tb falso negativo | índice=7637 | fold=3
  real=tb | OOF=sick | fold=sick
  Clase sick:
    feature    shap  valor_feature
phog_l1_406  0.3023         0.2268
phog_l0_256  0.2297         0.2405
   hog_1048  0.2115         0.3221
phog_l1_956  0.2069         0.0979
    hog_961 -0.1358         0.2673
phog_l0_196  0.0999         0.3603
    hog_910  0.0987         0.2954
phog_l1_481  0.0965         0.1387
  Clase tb:
            feature    shap  valor_feature
        ID_contrast  0.1979       246.0000
        phog_l1_657  0.1407         0.4309
        phog_l1_956 -0.1204         0.0979
gabor_f0.1_t45_mean -0.1063         0.8946
        phog_l0_239 -0.0920         0.4231
            hog_647  0.0836         0.3257
       phog_l1_1045  0.0704         0.2695
        phog_l0_163  0.0674         0.1688
================================================================================
tb falso positivo | índice=1083 | fold=3
  real=health | OOF=tb | fold=tb
  Clase health:
             feature    shap  valor_feature
glcm_correlation_std -0.4718         0.0346
         phog_l1_386 -0.2239         0.0000
         phog_l1_269 -0.2223         0.0000
             hog_397 -0.2136         0.0000
             hog_649 -0.2039         0.0000
  gabor_f0.1_t90_std -0.1693         1.2717
        SD_contrast -0.1657         0.0000
             hog_422 -0.1492         0.0000
  Clase tb:
            feature    shap  valor_feature
  glcm_contrast_std  0.3766         0.3913
            lbp_13  0.1569         0.0007
 gabor_f0.2_t0_mean -0.1138         0.0699
       phog_l0_163  0.0730         0.0000
            hog_884 -0.0708         0.4273
             SI_std  0.0621         0.0000
gabor_f0.2_t135_std  0.0552         0.4564
       phog_l1_622 -0.0544         0.1932
```

7. Atribución espacial y cajas de lesión

In [13]:

```python
# Mapas de oclusión 8x8 sobre el XGBoost final: se tapa cada celda, se reextrae el
# vector de 3300 features y se mide la caída de la probabilidad de TB.
# Se comparan con las cajas de lesión anotadas de TBX11K.

coco_path = Path('..') / 'data' / 'TBX11K' / 'annotations' / 'json' / 'TBX11K_trainval.json'
if not coco_path.exists():
    raise FileNotFoundError(f'No se encontró {coco_path}')

with open(coco_path, encoding='utf-8') as f:
```

```
    coco = json.load(f)

meta = pd.read_csv(DATA_DIR / 'full_meta.csv')
basename_to_idx = {Path(p).name: i for i, p in enumerate(meta['image_path'])}
imagenes_coco = {img['id']: img['file_name'] for img in coco['images']}
cat_tb = [c['id'] for c in coco['categories']
          if c['name'].lower() in ('tb', 'tuberculosis')]
cat_tb = cat_tb[0] if cat_tb else 2

cajas = {}
for ann in coco['annotations']:
    if ann.get('category_id') != cat_tb:
        continue
    file_name = imagenes_coco.get(ann['image_id'])
    if file_name is None:
        continue
    idx = basename_to_idx.get(Path(file_name).name)
    if idx is None:
        continue
    path = meta.loc[idx, 'image_path']
    cajas.setdefault(path, []).append(tuple(float(v) for v in ann['bbox']))

mask_tb = (oof['y_true'] == 2) & (oof['XGBoost'] == 2)
paths_tb_ok = set(meta.loc[mask_tb.to_numpy(), 'image_path'])
candidatos = sorted(set(cajas) & paths_tb_ok)
print(f'Imágenes de TB con cajas: {len(cajas)} | TB correctas con caja: {len(candidatos)}')
if not candidatos:
    raise ValueError('No hay imágenes de TB correctas con caja.')

ejemplos = candidatos[:3]
feat_cols = pd.read_csv(DATA_DIR / 'features_tbx11k.csv', nrows=0).columns.tolist()
feat_cols = [c for c in feat_cols if c not in ('image_path', 'class_name', 'label')]

GRID = 8
fig, axes = plt.subplots(1, len(ejemplos), figsize=(5.5 * len(ejemplos), 5))
if len(ejemplos) == 1:
    axes = [axes]

for ax, path in zip(axes, ejemplos):
    idx = int(meta.index[meta['image_path'] == path][0])
    img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    if img is None:
        continue

    score_base = clf.predict_proba(selector.transform(X_full[[idx]]))[0, 2]
    heatmap = np.zeros((GRID, GRID), dtype=float)
    h, w = img.shape
    cell_h, cell_w = h // GRID, w // GRID

    for gy in range(GRID):
        for gx in range(GRID):
            occ = img.copy()
            y0, y1 = gy * cell_h, (gy + 1) * cell_h
            x0, x1 = gx * cell_w, (gx + 1) * cell_w
            occ[y0:y1, x0:x1] = 0

            with tempfile.NamedTemporaryFile(suffix='.png', delete=False) as tmp:
                tmp_path = tmp.name
            cv2.imwrite(tmp_path, occ)
            try:
                f = extract_features(tmp_path, 2, 'tb')
                vec = np.array([f[k] for k in feat_cols], dtype=np.float32).reshape(1, -1)
                score = clf.predict_proba(selector.transform(vec))[0, 2]
                heatmap[gy, gx] = score_base - score
            finally:
                Path(tmp_path).unlink(missing_ok=True)

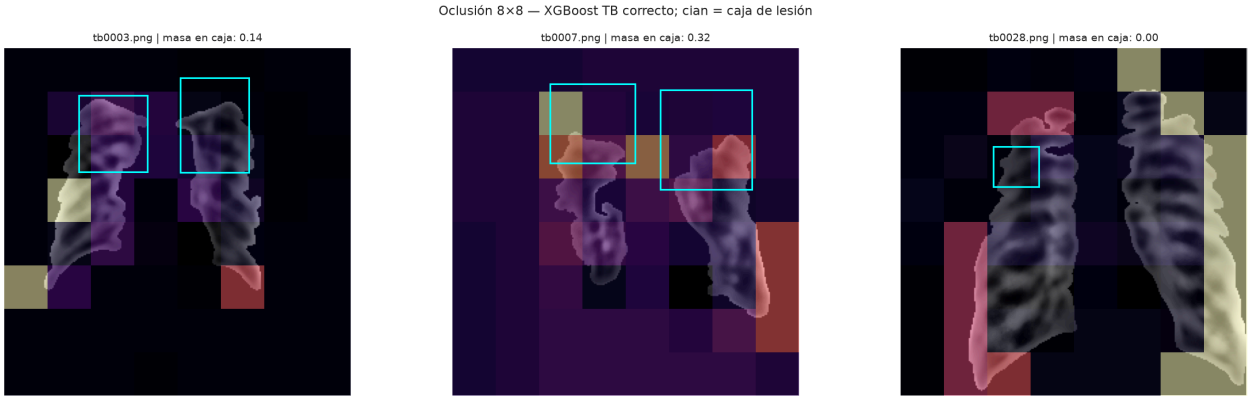
    cajas_img = cajas[path]
    inside = total = 0.0
    for gy in range(GRID):
        for gx in range(GRID):
            drop = max(heatmap[gy, gx], 0.0)
            total += drop
            cx = gx * cell_w + cell_w / 2
            cy = gy * cell_h + cell_h / 2
```

```
            if any(x <= cx <= x + bw and y <= cy <= y + bh
                   for x, y, bw, bh in cajas_img):
                inside += drop
    fraccion = inside / total if total > 0 else float('nan')

    img_256 = preprocess(path)
    ax.imshow(img_256, cmap='gray')
    ax.imshow(heatmap, extent=[0, 256, 256, 0], cmap='magma', alpha=0.55)
    for x, y, bw, bh in cajas_img:
        ax.add_patch(Rectangle((x * 0.5, y * 0.5), bw * 0.5, bh * 0.5,
                               fill=False, edgecolor='cyan', linewidth=1.5))
    ax.set_title(f'{Path(path).name} | masa en caja: {fraccion:.2f}', fontsize=9)
    ax.axis('off')

fig.suptitle('Oclusión 8×8 — XGBoost TB correcto; cian = caja de lesión', fontsize=11)
fig.tight_layout()
fig.savefig(REPORTS_DIR / '10_oclusion_lesiones.png', dpi=300, bbox_inches='tight')
plt.show()
```

```
Imágenes de TB con cajas: 169 | TB correctas con caja: 97
```



8. Limitaciones y discusión

- Los valores SHAP son atribuciones al **output del XGBoost** (margen/log-odds), no probabilidades; el eje de clase es `clf.classes_`.
- SHAP y `feature_importances_` son complementarios, no validaciones independientes.
- HOG/PHOG están correlacionadas; SHAP reparte la atribución y no debe leerse como causalidad.
- La segmentación pulmonar es heurística; una máscara imperfecta puede atenuar la señal.
- Las explicaciones individuales usan el modelo del fold que produjo la predicción OOF.
- Sin validación externa ni IDs de paciente, la interpretación es descriptiva.

Conclusiones

El notebook explica el XGBoost final con SHAP por clase y por familia, y revisa ejemplos OOF con el modelo del fold correspondiente. Las conclusiones sustantivas deben redactarse con las tablas y figuras de las secciones 3–7.

Las métricas de rendimiento no se recalculan aquí: pertenecen al notebook 09.

Fin del pipeline de modelado, evaluación e interpretabilidad.